MDFourier

Artemio Urbina

October 31, 2026

Abstract

MDFourier is an *open source* software solution created to compare *audio signatures* and generate a series of graphs that show how they differ. The software consists of two separate programs, one that produces the signal for recording from the console and the other that analyses and displays the audio comparisons.

The information gathered from the comparison results can be used in a variety of ways: to identify how *audio signatures* vary between *systems*, to detect if the *audio signals* are modified by *audio equipment*, to find if *modifications* resulted in audible changes, to help tune *emulators*, *FPGA* implementations or *mods*, etc.

This document serves as an introduction, a manual and a description of the methods used and how they work. Its intent is to guide new users in making simple comparisons while still providing advanced users with information to perform a more detailed analysis of audio signatures.

Contents

1 Introduction **7**

1.1 Possible applications 8

1.2 Disclaimer 10

2 Audio Basics **11**

2.1 Sound 11

2.2 Frequency 11

2.3 Amplitude 12

2.4 Digital representations of sound 14

2.4.1 Sample rate 15

2.4.2 Bit depth 15

2.4.3 Decibels in the digital domain 16

3 Working with *MDFourier* **18**

3.1 Workflow 19

3.2 How *MDFourier* works 19

3.3 File alignment 20

3.4 The heart of the process 20

3.5 Minimal significant amplitude 22

4 How to interpret the graphs **24**

4.1 Scenario 1: Comparing the same file against itself 25

4.2 Scenario 2: Comparing two different recordings from the same console 26

4.3 Scenario 3: Comparing against a modified file 28

4.4 Scenario 4: Comparing against digital low pass and high pass filters 29

4.5 Scenario 5: Comparing two recordings from the same console made with different Audio Interfaces 34

4.6 Scenario 6: Comparing *WAV* vs *MP3* 36

5 Results from vintage retail hardware 40

5.1 Sega Genesis Model 1 VA3 (US) vs Mega Drive Model 1 VA1 (JP) 41

5.2 Sega Genesis Model 1 VA3 (US): Headphone out vs AV out 42

5.3 Sega Genesis Model 1 VA3 (US) vs Sega Genesis Model 1 VA6 (JP) 43

5.4 Sega Genesis Model 1 VA6 (JP) vs Mega Drive Model 1 VA6 (US) 44

5.5 Sega Genesis Model 1 VA6 (US) vs Sega Genesis Model 1 VA6 (US) 45

5.6 Sega Genesis Model 1 VA3 (US) vs Sega Genesis Model 2 VA1.8 (US) 46

5.7 Sega Genesis Model 1 VA3 (US) vs Sega Nomad (US) 48

5.8 Sega Genesis Model 2 VA1.8 (US) vs Sega Nomad (US) 49

5.9 Sega Genesis Model 1 VA3 (US) vs Sega CDX (US) 50

5.10 Sega CDX (US) Line-out vs Sega CDX (US) Headphone port 52

5.11 Sega Genesis Model 1 VA3 w/Sega CD (US) vs Mega Drive Model 1 VA6 w/Sega CD (JP) 52

5.12 Mega Drive Model 1 VA6 w/Sega CD (JP) vs Mega Drive Model 1 VA6 w/Sega CD (US) 53

5.13 Sega Genesis Model 1 VA3 w/Sega CD (US) vs Mega Drive Model 1 VA6 w/Sega CD (US) 54

5.14 Sega Genesis Model 1 VA3 w/Sega CD (US) vs Mega Drive Model 1 VA6 w/Sega CD (US) 55

6 Conclusions **57**

Appendices **58**

A Downloads **59**

B How to use the *Analysis Software* **60**

B.1 Analysis Software Options 61

B.1.1 Window Functions 62

B.1.2 Color Filter Functions 62

B.1.3 Align Transform to 1Hz 63

B.1.4 Output Graphs 64

B.1.5 Extra Command 68

B.1.6 Verbose Log 70

C Configuration file **71**

C.1 Header and name 72

C.2 Video modes 72

C.3 Clocks 72

C.4 Mono balance block 73

C.5 Tone lines 73

C.6 No Sync profiles 76

D Clock measurement and adjustment **77**

D.1 Clock lines 79

D.2 Measurement 80

D.3 Identification 81

D.4 Adjusting clocks with *-j* 81

D.5 Optional clocks 82
D.6 Clock data in the graphs 82

E MDWave **83**

F Normalization and amplitude matching **85**

F.1 Frequency domain normalization 86
F.2 Time domain normalization 86
F.3 Highest fundamental average normalization 87

G Frame rates and their importance **88**

H Channel separation **91**

I Noise compared by bands **93**

J Stereo Audio Balancing **95**

J.1 Possible causes 95

K Differences due to temperature **97**

L Corner cases **99**

M Known Issues **102**

M.1 *MS-DOS* under *Windows* 102

N Window Function equations and graphs **104**

N.1 Tukey 104
N.2 Hann 105
N.3 Flattop 106
N.4 Hamming 106
N.5 No Window 107

O Color Filter Function details **108**

O.1 None . . . 108

O.2 Bright . . . 109

O.3 High . . . 110

O.4 Neutral . . . 111

O.5 Low . . . 112

O.6 Dimm . . . 113

P Requirements **115**

P.1 Audio capture device . . . 115

P.1.1 Sampling clocks . . . 116

P.2 Computer . . . 116

P.3 Game Consoles or emulators . . . 116

P.4 Flash cart, or means to run the *Tone Generator* . . . 117

P.5 Cables and adapters . . . 117

P.6 Audio capture software . . . 117

Q Compiling from source code **118**

Q.1 Linux . . . 119

Q.2 macOS . . . 119

Q.2.1 Intel and ARM . . . 119

Q.2.2 PowerPC . . . 120

Q.2.3 Redistributable static binaries . . . 120

Q.3 Windows with MSYS2 . . . 120

Q.3.1 Compiler and libraries . . . 120

Q.3.2 plotutils-2.6 . . . 121

Q.3.3 libFLAC 1.5.0 . . . 121

R Colors available for graphs **122**

S Licensing **124**

T Contact the author **125**

U Acknowledgments **126**

Chapter 1

Introduction

MDFourier is a free and open source software suite for analyzing and comparing *audio signatures*.

It can be used to identify the differences between two *audio recordings*. For instance, a *Sega Genesis Model 1 VA3* and a *Sega Genesis Model 2 VA 1.8* can be compared in order to verify how they differ across the human hearing frequency spectrum, using objective and repeatable results.

The process is not limited to one specific system, and the software can be configured to make comparisons for other platforms with the addition of new *profile* files.[1] At the moment there are *profiles* for these platforms:

- *SEGA*: *Mega Drive/Genesis*, *Mega/Sega CD*, the *SN76489 DCSG* family (*Master System*, *Game Gear*, *SG-1000* and *ColecoVision*) and the *Dreamcast VMU*.
- *Nintendo*: *Famicom/NES*, *Famicom Disk System*, *Game Boy* and *Super Famicom/Super Nintendo* (with *MSU-1* variant).
- *NEC*: *PC Engine/TurboGrafx-16* and its *CD-ROM*^{2} addons.
- *MSX*: *PSG*, *PSG+SCC*, *PSG+FM* and *PSG+SCC+FM*.
- *MS-DOS*: sound cards with *OPL2/OPL3 FM* and *PCM*.
- *Sharp X68000*.
- Arcade: *SNK Neo Geo*, *Capcom CPS1* (with an *ADPCM* only variant), *Taito F2*, *Irem M92* and *IGS PGM* (*ICS2115*).
- *Equipment*: profiles to evaluate audio interfaces and gear, at *32kHz*, *44.1kHz/48kHz*, *96kHz*, *176.4kHz* and *192kHz*.

[1] The term *system* will be used to cover vintage retail hardware, emulators, *FPGA* implementations and any other possible variant that executes binaries for the target hardware.

The *profiles* folder has the complete list for each release.

The intention is not to disparage any particular system, but to promote a better understanding of audio variations, while allowing those who wish to modify their consoles the ability to accurately identify the differences.

A secondary, but not less important goal is to create a community driven catalog of audio signatures from these systems and the variations between consoles for archival and preservation purposes.

It is my personal opinion that it would be useful to have configurations tuned to replicate the audio signatures of original retail hardware variations in emulators and *Field-programmable gate array* (*FPGA*) implementations. This, however, does not mean there in not room for improvement. Reducing noise while keeping the reference sound signature is one such scenario.

Although powerful industrial solutions for this kind of analysis must exist, they do not seem to have reached the enthusiast community. I'm not aware of any other similar effort to create public software analysis tools that are aimed at vintage console hardware.

It must be noted that there have been detailed comparisons, made in particular for the *MegaDrive/Sega Genesis*.[2]

1.1 Possible applications

- Figure out if *vintage hardware* retail variations really have different audio signatures when compared against each other.
- Help emulator and *FPGA* implementation authors with tools to tune *filter profiles* that match *vintage retail* hardware audio signatures.
- Help the modding community to compare how *audio signatures* change for each *synthesizer* within the targeted system while developing audio modifications.
- Determine if there were changes in the audio *spectrum* after modifications to a console, like *recapping* or changing the audio circuitry.
- Evaluate equipment, such as *switchers* and *upscalers* with audio *passthrough*. It can help Figure what effect they may have by recording with and without them connected in the *Audio/Video* chain.
- Help to recreate specific audio signatures.
- Help to tune a system to a particular taste.

There is also some interesting information that is derived from the analysis, and which is displayed as part of the output:

[2] See the forum post at [2]

- The *frame rate* of the console under analysis.
- Plots regarding the *Noise floor* from the recorded signals.
- *Audio balance* percentage difference between the stereo channels.
- When using *CD-ROM drives*, it can output the time it took the unit to start playback.
- The *clock* that drives some sound chips, measured from a known tone, and which known model it matches.[3]

[3]See appendix D.

1.2 Disclaimer

MDFourier and this documentation are a work in progress. Although I tried to adhere to best practices, my experience in *digital signal processing* was almost non-existent before this project.

Please contact me if you have corrections, improvements, suggestions or comments. These are encouraged and welcome. Contact information is available in appendix T.

This project was born out of my curiosity to compare the audio signatures of different revisions of the *Mega Drive/Genesis*, and verify them with a tool assisted analysis. Due to my interest in *game preservation* it naturally evolved into its current form, and will hopefully continue to grow.

Chapter 2

Audio Basics

This chapter is an introduction to the audio concepts that will be used throughout the document, such as: *frequency*, *amplitude*, *sample rate*, *decibel*, etc. Hopefully it can provide a simplified–but functional–perspective to those unfamiliar with audio and its *digital representations*.

2.1 Sound

Sound is a series of *vibrations* that travel through a medium. In our case this medium is typically air, but it can be water or even a solid. These vibrations are *waves of pressure* and–as any other *wave*–they have characteristics that we can measure in order to classify and understand them.

2.2 Frequency

We use the term *frequency* to refer to a repeating event–how *frequent* it is. In this case it means how many times the *wave* we are measuring repeats in a second. We measure this in *cycles per second*, and use a unit named *Hertz* (*Hz*).

As you can imagine these vibrations in the air can have a wide range of *frequencies*, but we can't hear all of them due to biological limitations. The range that humans can typically perceive is called the *human hearing spectrum*. A *spectrum* refers to a continuous range of frequencies, and in this case it is defined to start at *20Hz* and end at *20000Hz*–we commonly use the term *20kilo-Hertz*, since *kHz* refers to multiples of *1000Hz*.[1] These numbers are of course an average, and every one of us has a different *hearing spectrum* that also changes as we age.

[1] This is the *decimal-based multiplier* system defined by the *International System of Units* (SI).

In Figure 2.1 we can see a *frequency* of *1kHz* in an audio editor. We can tell it is *1000 cycles per second* since the darker selection spans for *0.01 seconds*, and it has *10* complete *wave cycles* in it.

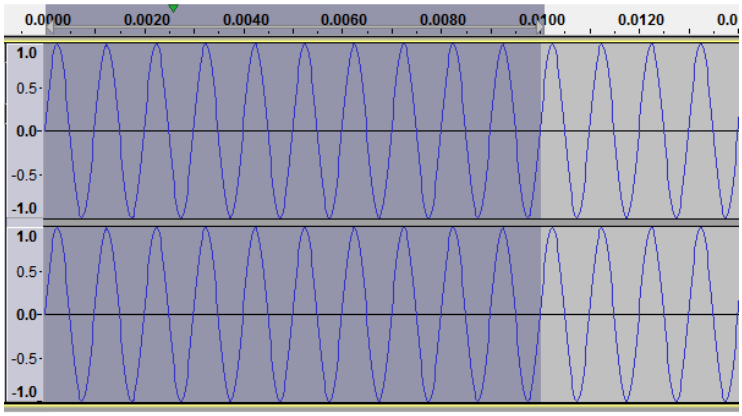


Figure 2.1 A *1kHz* signal in an audio editor.

We perceive *frequency* variations as *pitch differences*: lower *frequencies* are what we call *bass* and higher *frequencies* are *treble*–corresponding to lower and higher *pitched tones*.

2.3 Amplitude

The other *wave* characteristic we use to measure sound is called *amplitude*. This is how *powerful* the *wave* is when compared to the rest of the *signal* that is being analyzed.

Figure 2.2 shows a *1kHz* signal that starts at the highest *amplitude* in an audio editor and goes down to zero. This is a typical *fade out*, but done in a very short period in order to show both *frequency* and *amplitude* in the same image.

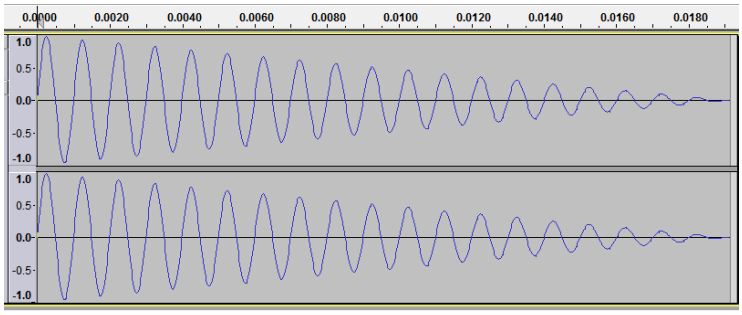


Figure 2.2 A *1kHz* signal with gradually lower amplitude.

We perceive this *amplitude* as *volume*, and the terms are directly related. In general, the higher the *amplitude* of a *sound pressure wave*, the higher the *volume* we perceive. But since a sound with the same *amplitude* but at another *frequency* can be perceived as a different *volume* by the human ear, the terms don't mean the same thing. Since the software can only use *amplitude* for its calculations, we'll use that term throughout the whole document.

The unit used to measure *amplitude* in *sound pressure waves* is called *decibel* (*dB*). These use a *logarithmic scale*, which means that in it each subsequent number is a multiple of a base quantity, instead of a sequence of incremental values by a unit as in a *linear scale*.

We use this scale since it closely models the way we perceive sound, and also because *sound amplitudes* have a very wide range. For instance, the use of a *logarithmic scale* allows us to simultaneously compare the variations of the ambient sound levels from a library against those from a rock concert–even if they are in opposite sides of the *human hearing spectrum*. You might be familiar with this concept from other *logarithmic scales*, like the *Richter scale* used to measure earthquake strength.

Using the scale might be confusing at first, so we'll use an example in order to show how the values behave. Imagine we have two sources that are individually measured as having a *sound pressure level* of *9dB*. If both sources produce sound at the same time, we will measure a combined *sound pressure level* of *12dB* instead of the *18dB* that one might have expected when using a *linear scale*.

This doesn't mean you'll need to use *logarithm algebra* when dealing with this document–or with sound in general. In practical terms whenever you see a *3dB* difference in sound *amplitude*, it means that you'll perceive *twice* the *volume*.

This scale starts at *0dB* which is the inferior limit of human hearing, and goes up to *194dB*; where the sound barrier is broken and it becomes a *shock wave*, at least in the earth's atmosphere. The following table has some common values:[2]

dBA	Source
0	healthy hearing threshold
10	a pin dropping
20	rustling leaves
30	whisper
40	babbling brook
50	refrigerator
60	conversational speech
70	vacuum cleaner at 1 meter
80	alarm clock
85	passing diesel truck
90	squeeze toy
95	inside subway car
100	handheld drill
110	rock band
115	emergency vehicle siren
120	thunderclap
125	balloon popping
130	peak stadium crowd noise
140	jet engine at takeoff
145	firecracker
150	fighter jet launch
180	rocket launch
194	sound waves become shock waves

2.4 Digital representations of sound

In this section, we will be describing how the audio is encoded in a format called *Pulse-code modulation* (*PCM*). It is what a *Waveform Audio File Format* (*WAV*) file uses internally, and the format *MDFourier* can use for analysis.[3]

Sound is a continuous signal that varies in time. This means that if you could zoom into it at the smallest fraction of time imaginable, you'd also have a tiny bit of sound happening during that lapse.

Unfortunately computers don't have the unlimited storage that would be able to replicate a continuous signal–and they also have speed limitations. In order to represent sound in the *digital domain* we use a process called *sampling*.

[2] Table values taken from [40]–you might notice they are listed as *dBA*, this is tuned to our combined *frequency* and *amplitude* response curve.

[3] *Free Lossless Audio Codec* (*FLAC*) is also supported, but for our purposes it can be considered as a compression method that contains a *WAV* file.

2.4.1 Sample rate

Sampling involves measuring the *amplitude* of the *sound pressure wave* at *regular cycles*–at a *frequency*–and assigning a value to each one of this measurements. The *frequency* at which we take each of these *samples* from the continuous *audio signal* is called the *sampling rate*.

The *sample rate* defines the *frequencies* from the signal that we will be able to represent in the digital domain–as the *sampling rate* is increased, so does the *frequency range* that can be portrayed by this digital representation of the signal.

If we want to represent a certain *frequency range* reliably, we must choose a *sample rate* that is *twice* as big as the highest *frequency* value in the desired *spectrum*. This principle is called *Nyquist–Shannon sampling theorem*, and it requires a *band limited* signal in order to work. For a signal to be *band limited* it must not contain frequencies higher than those that we wish to represent. In order to eliminate *frequencies* higher than a certain value, we use a *low pass filer* before digitizing the signal.

Since we want to cover the whole *human hearing range*–which goes from *20Hz* to *20kHz*–we must choose a *sample rate* of at least *40kHz*. This is the reason for the most common *sample rates* in digital audio to be *44.1kHz* and *48kHz*. The *Analysis Software* in *MDFourier* accepts these and higher rates such as *96kHz*, *192kHz* or even megahertz[4]. If a file has a *sample rate* too low for the analysis range, the range is limited to its *Nyquist* frequency.[5]

2.4.2 Bit depth

When we create a *digital representation* of the *amplitude* value for each *sample*, we must define a *bit depth*. In other words we must choose the amount of *bits*–the minimal piece of data in our current computing systems–that will be used to store each number.

A common choice is to use *16 bits* for each measurement. With this we can represent each *sample* with values that range from *-32,768* (-1×2^{15}) up to *32,767* ($2^{15} - 1$). We'll look into what this implies during the next section.

Among other things, this decision affects the resulting *file size* of the audio recording. If we use a *24 bit* depth, files will be *150%* bigger than their *16 bit* counterpart if no compression method is used.

[4] plgDavid has used 3Mhz files without an issue.
[5] See [21] for a detailed video demonstration of how *sampling rates* work.

2.4.3 Decibels in the digital domain

When we were measuring *amplitudes* before, we used the *dB*. In that scenario we were analyzing amplitudes that start at the minimum amplitude our human ears can detect and assigned a value of *0dB* to that level. Values can theoretically go up without limits–aside from that pesky sound barrier we mentioned before when using air as a transmission medium.

However when dealing with digital values that have a defined *bit depth*, our limit is fixed from the other side–the biggest amplitude we can represent with a limited number of *bits*. This number is easily calculated, for n bits we get 2^n possible values.

Because of this, when using digital files we use a variation of the *decibel* named *decibels relative to full scale* (*dBFS*). The highest possible amplitude starts at *0dBFS* and values go down into negative numbers, until they reach the lowest amplitude that can be represented by the selected *bit depth*. When using a *bit depth* of *16*, as used in *Compact Discs*, the minimum *amplitude* that can be represented is *-96dBFS*. The *Analysis Software* in *MDFourier* reads *8*, *16*, *24* and *32 bit PCM*, as well as *32* and *64 bit* floating point files.

This analysis works with the amplitudes of frequencies, not with samples. After both files are normalized, the strongest frequency found in either of them becomes the reference, and every other amplitude is measured against it. These levels are in *decibels relative to the strongest frequency* (*dBr*):[16] the strongest frequency is *0dBr*, and the rest are negative. A tone at *-12dBFS* in the file is still *0dBr* when it is the strongest one. Differences between the two files are plain *dB*, and *dBFS* is kept for sample levels, such as the bit depth limits above or the waveform graphs.

This is commonly referred to as the *dynamic range*, and it tells us how far apart the minimum and maximum *amplitudes* that can be represented in a *audio recording* are. For example, if we have a dynamic range of *96dB*, we can represent the relative amplitudes of *thunderclap* and *rustling leaves* within the same recording; but we could have no lower or higher amplitudes since they'd exceed the range.

Some common formats use *24 bit* audio, which has a wider *dynamic range* that can cover *144dB*. One must think that more is always better, but there are valid reasons that explain why this format is not widespread in consumer products.

Current *digital audio converter* technology can only deal with around *120dB*, which makes a big part of the advantage go to waste. And secondly, we rarely have a need for such a wide range in our typical recordings. For most of the cases that will be analyzed by *MDFourier* the *96dB* range from *16 bit* audio is enough. Recordings at *24 bit* are still preferred, since they keep the noise floor of the capture device below that of the system being recorded.

For studio and mixing applications *24 bit* audio is a huge advantage, since

the *dynamic range* available with the *16 bit* variant can be easily surpassed when mixing signals from similar amplitudes and end up causing undesirable saturation of the resulting signal.

Chapter 3

Working with *MDFourier*

The *software* components that allow *MDFourier* to make comparisons for a particular platform are:

- The *Tone Generator*: It is the piece of *software* that produces the audio signals from the system that is under analysis.
- The *Analysis Software*: This runs on a computer, and creates the graphics that show the results.

The whole process is shown in Figure 3.1, including the main *hardware* components.

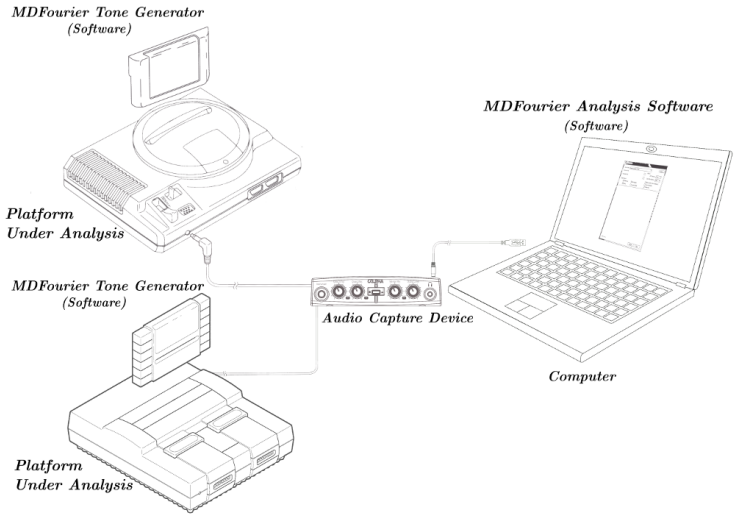


Figure 3.1 Relation between the different *hardware* and *software* components of *MDFourier*.

3.1 Workflow

MDFourier takes two signals, the first one is designated as the *Reference* file. The second one is the file under scrutiny, and is referred to as *Comparison* file.

The first step is loading the *Tone Generator* to the desired *system* that will be analyzed. The process for this varies with each platform: from placing the software in a *flash cart*, burning a *CD-ROM* or using a *custom loader*.

The next step is preparing an *audio capture device* and a computer, in order to record an audio file.[1] Once the *audio interface* is ready, and the cables are hooked up from the *system* to the *audio inputs*, the recording process is started from the *computer* and the *Tone Generator* should be activated from the *system* under analysis.

After a short lapse, usually around one minute, the *Tone Generator* will display a message indicating that the recording process can be stopped. When at least two files are available, one to be used as *reference*–which can be one of the files provided–and one to be used as *comparison* file; a comparison can be made with the *Analysis Software* from the computer.[2]

3.2 How *MDFourier* works

The *Reference* file is used as a control. This means that its characteristics are considered the true values to be expected and against which the *Comparison* file will be evaluated. In consequence, all results are relative between the signals.

Having these files, the software analyzes and compares their audio characteristics and generates graphs that visually show how they differ.

These files are audio recordings from the desired hardware, preferably captured with a *flat frequency audio interface* and produced by the *Tone Generator* that runs on the target platform.[3] This *Tone Generator* is specifically crafted for the particular hardware capabilities and frequency range of the target. Whenever possible, the *Tone Generator* will be included in the *240p Test Suite* for the platform to be analyzed.[4]

Although the *Analysis Software* is *command line* based–in order to be multi-

[1] Mono or stereo *WAV* and *FLAC* files can be used, with *8*, *16*, *24* or *32 bit PCM* or *32/64 bit* floating point samples, at *44.1kHz*, *48kHz* or higher sample rates. *MP3* is not supported, since its compression method alters the results in this type of analysis as shown in section 4.6.

[2] All software and some example recordings are available from the web page of the project, see appendix A for details.

[3] Flat frequency response refers to the capability of capturing the whole audible spectrum with little or no variation, regardless of frequency. A *non-flat frequency* response card can be used to gather relative information, but the captures won't match the ongoing catalog in order to make further comparisons. See *Audio capture device* in appendix P for more information.

[4] A homebrew software suite for video game consoles developed to help in the evaluation of TVs, upscalers, upscan converters, line doublers and video processing in general.[7]

platform and offer the tool on every operating system that has an *ANSI C99 compiler*–a *Graphical User Interface* (*GUI*) front end for *Microsoft Windows* is provided for simplicity and accessibility. Full Source code can be downloaded from *github*.[5]

3.3 File alignment

MDFourier takes both files and auto detects the starting and ending points of the recording. These points are identified by a train of pulses at a frequency defined in the profile, for example *8820Hz* in the *Mega Drive/Genesis* implementation. This process allows the calculation of the *frame rate* at which the system was running,[6] so that the recording can be precisely trimmed into the segments that are defined through the *configuration file* for further analysis.[7]

This process guarantees that the *reference* and *comparison* files are logically aligned, and that each note–or segment–is compared to its corresponding one. The results have no overlap, and no audio editing or trimming skills are required from the user. The current pulse detection accuracy is around 1/4 of a *millisecond*.

After alignment is complete, the software reports the starting and end points of both signals in seconds, samples and bytes, and the *frame rate* detected for each one.

Some recordings don't have the pulse trains, for example those made from purely digital sources or with programs that don't include them. These can be compared with a *No Sync* profile, where the start of the signal is found by other means but with a much lower precision and success rate.[8]

3.4 The heart of the process

A process called *Discrete Fourier Transform* (*DFT*) is used in order to analyze and compare the *amplitudes* from each of the *frequencies* and *harmonics*–the *spectrum*–that compose the audio signal.[9]

In order to compare the frequencies and amplitudes of each file against each other, a reference point must be set. This is achieved via a relative *normalization*

[5] See download link [6].

[6] The *frame rate* is the ratio at which the console sends video frames to a display. For more details see appendix G

[7] This file specifies the operating parameters for each hardware configuration. It is described in appendix C

[8] See section C.6.

[9] The *DFT* is the variant of *Fourier Transform* applied to discrete values, such as the ones we have in the audio file [1]. The software uses the *The Fastest Fourier Transform in the West* library in order to perform the *DFT*. [8]

based on the *maximum amplitude* of the *Reference* file.[10] A *local* maximum search is done at the same frame in the *Comparison* signal, and that amplitude is then used to normalize both files.

This is done in the *frequency domain*, in order to reduce amplitude imprecision when comparing recordings with different *frame rates*.[11] The software does have the option to do this in the *time domain*, but quantization and amplitude imprecision is to be expected if used.[12]

After normalization is finished, the frequencies of each *block* are sorted out by amplitude, from highest to lowest.[13] *MDFourier* can be configured to compare a range of these frequencies–by default *2000* of them are compared for each *block* defined in the *mfn* configuration file.[14] I've found such number to be more than enough, and usually the *minimum significant amplitude* limits this parameter to an even lower number.[15] In case a different comparison of frequencies is needed, the amount can be increased or decreased via the *command line* or *extra commands* option with *-f*.[16]

When the profile defines *clock* tones, the clock of each sound chip is measured. If both signals were generated with different clocks, the frequencies of every note will be slightly off and the comparison will show it. The *-j* option compensates for this on each clock separately before comparing.[17]

It can be helpful to listen to the results of these parameters and filters, in order to evaluate if things like limiting the test to *2000* frequencies is sufficient data for the analysis. For this and other purposes I made an extra tool named *MDWave*, which creates segmented audio files as internally processed by *MDFourier*, including all the processing filters and amplitude limits.[18]

After comparing these frequencies between both files, the corresponding matches are found and the amplitude differences are plotted to a graph. Please refer to chapter 4 for an explanation of the various graphs created by this program.

If you are interested in learning what the *Fourier Transform* does and how its *"magic"* works, there are several resources online. Here are a few of them:

- But what is the Fourier Transform? A visual introduction.
  `https://www.youtube.com/watch?v=spUNpyF58BY`
- An Interactive Introduction to Fourier Transforms.

[10] The *normalization* process and the available variants are detailed in appendix F.

[11] Basically all analysis from this point and forward takes place after the signal has been decomposed into its fundamental and harmonic sine waves by the Fourier Transform.

[12] In other words, normalize based on the samples and their values. This is detailed in appendix F.

[13] A *block* is the basic segment of audio to be compared, such a specific note by a synthesizer as generated by the *Tone Generator*.

[14] The profile configuration file is described in appendix C.

[15] See section 3.5 for details on how this amplitude is determined.

[16] See appendix B.1.5 for details on how to use these.

[17] See appendix D.

[18] *MDWave* and its options are detailed in appendix E.

`http://www.jezzamon.com/fourier/`

- The Uncertainty Principle and Waves.
  `https://www.youtube.com/watch?v=VwGyqJMPmvE`

3.5 Minimal significant amplitude

Although the whole frequency spectrum can be compared, there is little practical use in doing so–due to execution time and extra noise from low amplitude harmonics. As a result, rule of thumb defaults are set in order to minimize these issues.

One of these values is a *minimum amplitude* at which to stop comparing the fundamental frequencies that are found after decomposing the signal with the *Fourier Transform*. This is the *minimal significant amplitude* and it is the cutoff at which comparisons made by the software are stopped.

The value is derived from the *silence blocks* defined in the profile, as follows:

- The frequency with the highest amplitude found in the *silence blocks* of the *Reference* file is its *relative noise floor*, and it is used as the *minimal significant amplitude*.
- If the *Comparison* file has a louder noise floor–but not louder than *-40dBr*– that one is used instead, so that its noise is not compared as if it were signal. If both floors differ by more than *20dB*, a warning is drawn in the graphs.
- If there is no *silence block*, or no noise floor can be found, the default of *-66dBr* is used.
- A noise floor louder than *-40dBr* is reported as a warning, since the differences might not be visible. In that case a fixed value can be set with *-p*.

The software also predicts where the *vertical refresh* (the detected *frame rate*), the *horizontal scan rate* (around *15.7kHz* in *NTSC*) and half of it should appear, and lists their amplitudes in the output. It also looks for *mains* hum at *50Hz* or *60Hz* and its first two harmonics. It is reported when it stands at least *10dB* above the median level of the silence, and drawn as dashed lines in the *Noise Floor* graphs. In *NTSC* and *PAL* the vertical refresh is close to the mains frequency, so both fall in the same bin and the log says they can't be separated.

The *-p* option sets the *minimal significant amplitude* to a fixed value, for example *-p -80*. Using *-p 0* disables the automatic adjustment and uses the default.[19]

[19] See appendix B.1.5.

Recordings made from digital sources can have *digital silence*, where the samples in the silence block don't change at all or only at the very edges. Its noise floor is set from its *bit depth*, and no noise floor comparison is drawn for it.

Chapter 4

How to interpret the graphs

The main output of the program is a set of different graphs that vary in quantity based on the definitions made in the *mfn* file,[1] and the selected options.[2]

The graph files are saved under the folder *MDFResults*, inside a sub-folder named after the profile and another one named after the input files.[3] They are stored in *Portable Network Graphic* (*PNG*) format. Graphs are *1800x900* by default, and the size can be changed with the *-L* option or from the *GUI*.

For the current document *800x400* graphs were used in order to fit within a *PDF* or *HTML* presentation. The output graphs that are created by the software are listed and described in appendix B.1.4.

In our examples for the *Mega Drive/Genesis*, there are three *active blocks*[4]: *FM*, *PSG* and *Noise*. These will result in a graph of each type, plus a general one named *ALL*.

We'll follow a series of results from different input files to *MDFourier*, starting with cases that have either none or a few differences and build on top of each one, so you can familiarize with what to expect as output.

[1] Detailed in appendix C.

[2] See appendix B.

[3] For example `MDFResults/SEGA-MegaDrive_SegaGenesis/Reference_vs_Comparison_0000`, the profile name loses its spaces and some symbols. The number at the end increases with each new comparison of the same files, so previous results are kept.

[4] *The Mega Drive/Sega Genesis* has two audio synthesizers: a *Yamaha 2612* (YM2612) for *Frequency Modulated* (FM) audio, and a *Texas Instruments SN76489 Programmable Sound Generator* (PSG)–or equivalent in an *ASIC*–for *Sega Master System* compatibility and extra audio channels.

4.1 Scenario 1: Comparing the same file against itself

The first scenario we'll cover is the basic one, the same file against itself. Let's keep in mind that *MDFourier* is designed to show the relative differences between two audio files.

So, what is the expected result of comparing a file to itself? No differences at all. An empty graph file as shown in Figure 4.1.

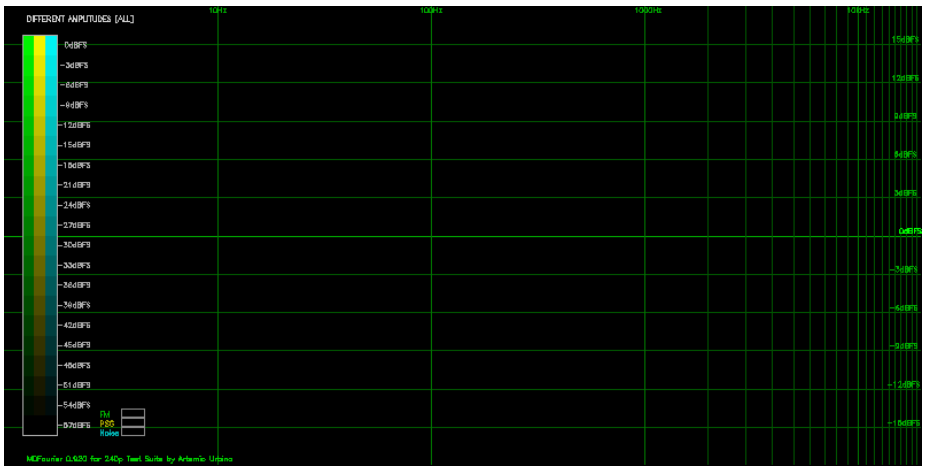


Figure 4.1 Different Amplitudes result file when comparing the same file against itself.

Of course all of the *Differences* and *Missing* graphs will only have the grid and reference bars, with no plotted information since both input files are identical.

All graphs use the horizontal axis for frequency, in a logarithmic scale by default.[5] Values start from *1Hz* on the left and end at *20kHz* on the right, which covers the human hearing spectrum. The analyzed range is *20Hz* to *20kHz* by default, and can be changed with *-s* and *-e*.[6]

For *Difference* graphs the vertical axis is the amplitude difference in *dB*, and a central axis for *0dB*. Values rise to *18dB* towards the top and fall to *-18dB* on the lower part of the graph by default. When *25%* or more of the differences of a type fall outside that range, the software widens it automatically and reports it in the log. A fixed range can be set with *-d*.

There will be two sets of *Spectra*, one for the *Reference* file and one for the *Comparison* file, with one graph for each defined *type* plus the general one.

[5] A linear scale can be selected with *-N*.

[6] Although the plotted range can be altered to cover only a fraction of the spectrum, the resulting graph will hold the same scale.

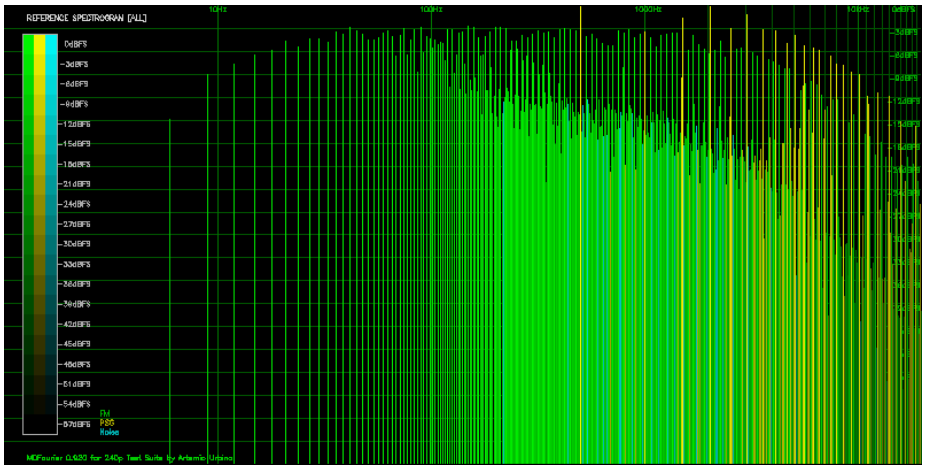


Figure 4.2 The Spectrum for a Genesis 1 VA3 via headphone out.

The Amplitude of each of the fundamental *sine waves* that compose the original signal is represented by vertical lines that reach from the bottom to the point that corresponds to the amplitude in *dBr*. The line is also colored to represent that amplitude with the scale on the left showing the equivalence.

Three colors–as defined from the *mfn file*[7]–are used to plot the graph, with each one of them plotting the frequencies from each corresponding *type* from the *WAV* file.

The top of the graph corresponds to the strongest frequency in either file, which is *0dBr*. The bottom of the graph corresponds to the *minimum significant amplitude*, as described in section 3.5.

As expected, both sets of *spectra* are identical in this case, since we used the same file compared against itself.

4.2 Scenario 2: Comparing two different recordings from the same console

This is another control case. What should we expect to see if we record two consecutive audio files from the same console using the same audio interface?

[7] The configuration file is described in appendix C.

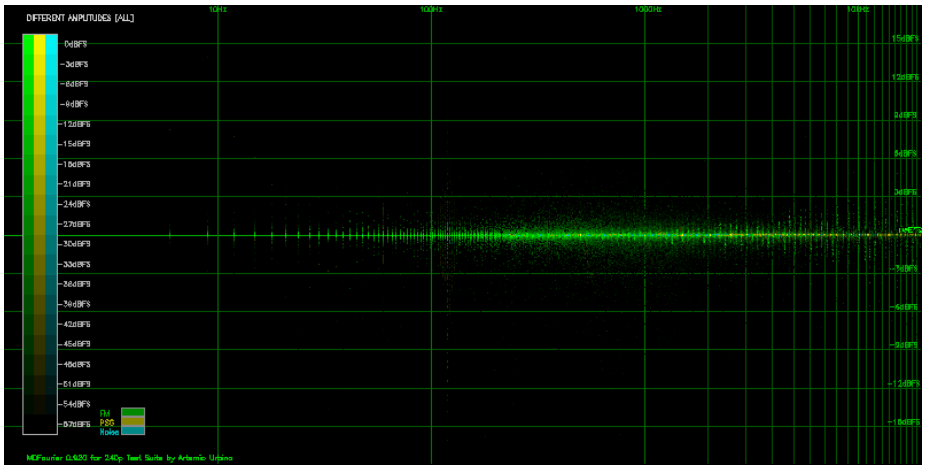


Figure 4.3 The same console, two different recordings.

As you can see, we have basically a flat line around zero. This means that there were no meaningful differences found.

But wait, there *are* differences. Why is that? Due to many reasons: analogue recordings are not always identical, but they should be lower than this. There are also variations from the analogue part of console itself, and probably from the internal states and clocks from the digital side of the process.

But in this particular case they are fluctuations present in the *FM signal*– shown in green here–as demonstrated by comparing other kind of *digital audio* recordings with the *Analysis Software*.[8]

We now know that there can be certain amount of *fuzziness*, or variation, around each graph when comparing certain kind of signals due to this subtle performance nuances. It is a normal situation that is to be expected, and a baseline for further results.

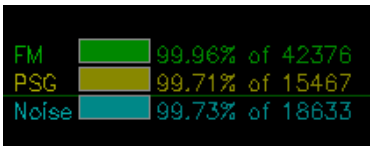


Figure 4.4 Bars for percentage matched.

Until now we haven't discussed the small bars at the lower left quadrant of the graphs. These represent the percentage of matches that were made. In this case we get all above 99.8% as shown in Figure 4.4. This serves as a quick guide on how relevant the plotted differences are when comparing both signals. The numbers to the right are the amount of values that were compared.[9]

[8] One such case is shown in section 5.11, in which very well defined results can be achieved.
[9] Only drawn at higher resolutions, since they aren't readable in the *800x400* mode.

4.3 Scenario 3: Comparing against a modified file

For demonstration purposes, the same *Reference* file was modified to add a *500Hz 6dB* parametric equalization across all the signal. This is an artificially modified and controlled scenario in order to demonstrate what the graphs mean.

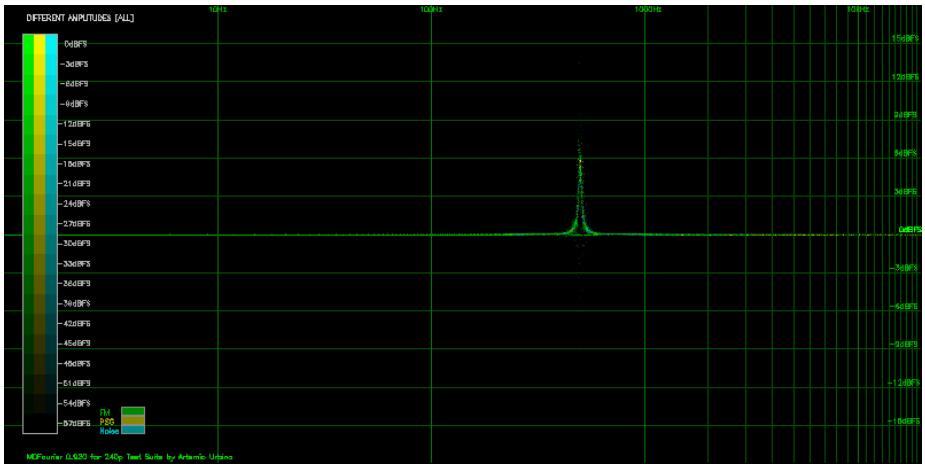


Figure 4.5 Compared against itself modified with a *500Hz 6dB* equalization.

As expected all three *types* (*FM*, *PSG* and *Noise*) were affected and show a spike, exactly *6dB* tall and centered around *500Hz*.

It is interesting to contrast both spectra (figures 4.6 and 4.7), since the *500Hz* spike is also shown there.

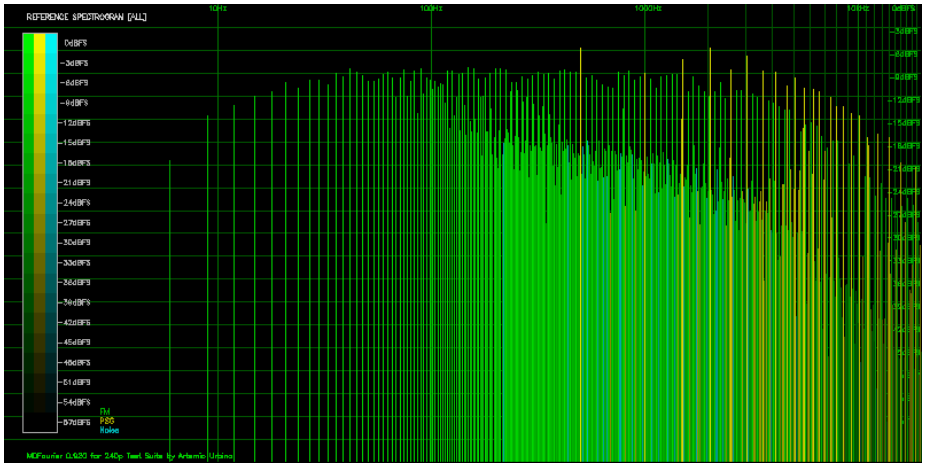


Figure 4.6 Spectrum for the *Reference* File.

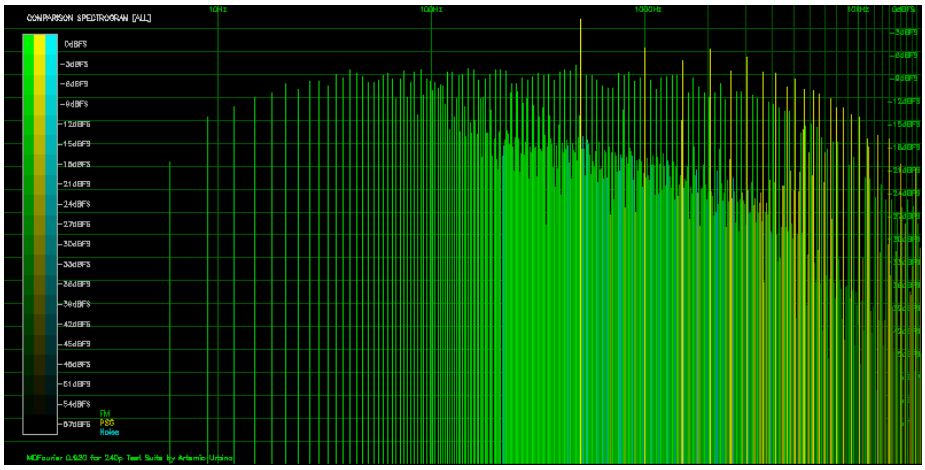


Figure 4.7 *Comparison* file modified with *500Hz* peak.

And the *Missing Frequencies* graphs are basically empty, since no relevant frequencies are missing from the *Comparison* file.

4.4 Scenario 4: Comparing against digital low pass and high pass filters

We will use the same *Reference* file, and compare it to a file modified by several digital filters that were inserted via an audio editor:

- A *low pass filter*[10] to the *FM* section of the file.
- A steeper *low pass filter* at a different cutoff frequency to the *PSG* section.
- A *high pass filter*[11] to the *Noise* section at a different frequency.

Figure 4.8 shows the general graph for the three types:

[10] A *low pass filter* allows only frequencies lower than a specified value to pass through.
[11] A *high pass filter* allows only frequencies higher than a specified value to pass through.

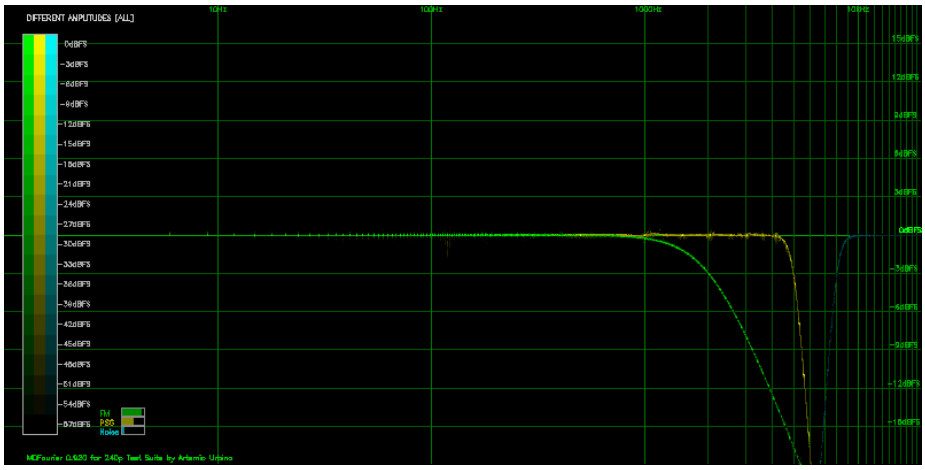


Figure 4.8 *FM* with a low pass filter (in green), *PSG* with a low pass filter (in yellow) and *Noise* with a high pass filter (in aqua).

We can now see that the higher frequencies above *1kHz* in the *FM* graph quickly fall down towards $-\infty dB$, so the first low pass filter is there.

The second low pass filter for *PSG* is at *3kHz*, and it is steeper.

But we can barely see what is going on with the *Noise* part of the graph. We can see that there is some black dots on top of the *0dB* line.

In order to better see what is going on, we'll change the *Color Filter Function* to *"Bright"* so we can have higher contrast against the background.[12]

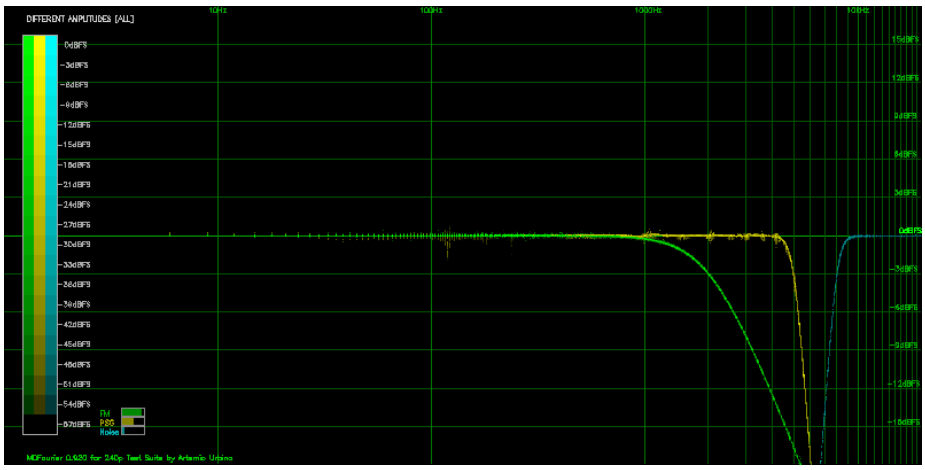


Figure 4.9 Using the *"Bright"* setting for the *Color Filter Function*.

With the new emphasis, we can now make out the curve that raises from

[12] Described in appendix B.1.2.

$-\infty dB$ to $0dB$, and it aligns with $8kHz$.

We can still do better than that, by using the *Average Graphs* option.[13]

Here is the resulting graph for only the *Noise* section of the signal, with average enabled:

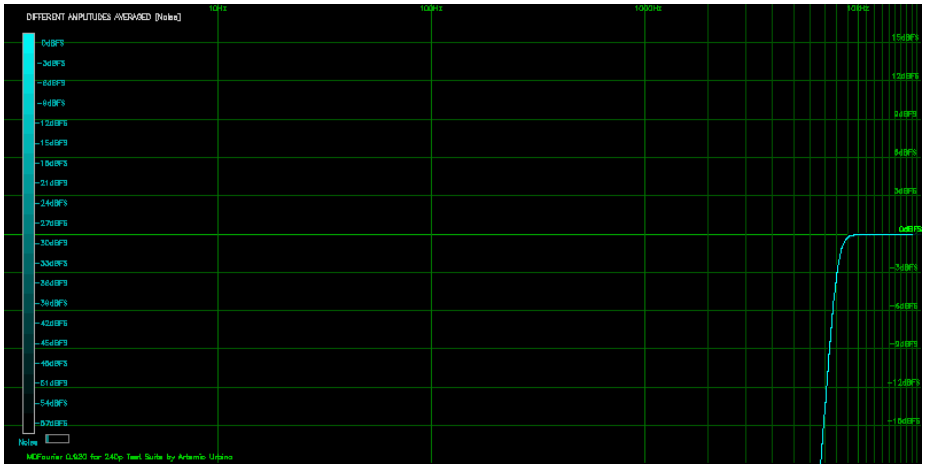


Figure 4.10 *Noise* plot with the *Average Graphs* option enabled.

There are some other interesting graphs that result from this experiment. For example, the *Missing* graphs now show all the frequencies the *low/high pass* filters cutoff.

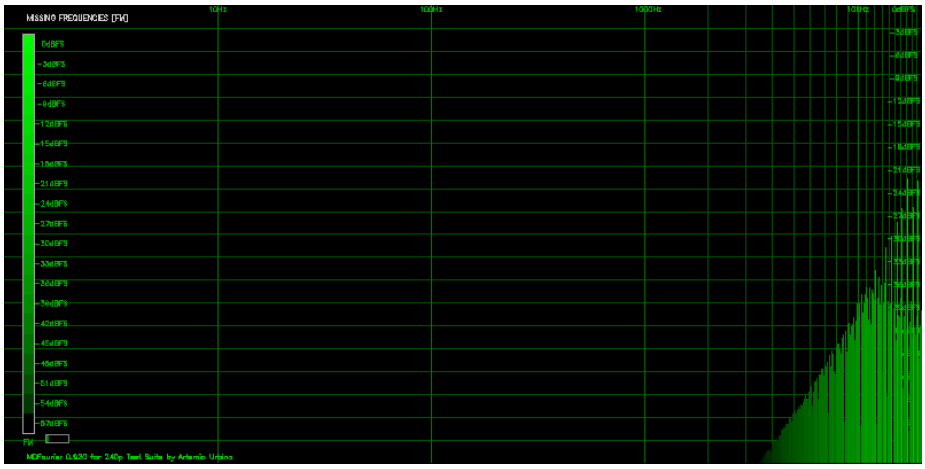


Figure 4.11 Missing frequencies in *FM* cutoff by low pass filter.

As shown in Figure 4.11, there is a curve in the *spectrogram* and only frequencies above *1kHz* show up, slowly rising in amplitude. Many of these are not

[13] See section B for details.

shown since they are below our defined *minimum significant amplitude*.

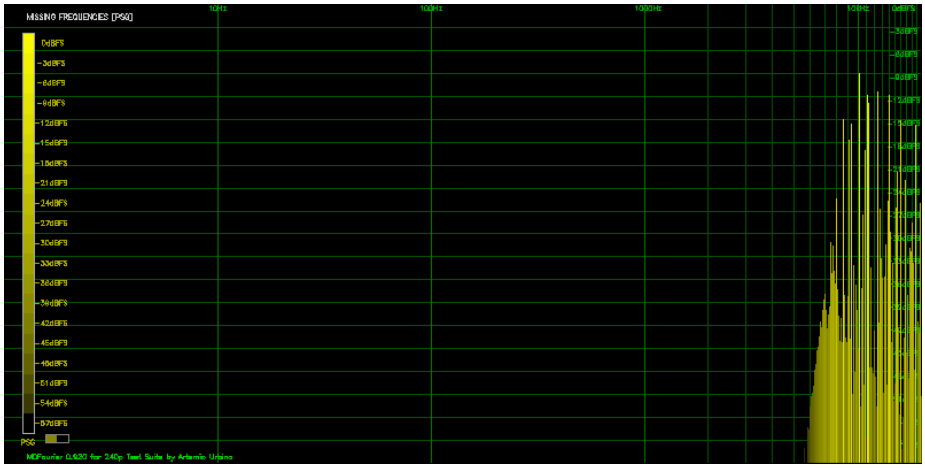


Figure 4.12 Missing frequencies in *PSG* cutoff by low pass filter.

The same behavior can be observed in Figure 4.12. This is the *PSG spectrogram*, and it shows a different curve that starts at *4kHz*.

As you might have noticed, a bar is also present in the *Missing graph*. But in this case it represents the percentage of unmatched frequencies, as shown in Figure 4.13.

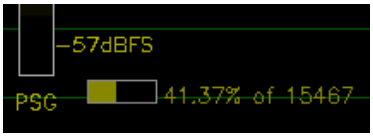


Figure 4.13 Bar for missing frequencies in *PSG* cutoff by low pass filter.

And finally, Figure 4.14 shows the opposite kind of curve, the *high pass filter* that cuts off everything higher than *8kHz* in the *Noise* section.

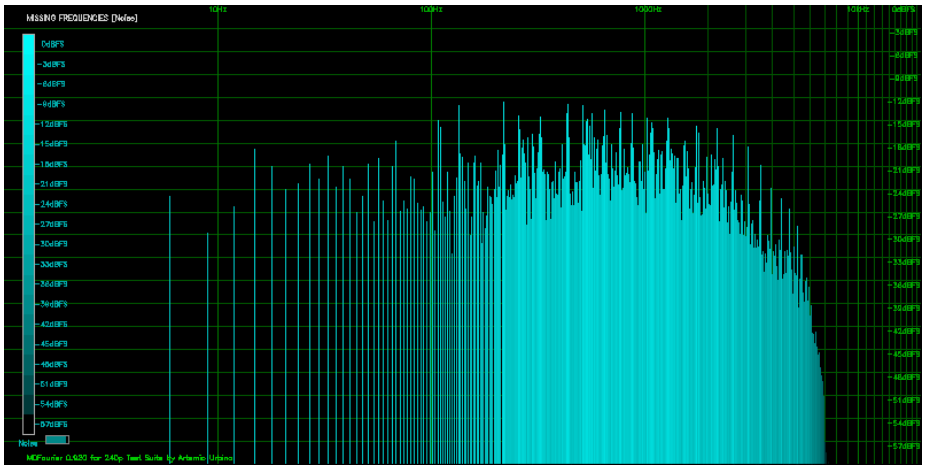


Figure 4.14 Missing frequencies in *Noise* cutoff by high pass filter.

It is a good moment to emphasize that these are relative graphs. They show how different the *Comparison* signal is to the *Reference* signal. And so far we've compared the same signal to itself, although modified though very precise digital manipulations. An analog filter would look the same, but a bit fuzzier.

However, some interesting ideas arise. What would happen if we take this *low/high pass filter* signal and use it as *Reference* and the original one as *Comparison*?

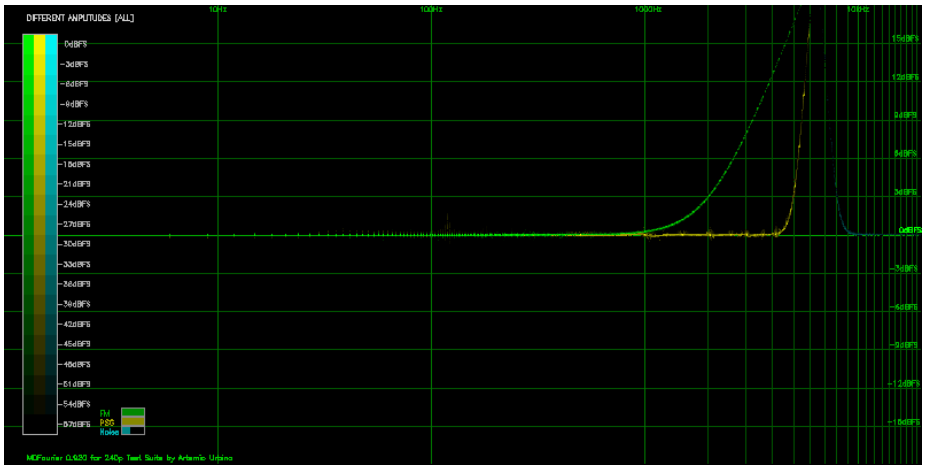


Figure 4.15 Results when using modified signal as *Reference*.

Based on this, one could jump to the conclusion that everything will simply be inverted. After all, the original signal now rises to $+\infty dB$ at the same spots–and that makes complete sense–since those frequencies now have a higher amplitude.

Although the *Differences* graphs will indeed be inverted under these controlled conditions, the *Missing* graphs are different. Most of them are now empty:

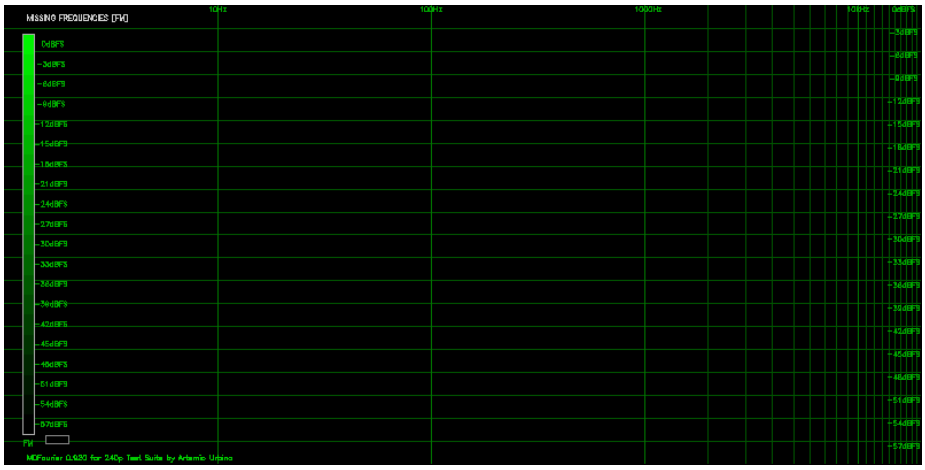


Figure 4.16 *Missing Frequencies* graph for *FM* is empty.

This happens because although we cut a lot of frequencies with such steep *low* and *high pass filters*, all the frequency content from this modified signal is present in the original, but not the other way around as shown above.

4.5 Scenario 5: Comparing two recordings from the same console made with different Audio Interfaces

We'll now compare the same console using two different recordings, one made with the *USB Lexicon Alpha* and the other with a *USB M-Track*[14]. Figure 4.17 shows the results:

[14]See appendix P.1 for details on the audio interfaces used.

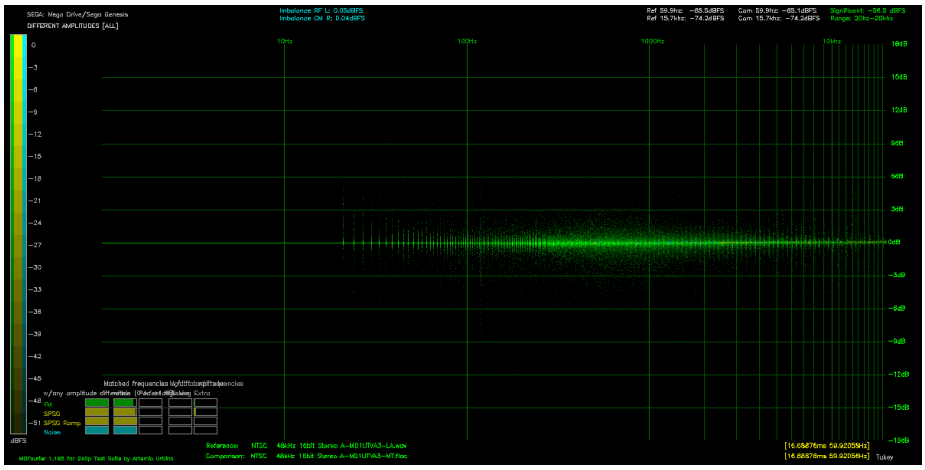


Figure 4.17 Differences using same system and cables, but different audio interfaces.

There is some scatter, as it was to be expected from the results of *Scenario 2.*[15] We can tell that the scatter is centered around the *0dB* line, which means that even using different audio interfaces we discern differences between systems.[16]

Also, there was a slight difference in the detected frame rates. This happens since the sampling clock is not exactly the same in both audio interfaces.[17] Here is the output text from the analysis that shows the detected frame rates:[18]

```
* Loading 'Reference' audio file A-MD1UTVA3-LA.wav
- Audio file header reports 48000Hz 16bits PCM Stereo and 63.76 seconds long
  1.88144s [90309 samples|361236 bytes|361280 bytes/head]
  to 60.2921s [2894020 samples|11576080 bytes|11576124 bytes/head]
- Detected 59.920584 Hz signal (16.688756ms per frame) from Audio file

* Loading 'Comparison' audio file A-MD1UTVA3-MT.flac
- Audio file header reports 48000Hz 16bits PCM Stereo and 62.5774 seconds long
  1.44967s [69584 samples]
  to 59.8603s [2873296 samples]
- Detected 59.920562 Hz signal (16.688762ms per frame) from Audio file

* Comparing Stereo channel amplitude
- Mono block used for balance: SPSG# 0
- Reference signal stereo imbalance: left channel is higher by 0.0497657 dB (0.57459% in a
- Comparison signal stereo imbalance: right channel is higher by 0.0423347 dB (0.48859% in
```

[15] See section 4.2.

[16] Under the assumption both interfaces have a relatively flat frequency response, like the ones used here. For specifications see the *Audio capture device* appendix P.1.

[17] More information in appendix P.1.1.

[18] This output is from an earlier version. Current versions show the same values with more detail, such as the positions in samples and the *frame rate* difference against the profile.

At *48kHz* both cards have very accurate sampling clocks with a minimal difference[19] and *MDFourier* compensates for such issues.[20] Differences can be more prominent when using *44.1kHz*, but the software can still compensate for them without issues.

Here is the graph with the *Average Graphs* option turned on:

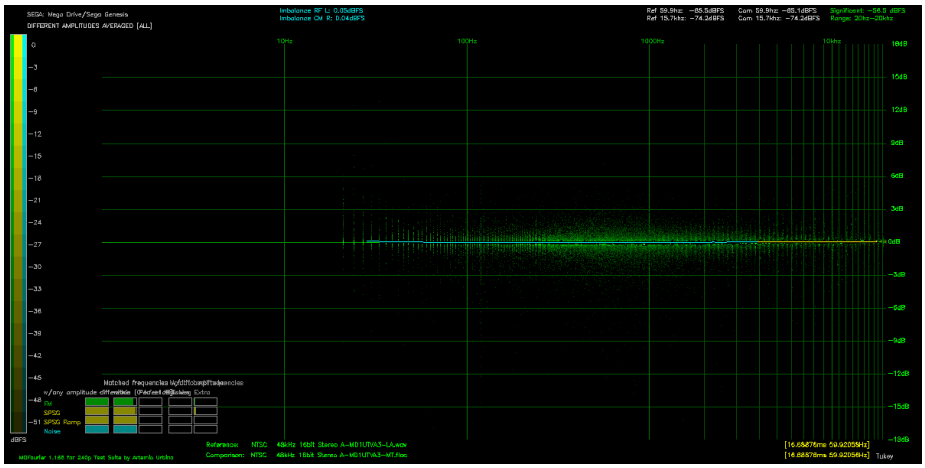


Figure 4.18 Differences using same hardware and cables, different audio interfaces with average enabled.

The results obtained are quite similar to the ones from *Scenario 2*, even to the point of getting almost the same percentages as shown in Figure 4.19. So as long as we are using a relatively flat frequency response audio interface, we'll be able to compare results from different systems even when using a variety of audio interfaces.

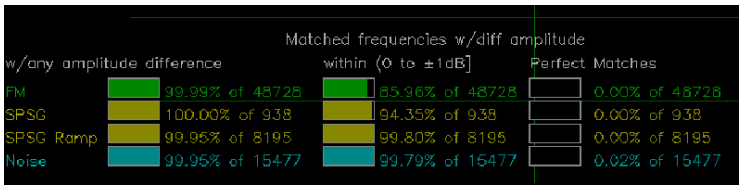


Figure 4.19 Percentage match Bars using same hardware and cables, different audio interfaces.

4.6 Scenario 6: Comparing *WAV* vs *MP3*

The audio file from *Scenario 1* will be used as *Reference* and *Comparison* file, but this time encoded as *MP3* and decompressed to *WAV* in order to use it with

[19] We are talking *0.0001 ms*. In this case, that is *0.0000001 seconds*!

[20] The expected frame rate from the vintage console analyzed here is *16.688 ms* per frame, or *59.92Hz* as measured with a scope. See appendix G

the software.[21]

If the files were identical, an empty graph would result as shown in Figure 4.1 from *Scenario 1*. However we get something slightly different:

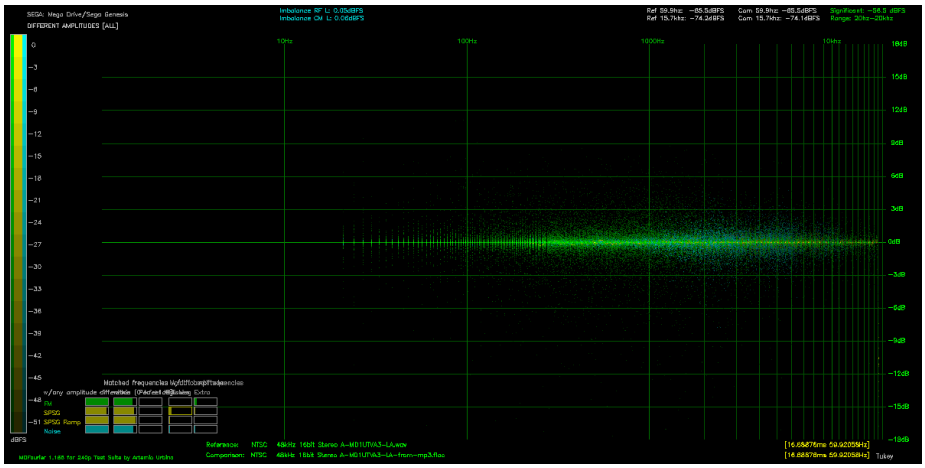


Figure 4.20 Differences between *WAV* as *reference* and *MP3* as *Comparison*.

We get everything centered around the *0dB*–as it should be–which indicates the *audio signatures* are very similar. Variations are correspondent to what we found in *Scenario 5* while we were comparing the same console with different audio interfaces. It must be noted that we used high quality compression for the test, and lower settings will provide different results. The reader is invited to experiment with the tools.

However in this case some more differences show up if we enable the *Average Graphs* option:

[21] The file was created with the *MP3 LAME* encoder using *variable bit rate* in high quality mode, with slow encoding enabled and using real stereo.

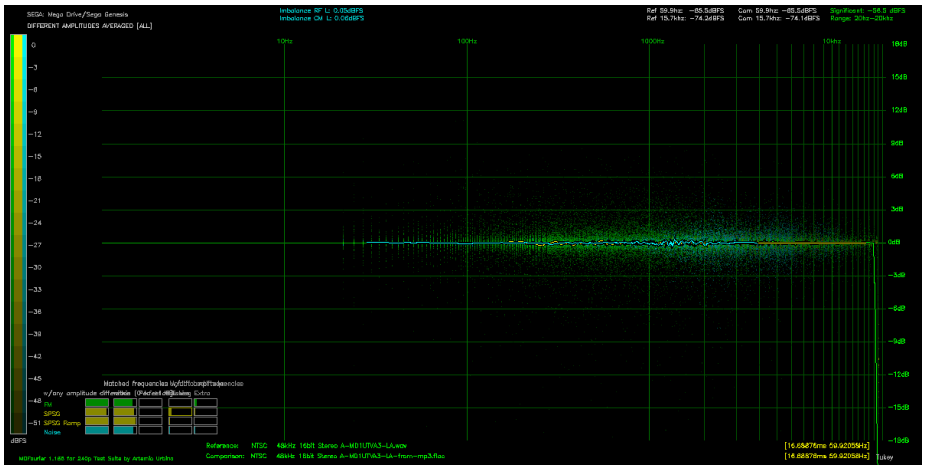


Figure 4.21 Differences between *WAV* as *reference* and *MP3* as *Comparison*, averaged.

As it can be appreciated in the rightmost part of the graph, there is a low pass filter around *18kHz*. These frequencies are at the border of acoustic human limits, and most adults won't even be able to hear them. In part this is what *MP3* compression relies on for reducing the required storage size of each audio file: cutting off frequencies that human beings are unlikely to hear. This is how the *Fourier Transform* helps to create smaller audio–and video–files.

Another side effect of using *lossy* compression[22]–such as *MP3*–is the fuzzy cloud of amplitude differences present in the *500Hz* to *8kHz* range. We are highly tuned to this part of the spectrum, since most of the frequencies that typically compose the human voice reside here.[23] This is most likely what some people perceive as lower quality audio when using these type of files.

Although impressive this is not good enough for our purpose, since we are trying to measure small differences between signals. This kind of noise would make it harder to identify the real source of the differences, specially with different encoding settings, audio interfaces and between consoles.

Another graph that shows the same differences is the *Missing* frequencies graph:

[22] This means it loses some information, as contrasted to lossless.
[23] See [39].

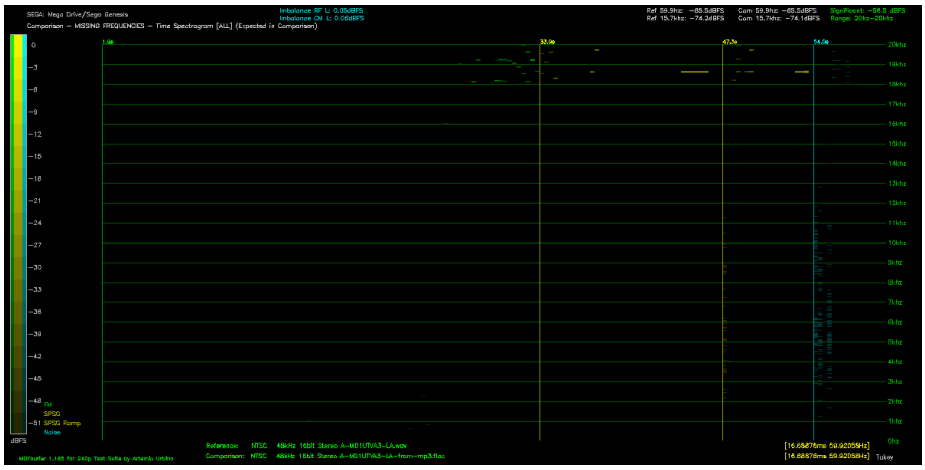


Figure 4.22 Missing frequencies from *MP3*.

It can be appreciated that the missing frequencies were detected after the *18kHz* line, aside from some harmonics below *-45dBr*.

Chapter 5

Results from vintage retail hardware

The following table lists all the hardware used to make the recordings for the graphs in this chapter. All systems had stock parts at the time of recording, no modifications and used original power supplies.[1].

The *MDFourier ID* value in the following table refers to the file name in the catalog, and will be used throught the text to identify the systems being compared.[2]

Type	Model	Revision	FCCID	Serial	Region	Made in	MDFourier ID	Recorded from
Model 1	HAA-2510	VA1		89N61751	Japan	Japan	A-MD1JJVA1	Headphone Out
Model 1	1601	VA3	FJ846EUSASEGA	30W59853	USA	Taiwan	A-MD1UTVA3	Headphone Out
Model 1	1601	VA3	FJ846EUSASEGA	30W59853	USA	Taiwan	A-MD1UTVA3-AV	AV Out
Model 1	1601	VA6	FJ8USASEGA	B10120356	USA	Japan	A-MD1UJVA6	Headphone Out
Model 1	1601	VA6	FJ8USASEGA	59006160	USA	Taiwan	A-MD1UTVA6-1	Headphone Out
Model 1	1601	VA6	FJ8USASEGA	31X73999	USA	Taiwan	A-MD1UTVA6-2	Headphone Out
Model 1	HAA-2510	VA6		A10416197	Japan	Japan	A-MD1JJVA6	Headphone Out
Model 2	MK-1631	VA1.8	FJ8MD2SEGA	151014280	USA	China	A-MD2UCVA18	AV Out
Nomad	MK-6100		50059282		USA	Taiwan	A-NMUT-AV	AV Out
Nomad	MK-6100		50059282		USA	Taiwan	A-NMUT	Headphone Out
CDX	MK-4121		Y40 014198		USA	Japan	A-CDXUJ-LO	Line Out
CDX	MK-4121		Y40 014198		USA	Japan	A-CDXUJ-HP	Headphone Out
CDX	MK-4121		Y40 014198		USA	Japan	A-CDXUJ-AV	AV Out
Sega CD	1690			A20113193	USA	Japan	SCD	System

It must be considered that this is a very small sample size, and no results or interpretations should be extrapolated as generalized conclusions about differences between models.

However, since several common trends are found and the systems were

[1] They were all connected to a PVM-5041Q–a 4" *CRT*–via *RGB*, although the *CRT* was turned off while recording. Since *video refresh* noise is a very isolated frequency range it is not filtered out for comparisons, however it is detected and reported by the software for several purposes.

[2] All the recordings are available from the *downloads page* from the *MDFourier* website. See appendix A.

sourced from very different providers across a long period, it would be too much of a coincidence for these results to be by pure chance. I expect that we–as a community–can grow a bigger data set of recordings, in order to make better assessments in the future.[3]

The recordings presented in this chapters were made using the *USB Lexicon Alpha* audio interface, due to the following reasons: it has a flat frequency response, it is available in the market at the time of writing, it works via USB and it is not expensive.[4] The first system is the *Reference*, and the second one is the *Comparison* signal.

Even if the system volume settings won't affect the analysis since *normalization* is performed in software,[5] the volume control was set to its maximum when using the *headphone out* port. This was made in order to get consistent results, since variations could arise from the performance of the system amplifiers.

5.1 Sega Genesis Model 1 VA3 (US) vs Mega Drive Model 1 VA1 (JP)

These models are an interesting starting point since the *Reference* system–the *VA3*[6]–is usually considered one of the best sounding versions of the system family line.[7]

On the other hand, the Japanese *VA1* is the first revision to the launch *JP VA0* system. This will give us an idea of how much the audio signatures changed from the launch units in Japan to the *US VA3*–which is also the first revision to the USA system, where the launch unit was the *VA2*.[8]

[3] I've considered the viability of creating "averaged" system profiles in the future–using an extra tool yet to be developed–that could give us a generalized profile created from multiple samples.

[4] See appendix P.1.

[5] For details in the *normalization* procedures used, see appendix F.

[6] This refers to the motherboard revision in the system, see [2].

[7] See [3] for more information.

[8] See [4] for information on hardware revisions.

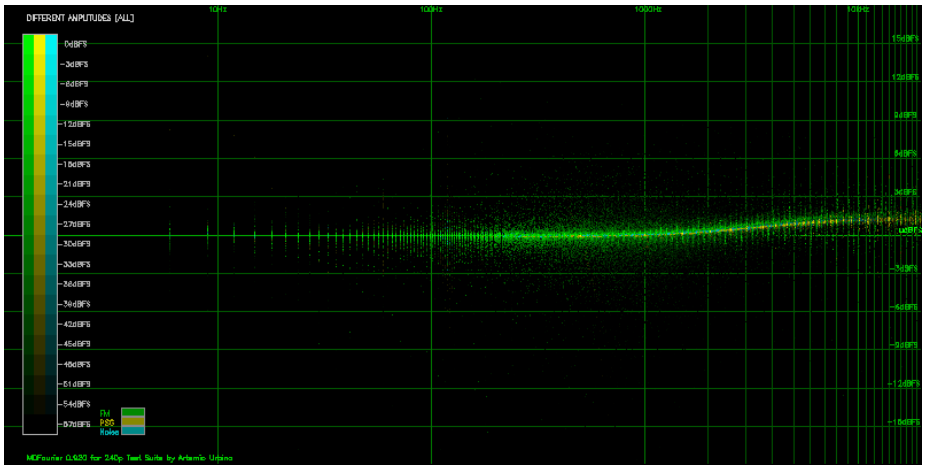


Figure 5.1 Differences between A-MD1UTVA3-LA and A-MD1JJVA1-LA.

The resulting graph shows that the signals are quite similar, where only the higher frequencies–the treble–exhibit some deviation. The curve starts around *2kHz* and slowly rises about *1dB* above the *0dB* line.

It is generally accepted that the minimum perceivable difference–under normal conditions–is around *3dB*.[9] So although a measurable difference exists–which is possibly noticeable under ideal listening conditions–the systems are very similar.

It is also noticeable that the percentage match bars are full, meaning that practically all the frequencies compared were matched.

5.2 Sega Genesis Model 1 VA3 (US): Headphone out vs AV out

Ever wondered if something was lost–besides stereo–when using the *AV out* port from the console instead of the *headphone out*? Figure 5.2 shows both ports compared using the same system.

[9] Under ideal conditions *1dB* can be achieved, see [13].

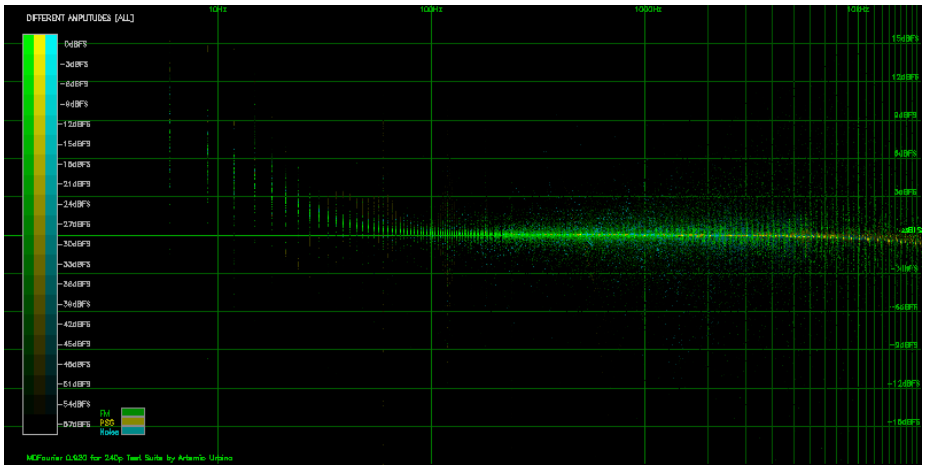


Figure 5.2 Differences between *Headphone out* and *AV out* for the A-MD1UTVA3-LA.

Differences are present only in *bass*[10] and *treble*[11]. In the higher end of the spectrum, differences are less than *1dB* and higher than *17kHz*. The low end differences are more interesting–but still at a frequency range that would hardly matter for most setups, aside from the lost stereo. These reach a *3dB* difference around *30Hz*, so you get deeper bass when using the *AV out* if the music[12] reaches those notes and your audio system has the means to reproduce it.

5.3 Sega Genesis Model 1 VA3 (US) vs Sega Genesis Model 1 VA6 (JP)

We'll again be using the *US VA3* as *reference*, now contrasted against a *Japanese VA6*. The *VA6* is regarded as a good sounding unit in par with the *VA3*,[13] and is also very common.

[10] Low frequency tones.
[11] High frequency tones.
[12] Yes, *Bare Knuckle/Streets of Rage* does reach this part of the spectrum. It must be noted that most of the bass range is really between *60Hz-500Hz*. See [41].
[13] See [2] for details.

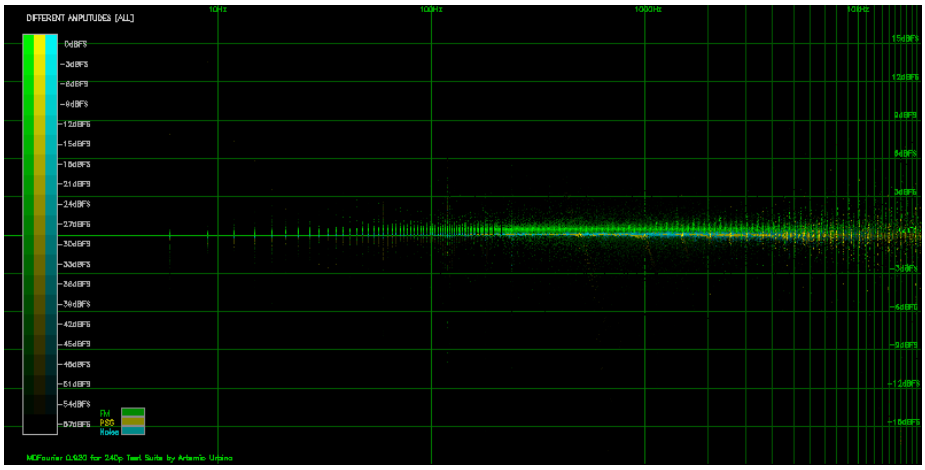


Figure 5.3 Differences between A-MD1UTVA3-LA and MD1JJVA6-LA.

The curves closely follow the *0dB* line, however the *YM2612*[14] amplitude is higher than the *PSG* amplitude by a really low margin, around *0.4dB*. Although measurable, this would be even less noticeable than the difference against the *VA1*.

5.4 Sega Genesis Model 1 VA6 (JP) vs Mega Drive Model 1 VA6 (US)

We'll now compare two systems with the same *VA6* motherboard revision. However, one is Japanese and the other is from North America. Results are displayed in Figure 5.4.

[14]The *Yamaha 2612* synthesizer–also known as *YM2612*–is responsible for the *FM* sound in this systems.

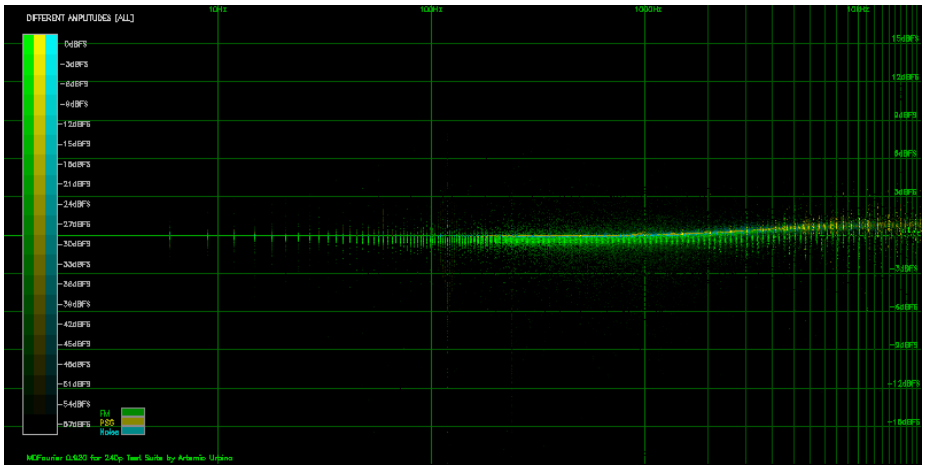


Figure 5.4 Differences between A-MD1JJVA6-LA and A-MD1UJVA6-LA.

Once again, differences are minimal and quite similar to some previous results–in particular to the ones between the *US VA3* and the *JP VA1*. But now *FM* seems to be closer and *PSG* is the one that is less than *1dB* higher on the *US* system.

5.5 Sega Genesis Model 1 VA6 (US) vs Sega Genesis Model 1 VA6 (US)

These are two other *US VA6* systems, both made in *Taiwan*. And here are the results:

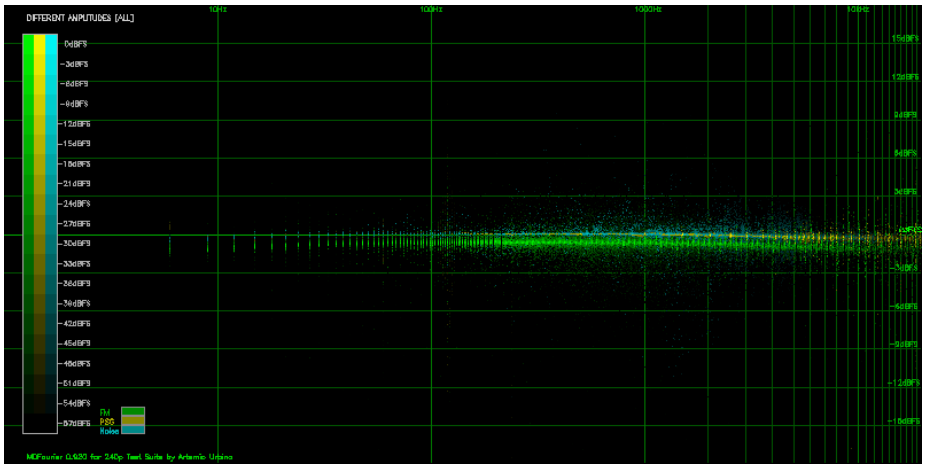


Figure 5.5 Differences between A-MD1UTVA6-1-LA and A-MD1UTVA6-2-LA.

This is an interesting case since both consoles are the same motherboard revision and were manufactured in the same country.

Acoustically, these are very close matches with differences below *1dB*. In this case *FM* is lower by that same amount and *PSG* matches the reference unit.

5.6 Sega Genesis Model 1 VA3 (US) vs Sega Genesis Model 2 VA1.8 (US)

And now for something a bit more interesting. The same American *VA3* unit we've compared to a few others, but contrasted against a *Model 2 VA1.8*. These *Model 2* systems don't use the *YM2612*, but a *Yamaha YM3438* in an *Application-specific integrated circuit* (*ASIC*), which is a variation used for some later models.

These *Model 2* versions–grouped with *VA7* and other revisions–are regarded as having a lower quality sound output. This is due to various factors: the variation in *FM* synthesizer, the amplifier circuits used and the general audio circuit design.

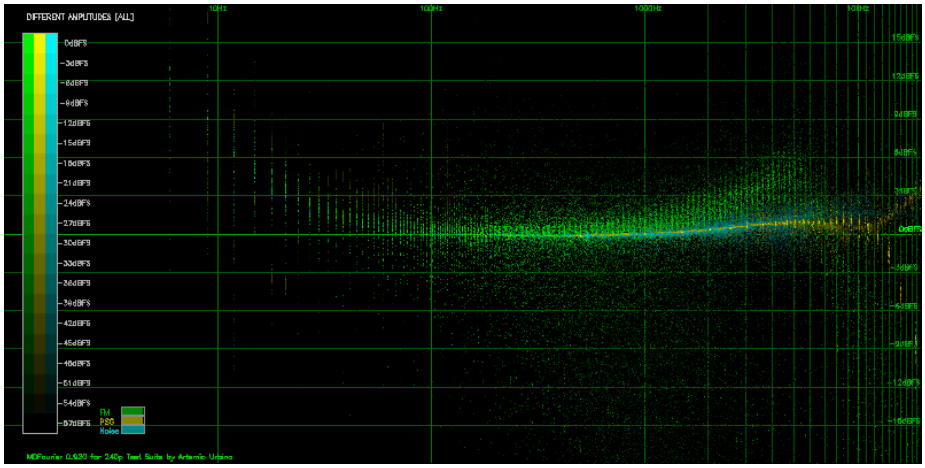


Figure 5.6 Differences between A-MD1UTVA3-LA and A-MD2UCVA18-LA.

The differences in this case are more noticeable: *FM* raises above *3dB*, which is generally accepted as a discernible threshold,[15] and does this around frequencies to which our hearing is quite sensitive,[16] from *2kHz* to around *8kHz*.

But since a lot of data points are highly scattered, we can gather more accurate information for analysis using the *Average Graph* option as shown in

[15] See [13] for results from different studies.
[16] Since this range falls within the spectrum for human voice, see [39].

Figure 5.7.

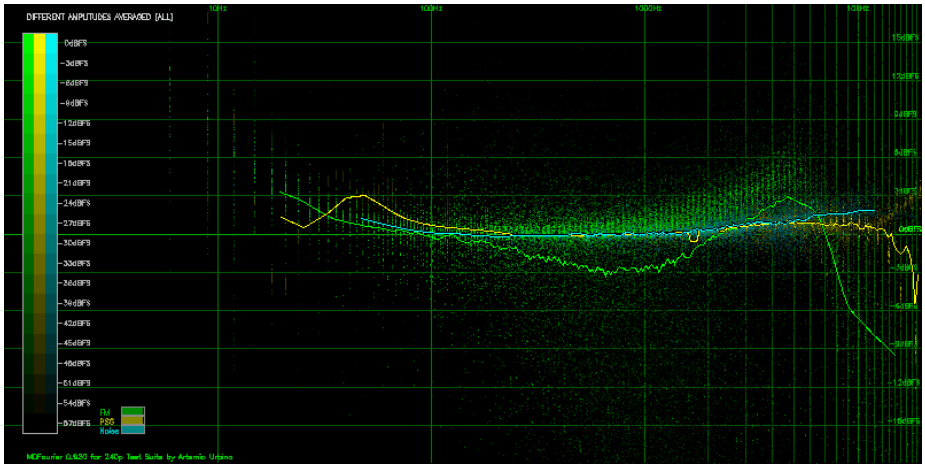


Figure 5.7 Averaged Differences between A-MD1UTVA3-LA and A-MD2UCVA18-LA.

We can now appreciate that there are some relevant frequencies–that is, *high amplitude* in the *reference* signal–that pull the averaged graph lower between *200Hz* and *3kHz-FM* spectrum, and these go down up to *3dB*.

We can also make out that peak in the human voice spectrum that goes *3dB* higher, and then it dips in the treble above *6kHz* to amplitudes lower by *6dB* after *10kHz*.

On the other hand, *PSG* seems quite similar aside from a bump around *60Hz* and dipping in amplitude after *14kHz*.

It must also be noted that, for the first time, there is a drop in the percentage of matched frequencies, shown in the bars detailed in Figure 5.8.

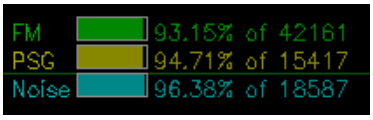


Figure 5.8 Percentage Match Bars between A-MD1UTVA3-LA and A-MD2UCVA18-LA.

So far we had been getting results above *99.7%*, and this time it dropped to *93.15%* for *FM*.

5.7 Sega Genesis Model 1 VA3 (US) vs Sega Nomad (US)

The *Sega Nomad* is a portable version of the *Sega Genesis*, and it had only a single motherboard revision. Recordings were made via both: *AV out* and *headphone out*.

Our first contrast example will be using the *headphone out* port from both consoles, shown in Figure 5.9.

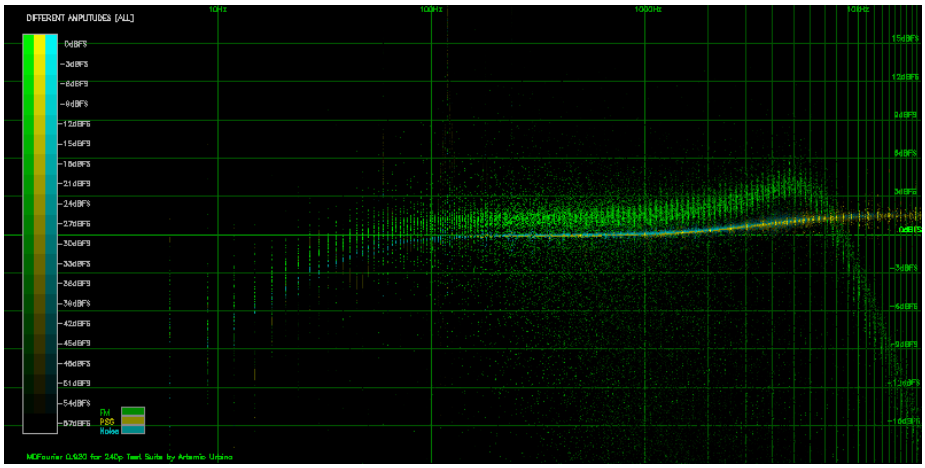


Figure 5.9 Differences between A-MD1UTVA3-LA and A-NMUT-LA.

Some resemblance to the previous comparison to the *Model 2 VA1.8* can be seen. In order to contrast both signals, an *averaged graph* is shown below.

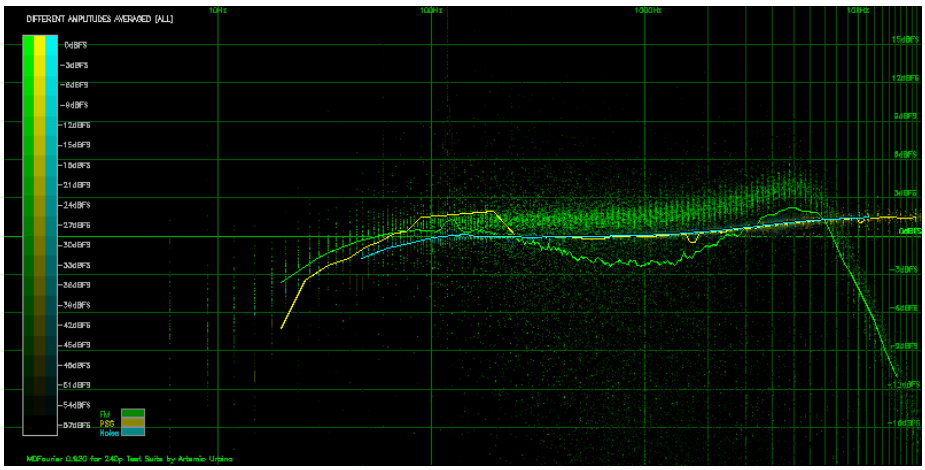


Figure 5.10 Averaged Differences between A-MD1UTVA3-LA and A-NMUT-LA.

The same general behavior from Figure 5.7 can be seen in *FM*, but *PSG* doesn't fall in the same way in the high frequency range. This must be due to differences in the *mixing circuit* or amplifiers.

And if you've wondered about the difference between using the headphone out or the AV out in a Sega Nomad, here are the results:

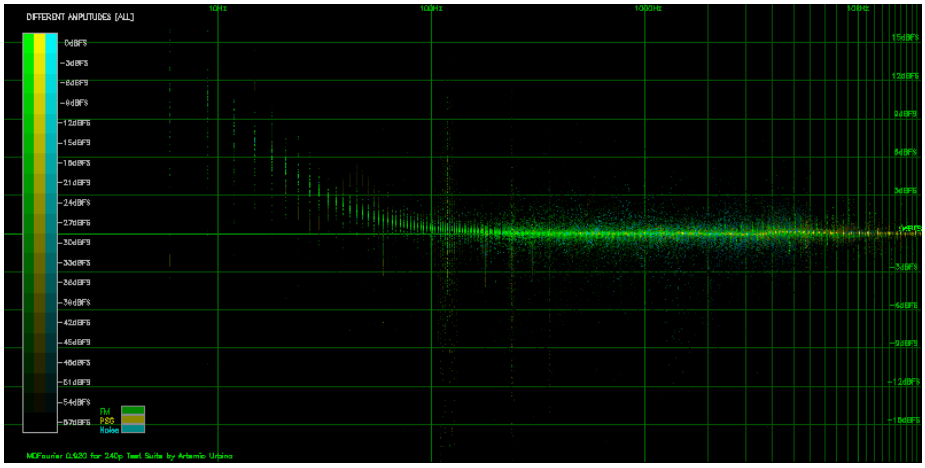


Figure 5.11 Sega Nomad: *headphone* vs *AV out*.

In this case there are no differences above *100Hz*. However the AV out has a more low frequency emphasis that is *3dB* higher below *40Hz*.

5.8 Sega Genesis Model 2 VA1.8 (US) vs Sega Nomad (US)

Due to the similarities shown between the *Nomad* and the *Model 2 VA 1.8* when contrasted against the *Model 1 VA3*, here is the comparison between them using the *AV out*.

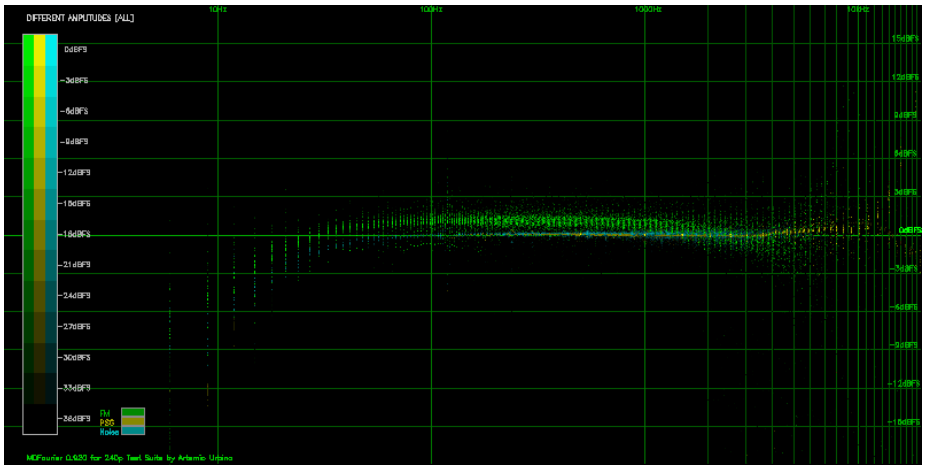


Figure 5.12 Differences between A-MD2UCVA18-LA vs A-NMUT-LA.

The consoles are indeed similar between themselves, with differences probably arising from amplification circuits. The *Low pass filter* is steeper in the *Sega Nomad* for *FM*, among other differences in the high rage regarding *PSG*.

Another interesting point is that both of these systems exhibit video refresh noise at a higher amplitude via the *AV out* than their *Model 1* counterparts: around *-45dBr* instead of *-70dBr*.

5.9 Sega Genesis Model 1 VA3 (US) vs Sega CDX (US)

The *Sega CDX* is an all-in-one system that includes the *Sega CD* unit. It has three different output options: *headphone out*, *line out* and *AV out*.

We'll start by with the results of the *Model 1* as *reference* contrasted to the *CDX line out*.

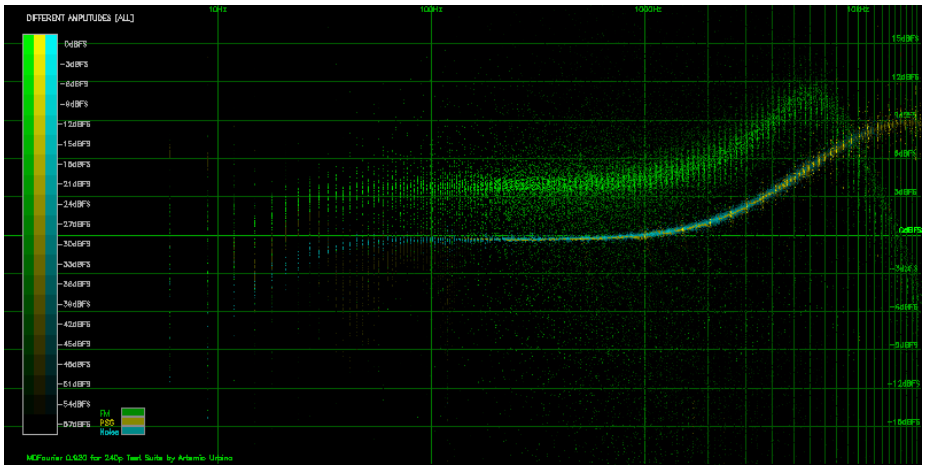


Figure 5.13 Differences between A-MD1UTVA3-LA and A-CDXUJ-LO-LA.

These are easily the most notorious differences between *vintage retail hardware* I've found so far.

The *CDX PSG* does not have the low pass filter present in *Model 1* systems. The *FM* output is higher by at least *3dB*, and it has a *low pass filer* around *9kHz*.

Figure 5.14 shows the results when the *average* option is enabled.

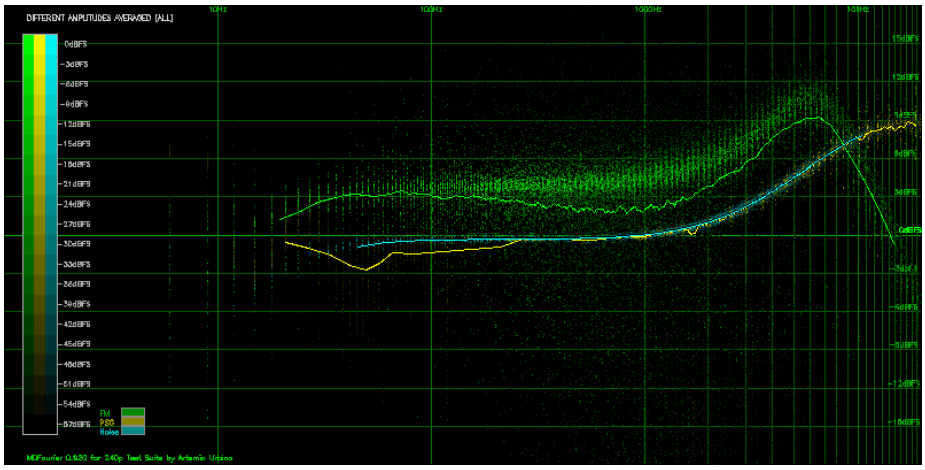


Figure 5.14 Differences between A-MD1UTVA3-LA and A-CDXUJ-LO-LA Averaged.

This makes it easier to discern the curve that the *FM* audio follows when compared to the *Model 1*. It reaches as up to *9dB* higher between the *4kHz-9kHz* range.

We must remember that the *Model 1*, used here as reference, has it's highest

amplitude in the PSG signal with the current test. That's why the results show FM higher instead of a lower *PSG*, since *MDFourier* does relative comparisons based on the *Reference* signal.

5.10 Sega CDX (US) Line-out vs Sega CDX (US) Headphone port

And for completeness, here is the *differences* graph when contrasting the *Sega CDX line out* vs the *headphone out*.

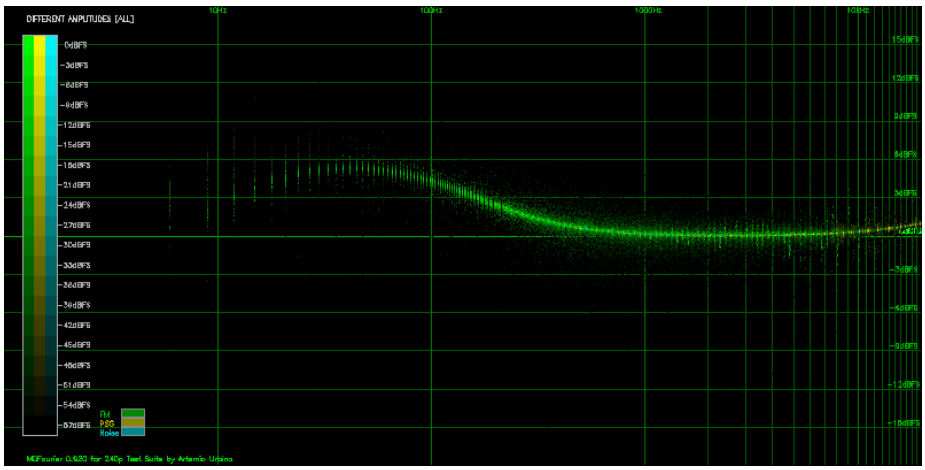


Figure 5.15 Differences between A-CDXUJ-LO-LA and A-CDXUJ-HP-LA.

The *headphone out* has clear emphasis in the low frequency range, and even below *1.5kHz*.

5.11 Sega Genesis Model 1 VA3 w/Sega CD (US) vs Mega Drive Model 1 VA6 w/Sega CD (JP)

We are now comparing an extra line, *Compact Disc Digital Audio* (*CDDA*) from the *Sega CD*. It will be represented by an orange line in the graphs.

For our first case, we will compare if an American *VA3* is any different from a Japanese *VA6* when connected to the same *Sega CD* system.

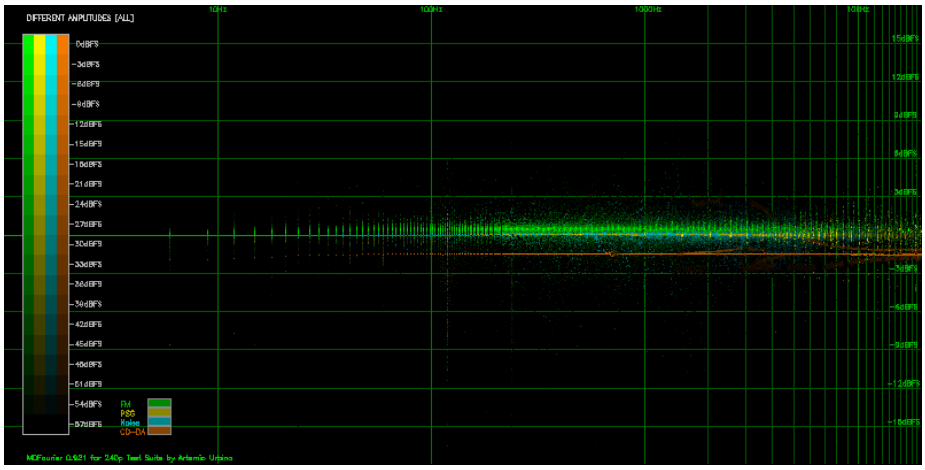


Figure 5.16 Differences between *A-MD1UTVA3-SCD* and *A-MD1JJVA6-SCD*.

The Japanese *VA6* shows a lower amplitude for the *CDDA* audio in the final mix, at around a constant *1.5dB* across the whole spectrum.

It is interesting to note that the differences in *amplitude* follow a very well defined pattern, as opposed to the *fuzziness* found when comparing *FM* audio. This demonstrates the natural harmonic variations an *FM signal* can present.

As a positive side effect from doing a precise frame alignment of the *CDDA* signals for the comparison process, the *Analysis Software* also displays the time it took the *CD unit* to start playing the audio track:

```
- CD-ROM command delay: 9.8195 frames
```

In this case the *Sega CD* took almost *9 frames* to start playing the CD track from the moment the *Basic Input/Output System* (*BIOS*) was requested to do so. These results can be used to determine differences between drives–and could potentially tell us something about their maintenance status.

The difference in amplitude between regions is intriguing. It could of course be present just between these particular units. We'l explore it further in the following cases.

5.12 Mega Drive Model 1 VA6 w/Sega CD (JP) vs Mega Drive Model 1 VA6 w/Sega CD (US)

In Figure 5.17 we are contrasting the differences that arise when using the same *US Sega CD* unit connected to Japanese and American *VA6* systems, both

made in Japan.

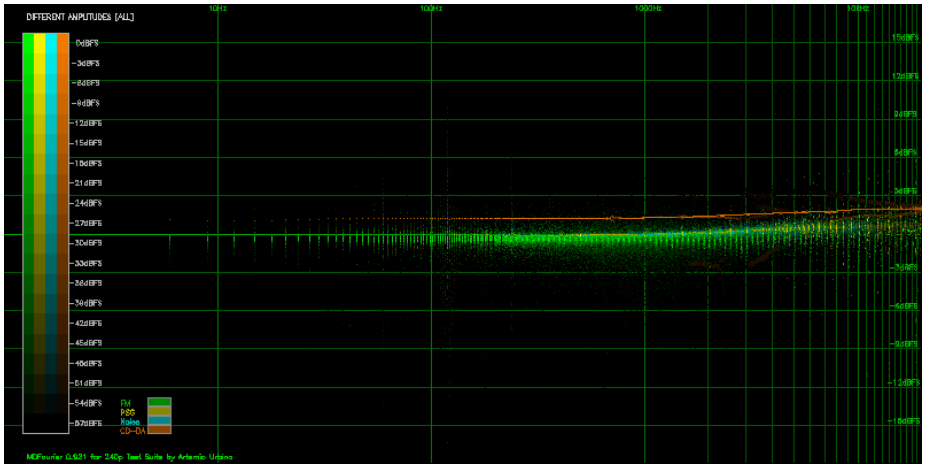


Figure 5.17 Differences between the Japanese *MD1JJVA6-SCD* and the American *MD1UJVA6-SCD*.

It can be appreciated that the *US VA6* unit shows a higher amplitude for the *CDDA* mix–at around *1.5dB*–for the whole spectrum. This is the opposite from the previous case, but this time we are using the Japanese unit as *reference*.

This shows that the two American systems might precisely match in amplitude in all the compared signals.

5.13 Sega Genesis Model 1 VA3 w/Sega CD (US) vs Mega Drive Model 1 VA6 w/Sega CD (US)

Although I don't have other Japanese systems to compare if the amplitude balance against *CDDA* is a regional difference, we can at least verify if American systems show the same balance. Figure 5.18 contrasts a *VA3* and a *VA6* both using the same *Sega CD* unit.

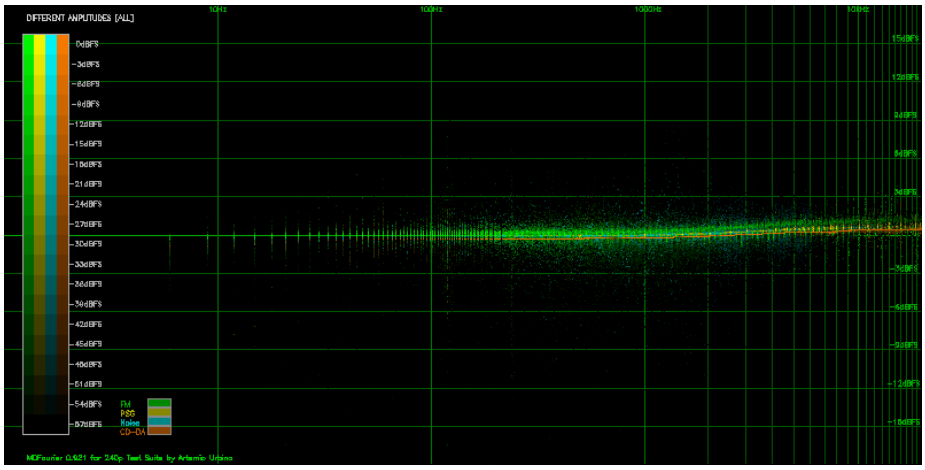


Figure 5.18 Differences between a Japanese *MD1UTVA3-SCD* and *MD1UJVA6-SCD*.

And yes, we do have a perfect match across the whole spectrum and for all the compared audio sources.

5.14 Sega Genesis Model 1 VA3 w/Sega CD (US) vs Mega Drive Model 1 VA6 w/Sega CD (US)

We are already familiar with the results of comparing a *Model 1 VA3* and a *Model 2 VA1.8* as shown in section 5.6. Here is the same comparison but with *CDDA* added to the mix:

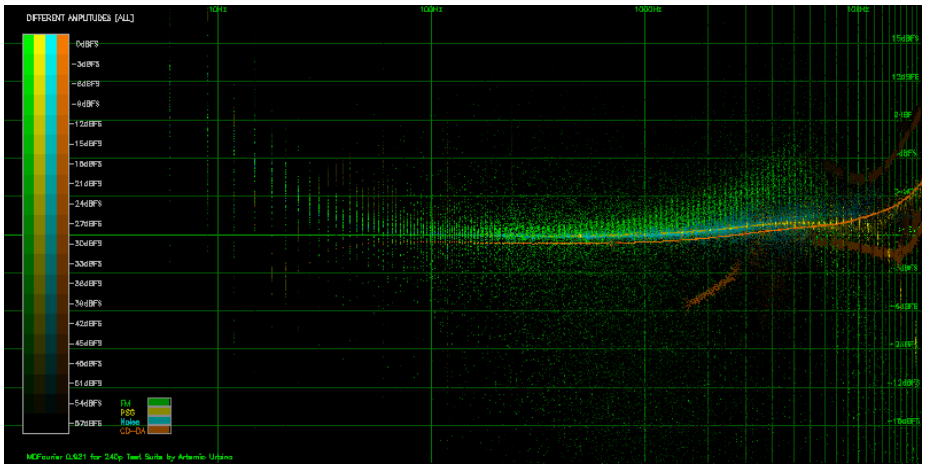


Figure 5.19 Differences between *MD1UTVA3-SCD* and the *MD2UCVA18-SCD*.

It can be appreciated that the *CDDA* mix has a lower low pass filter in the Model 2 unit, starting at around *3kHz*. For clarity, Figure 5.20 shows the results from the comparison just for *CDDA*.

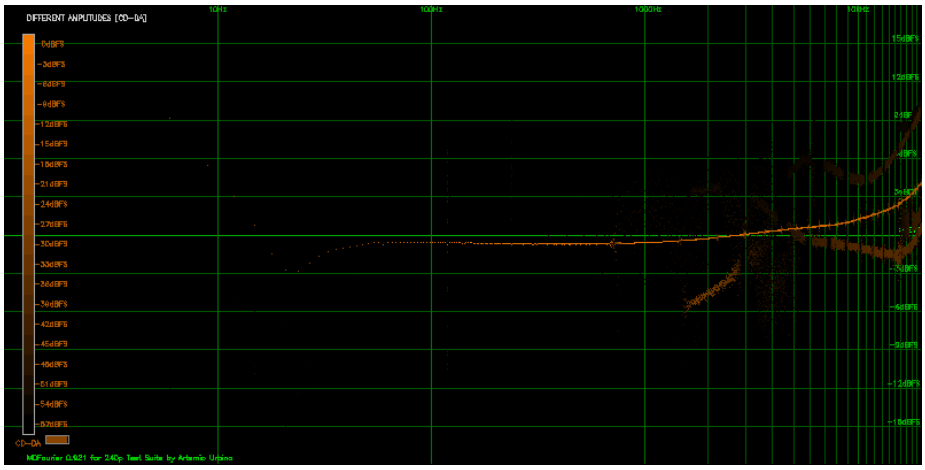


Figure 5.20 Differences between *CDDA* mixing between a *Sega Genesis Model 1 VA3* with *Sega CD* and a *Sega Genesis Model 2 VA1.8* with *Sega CD*.

There are also other lines present in the results, which are probably the result of different harmonics that are created during the filtering and mixing process.

Chapter 6

Conclusions

So far *MDFourier* has shown results that are consistent with previous empirical evaluations, and can support many such claims with data. I hope it can provide the community with a process that is repeatable and objective that can be used in any project where someone wants to preserve, compare, replicate or play with the results.

I believe these cases support the idea that this process has value, and that it can be replicated in as many platforms as possible. Audio has been historically relegated to a secondary position, when the audio visual experience is one–as a whole. Games are not the same without their acoustic experience.

This could help to document the original audio signatures of old systems. And of course, to enable the community to play with variations and modifications that are as valid as the ones the systems originally had at launch.

Appendices

Appendix A

Downloads

The project web page is available at `http://junkerhq.net/MDFourier/`, where the latest version of this document is available in *PDF* and *HTML*.

The *Downloads* page contains the following items:

- A pre-compiled *Microsoft Windows* executable of the latest build with *GUI* for the *Analysis Software*.
- The *Tone Generator* software for each supported platform.
- Updated source code for all tools.
- Sample recordings used for the creation of this document as detailed in chapter 5.

Appendix B

How to use the *Analysis Software*

The current version of the *Analysis Software* allows access to the main options of *MDFourier*. It is a *Windows* executable and all corresponding files must be placed in the same folder. *Uncompressing* the package should have all that is necessary to run the program.

After executing *MDFourierGUI.exe* you should be presented with the following interface:

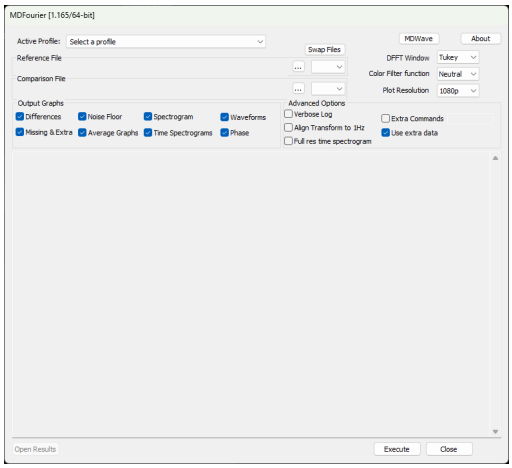


Figure B.1 *MDFourier* Windows Front End for the *Analysis Software*.

In order to generate the output graphs, two files must be selected to compare them. One as a *Reference* and the other as the *Comparison* file, as detailed in section 3. You can also drag & drop them into the window, sequentially or as a pair.

The following sequence of steps indicates the typical work flow within the *GUI*:

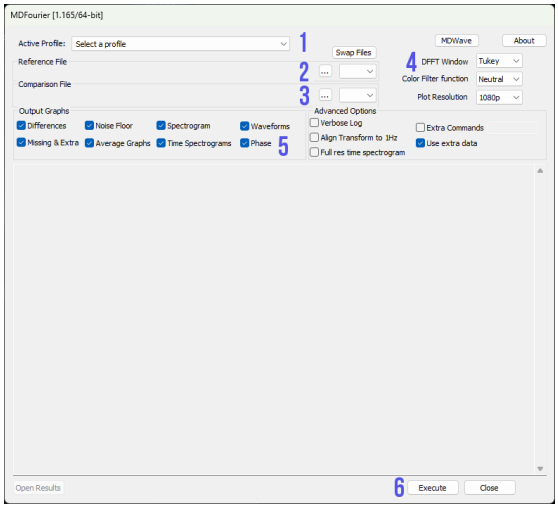


Figure B.2 Typical sequence of steps.

1. Select a *Profile* for the analysis.
2. Select a *Reference* file and its video format, as defined in the profile (for example *NTSC* or *PAL*).
3. Select a *Comparison* file and its video format.
4. Change the default *options* if needed.
5. Select the desired output *graphs*.
6. Execute *MDFourier*.
7. When execution ends, open the *results folder*.

The default options will generate graphs that work on most situations. In some cases fine tuning the results might highlight specific aspects. These options will be described in the following sections.

The *Analysis Software* will display the output text from the command line tool, including any errors or progress as it becomes available.

Keep in mind that the *Open Results* button will only be enabled after a successful comparison between files has finished, and it won't open a second instance of the window if you have one already present.

B.1 Analysis Software Options

The currently available options in the *Analysis Software* are:

B.1.1 Window Functions

In order to reduce *spectral leakage*[1] when applying the *DFT*, a *filtering window*[2] is applied to each element to be compared between both signals. Since we are producing the sound ourselves from the *Tone Generator*, the signal can be analyzed as *periodic*,[3] and has a natural *attack* and *decay* rate if possible.

By default we use a custom *Tukey window* with very steep slopes.[4] *MD-Fourier* does offer alternate windows as options for further analysis.

B.1.2 Color Filter Functions

Each dot in the *Differences* graph uses the *X axis* for the frequency range and the *Y axis* for the amplitude difference between the *Comparison* and *Reference* signals.[5]

Color intensity of each dot is used to represent the amplitude for that frequency in the *Reference* signal. In other words, how relevant it was to create the original signal in that note. Please refer to chapter 4 in order to see examples of their use.

A color scale is presented in each graph, with the color graduation and the corresponding amplitude level.

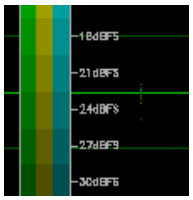


Figure B.3 Detail of color scale in graph.

The options are useful to *highlight* or *attenuate* these differences by applying the range to one of the following functions.

They are sorted in descending order. The topmost option will highlight all differences; and the bottom one will attenuate most of them, and show just the ones with highest amplitudes in the *Reference* signal.

All filters, their graphs and effects are listed in appendix O. From the com-

[1] This term describes new frequency components created when applying a Fourier Transform to a segment of a signal. See [22].

[2] All details regarding the windows used, their formulas and graphs are in appendix N.

[3] In theory this means we could get away without applying a window, but in practice the signal is not perfect and *windowing* the data helps to eliminate noise.

[4] This is not a common choice for window function, but was selected due to the above reasons after comparing results with the rest of the available options.

[5] See section B.1.4 for output file details.

mand line they are selected with *-o*, from *0* for *None* to *5* for *Dimm*, in the order of that appendix. The default is *3*, *Neutral*.

B.1.3 Align Transform to 1Hz

When designing the *audio test signal* for use during analysis, one consideration is how to balance gathering more information versus the length of each recorded audio signal.

Having a short audio test signal is desirable for several reasons: it reduces the time taken to record from each system, minimizes the storage space required for archival, the analysis software is much faster with shorter archives, and it also makes distributing the audio recordings easier, due to a smaller resulting file size.

In contrast, when applying the *Fourier transform* a compromise is made between frequency detail and time accuracy, very similar to the Heisenberg's uncertainty principle. If we compare a longer signal for each element, we end up with more frequency information. In our case, we don't care much about time accuracy but we do care about the length of the test. Nobody wants to record a 5 or 10 minute signal for each test to be made.

The compromise made is to use *sub second signals* for each element to be compared. Since the time and sample rate determine how the *DFT frequency bins*[6] are spaced after analysis–and how much information we end up analyzing– the end result is a lower graph resolution.

Zero padding the input signal for Fourier analysis is a controversial subject[7], but for the present application no adverse effects have been found. This might be the case since we control how the source signal is generated, with a predetermined *attack* and *decay*. However, it is disabled by default so the results presented can be free from questioning regarding the effects, and available for cases where more precise frequency information is needed with no spectral leakage to adjacent *frequency bins*. From the command line it is enabled with *-z*, and it is always enabled when clocks are adjusted with *-j*.[8]

When the blocks in a profile have different durations, the shorter ones are padded to the length of the longest, so that the amplitudes are comparable between all blocks.

[6] The size of each slice in Hertz for the frequency spectrum.
[7] Zero-padding a signal does not reveal more information about the spectrum. See [23] [24].
[8] See appendix D.

B.1.4 Output Graphs

Several graphs will be generated as a result of the analysis, in the *output folder.*[9] There is one graph for each *type* of block, and one named *ALL* that summarizes all types. When a type is *stereo*, graphs for the left and right channels are created as well, ending in *_l* and *_r*.

The general graphs are saved in the output folder itself, and the ones for each type are stored in subfolders: *Difference*, *MissingAndExtra*, *Spectrograms*, *TimeSpectrograms*, *Phase*, *NoiseFloor*, *Waveforms* and *CLK*.

All graphs are saved in the *PNG*[10] format, which is lossless and open source. Please read section 4 for examples and a guide to interpret their meaning. In the following file names, *A* stands for the *Reference* file and *B* for the *Comparison* file.

Different Amplitudes

Enabling this option creates the most relevant output graphs from *MDFourier*. These contain the amplitude difference for the frequencies common to both files across the hearing spectrum using the *Reference* file as control. It is disabled with *-D*.

If the files are identical, the graph will be a perfect line across the *0dB* line. In case the signals differ, a scatter graph will show how it behaves across the typical human hearing spectrum.[11]

The bars at the bottom show, for each type, the percentage of compared frequencies with any amplitude difference, those within *1dB*,[12] the perfect matches, and the *missing* and *extra* frequencies.

The file names for these graphs have the format:
`DA__ALL_Reference_vs_Comparison.png` and `DA_Reference_vs_Comparison_#Type.png`

Missing and Extra Frequencies

Graphs the frequencies available in the *Reference* file but not found in the *Comparison* file within the significant amplitude range, and the frequencies only present in the *Comparison* file. They are drawn over time, as a *time spectrogram*. It is disabled with *-M*.

The file names for these graphs have the format:

[9] It is named *MDFResults*, with a subfolder for the profile and another one named as the compared files. It can be accessed from the *Analysis Software* with the *Open Results* button.
[10] *Portable Network Graphic* support via *libpng* [9].
[11] Typically defined as *20-20000Hz*, but the higher end tends to be lost with age.
[12] The range can be changed with *-b*.

`MISSING-A-T_SP_Reference_channel.png` and `MISSING-EXTRA_T_SP_Comparison_channel.png`

Average Graphs

This option traces an *average line* on top of the *Difference* graphs, making it easier to follow the trend when the output has severely scattered data. It is *on by default*, and can be disabled with *-g*.

The curve is created by averaging time segments from the frequency data sorted in ascending order. A *Simple Moving Average*[13] is then calculated to smooth out the results.

The curve is weighted according to the *Color Filter Functions* described in section B.1.2, by repeating each data point by the amount mapped in the *0-1* interval described by the function. The *-A* option creates the averages without weighting.

As a result, the average will follow the relative amplitudes from the *Reference* signal proportional to the selected *filter function*.

The *-G* option moves the whole *Difference* graph so that its average difference is at *0dB*. This helps when both signals differ by a constant level, and the original average is noted in the graph.

The file names for these graphs have the format:
`DA__ALL_AVG_Reference_vs_Comparison.png` and `DA_Reference_vs_Comparison_#Type_AVG.png`

Noise Floor

This option generates a noise floor spectrum for each file, and an averaged comparison graph of both. They are plotted with the same scale, so they make sense when compared. It is disabled with *-F*. When one of the files has *digital silence*, only the noise floor of the other file is drawn, since there is nothing to compare.[14]

The file names for these graphs have the format:
`NF_SP_A_Reference_-1Silence.png`, `NF_SP_B_Comparison_-1Silence.png` and `NF__Reference_vs_Comparison_-1Silence_AVG_.png`

[13] This is a type of average that is intended to smooth out sudden peaks in the data. Often used in exchange rate graphs. See [38].

[14] See section 3.5.

Spectra

Plots all the frequencies available in each file.[15] Two sets of spectra are generated, one for the *Reference* file and one for the *Comparison* file. It is disabled with *-S*. Each one shows the amplitude of every frequency of a type, from all its blocks together, with no time axis; the *Time Spectrograms* are the ones that show them over time. These graphs are saved in the *Spectrograms* folder, which keeps its name.

The file names for these graphs have the format:
`SP__ALL_A_Reference.png` and `SP_A_Reference_#Type.png`

Time Spectrograms

Plots the frequencies of each file across time, block after block, with the amplitude as color. It is disabled with *-t*. By default the frequency range is limited to the analyzed frequencies, *-E* draws the full range and *-8* uses a linear scale.

The file names for these graphs have the format:
`T_SP_A_Reference.png` and `T_SP_#_Type_A_Reference.png`

Phase

Plots the phase difference between the *Comparison* and the *Reference* for the tones common to both files. A frequency is used when it is at least *20dB* above the *minimum significant amplitude* and no louder frequency is next to it, since the sides of each peak and the noise have no stable phase. It is disabled with *-O*.

A time offset between the files adds a phase difference that grows with frequency and hides everything else. It comes from the precision of the sync and from when each sound source starts: a *DMA* transfer can start later than an *FM* note, for example. So a delay is estimated for each type, the one that best lines up its tones, and removed before plotting.

With *-v* the log lists each delay with how well it fits, from 0 to 1, for example *Phase [FM]: Comparison 23.78 us earlier than the Reference, removed (2175 tones, fit 0.96)*. Below *0.5* the phase of that type is not reliable.

The file names for these graphs have the format:
`PHASE_DIFF__ALL_Reference_vs_Comparison.png` and `PHASE_DIFF_Reference_vs_Comparison_#Type.png`

[15] As limited by the analysis parameters, such as amplitude and frequency count per block.

Waveforms

Plots the samples of selected blocks from both files. By default these are the *sync* pulses and the blocks of type *time domain* in the profile. It is disabled with *-Q*.

The *-u* option creates them for every block: *-uu* plots the samples after applying the window used for the *DFT*, and *-uuu* creates both. *-uuuu* does the same, and shows the zero padding as used, without clipping it when it is present in both files. The *-H* option creates them only for the blocks with a high percentage of differences, for example *-H 5* for those with more than *5%*. The vertical scale can be zoomed with *-a*.

The file names for these graphs have the format:
`TD_###_Type_###_Reference_vs_Comparison.png`

Clocks

When the profile measures *clocks*,[16] a graph of the clock tone is created for each clock and file in the *CLK* folder. It shows *100Hz* to each side of the tone, centered on the *Reference* peak so both files can be compared at the same position. If the tone was not found in the *Reference*, it is centered on the *Comparison* peak, or on the expected frequency when neither has it. The vertical scale is in *dBr*, relative to the loudest tone of that file, at least *100dB* deep, with a line at the *minimal significant amplitude*. When the tone was not found, the graph says so and only the noise is shown. The name of the clock and its identity are in the title.

The file names for these graphs have the format:
`CLK_01_Name_A_Reference.png`, where the number is the order of the clock in the profile, and *A* or *B* is the *Reference* or *Comparison*.

Channel Separation

When the profile has *channel separation* blocks and both files are stereo, a graph summarizes the results for each of those blocks.[17] Each block has a pair of bars, one for the *Reference* and one for the *Comparison*, labeled below each bar, with the separation in *dB* and the active channel on top. Higher is better. A hollow bar means the leak into the silent channel is below its noise floor, so the value is only a minimum. When the tone plays in both channels, the level difference between left and right and their phase difference are shown instead of bars.

Below each block are its name and its state: the active channel and frequency, *L/R inverted* when the tone is in opposite channels in each file, or *channels differ*

[16] See appendix D.
[17] See appendix H.

when one file plays it in both channels and the other in one. A warning is drawn when the channels are inverted.

The file names for these graphs have the format:
`SEP_Reference_vs_Comparison.png`

Other output

The log file is saved next to the graphs as `Log_Reference_vs_Comparison.txt`, unless *-l* is used. The *-C* option creates a *CSV* file with the values of the *Difference* graphs, and *-7* plots every window used for the *DFT*.

The appearance of all graphs can be changed with these options: *-W* for a white background, *-L* for the resolution, *-d* for the vertical range in *dB*, *-N* for a linear frequency scale, *-J* to limit the horizontal range to the analyzed frequencies, and *-q* to draw a pair of bisection lines at a given frequency and amplitude.

B.1.5 Extra Command

This checkbox enables the text field to send any extra commands that are not available via the *GUI* to *MDFourier*.

Current options as of version *1.20* are:

```
MDFourier 1.20 240p Test Suite Fourier Audio compare tool]
usage: mdfourier -P profile.mdf -r reference.wav -c compare.wav
 FFT and Analysis options:
  -w: enable <w>indowing. Default is a custom Tukey window.
     'n' none, 't' Tukey, 'h' Hann, 'f' FlatTop & 'm' Hamming
  -f: Change the number of analyzed frequencies to use from FFTW
  -s: Defines <s>tart of the frequency range to compare with FFT
  -e: Defines <e>nd of the frequency range to compare with FFT
  -i: <i>gnores the silence block noise floor if present
  -z: Uses <z>ero Padding to equal 1 Hz FFT bins
  -n: <N>ormalize:
     'f' Frequency Domain Max, 't' Time Domain, 'a' Average
     'n' No normalization
  -B: Do not do stereo channel audio <B>alancing
  -I: <I>gnore frame rate difference for analysis
  -p: Define the noise floor value in dBr
      (0 to disable auto adjust)
  -T: Increase Sync detection <T>olerance (ignore frequency, can be added up to -TTT)
      -TTTT also accepts pulses of half a frame (emulated audio)
  -Y: Define the Reference Video Format from the profile
  -Z: Define the Comparison Video Format from the profile
  -m: Set <m>anual sync samples,
```

```
     takes format [r|c]:<start>:<end>
    where r is for reference and c is for comparison
    example: -m r:64001:12236801
    sample values must be the start of each pulse sequence
 -q: Set bisection lines to pinpoint in plots,
     takes format <hertz>:<amplitude>
    example: -q 1825.26:3.75
 -R: Adjust sample <R>ate if duration difference is found
    'r' Reference, 'c' Comparison or 'b' Both
 -j: Ad<j>ust clock (profile defined) via FFTW
     if difference is found
 -k: cloc<k> FFTW operations
 -X: Do not use E<x>tra Data from the Profile
Output options:
 -l: Do not <l>og output to file [reference]_vs_[compare].txt
 -v: Enable <v>erbose mode, spits all the FFTW results
 -C: Create <C>SV file with plot values.
 -b: Change <b>ar value for frequency match, default is 1.0dB.
 -A: Do not weight values in <A>veraged Plot (implies -g)
 -G: Adjust difference plots around avera<G>e difference.
 -W: Use <W>hite background for plots.
 -d: Max <d>B for difference plots vertically
 -a: Zoom in to dB in waveform plots
 -L: Plot resolution:
    0: 800x400   1: 1200x600  2: 1800x900 3: 1920x1080
    4: 3600x1800 5: 3840x2160 6: 7680x4320
    7: 15360x8640 8: 30720x17280 9: 61440x34560
 -D: Don't create <D>ifferences Plots
 -g: Don't create avera<g>e points over the plotted graphs
 -M: Don't create <M>issing Plots
 -S: Don't create <S>pectrum Plots
 -F: Don't create Noise <F>loor Plots
 -t: Don't create Time Spectrogram Plots
 -O: Don't create Phase Pl<O>ts
 -Q: Don't create Time Domain Plots
 -H: Output waveform plots for <H>ighly different notes, takes
     a value in percentage
 -o: Define the output filter function for color weights [0-5]
 -u: Create waveform plots for all notes
      -uu:  Create DFT windowed waveform plots for all notes
      -uuu: Create waveform plots for all notes,
            including DFT windows
      -uuuu: Same as -uuu, showing the real zero padding
 -E: Defines Full frequency rang<E> for Time Spectrogram plots
 -N: Use li<N>ear scale instead of logarithmic scale for plots
 -8: Use linear scale for Time Spectrogram plots
 -x: (text) Enables e<x>tended log results.
     Shows a table with matches
 -O: Change output folder
 -y: Output debug Sync pulse detection algorithm information
```

```
-7: Creates plots for all windows used
-9: Debug, flattens the profile into one block per type
-J: Limits plot horizontal display
```

Sending *-h* in this field will enlist all the currently supported options for your version.

B.1.6 Verbose Log

A log is always created by default, unless *-l* is used. This option, *-v* from the command line, enables a verbose version with the whole frequency analysis and many other details dumped to the file.

Useful for reporting errors or unexpected behavior. *Please send the audio files if possible as well!*[18]

[18] Contact details are available in appendix T.

Appendix C

Configuration file

All these parameters are defined in the *mfn* configuration files inside the *profiles* folder.[1] The current format is version *2.3*. Here is the profile used to compare *Mega Drive/Genesis* audio characteristics:

```
MDFourierAudioBlockFile 2.3
SEGA: Mega Drive/Sega Genesis
[VideoModes] 2
NTSC 16.6884 262 8820 14 10
PAL  20.1205 313 8820 17 10
CLK n
[MonoBalanceBlock] 99      ;Use 1khz-ish, SPSG# 1
[ToneLines] 10
Sync            s   1  20  0 red     m
WaitDecay       k   1   5  0 red     m
Silence         n   1  15  0 gray    m
FM              1  96  20  0 green   S *
SPSG            2  40  20 -1 yellow  m *
SPSG_Ramp       4  20  20  0 yellow  m *
Noise           3  16  20 -1 aqua    m
WaitDecay       k   1   5  0 red     m
Silence         n   1  15  1 gray    m
Sync            s   1  20  0 red     m
```

This file defines what *MDFourier* must do and how to interpret the audio files. Anything after the expected values in a line is ignored, so comments can be added after a semicolon as shown above.

[1] Different profiles can be used and selected via the *Analysis Software* or with the *-P* command line option. There is at least one profile for each supported hardware platform.

C.1 Header and name

The first line is the header, so that the program knows it is a valid file and in the current format. Profiles from older versions must be updated.

The second line is the name of the profile. It is shown in the graphs, and it is used to name the results folder.

C.2 Video modes

The *[VideoModes]* line indicates how many video modes follow, each one in its own line. A mode is selected for each file from the *GUI* or with *-Y* and *-Z*, by number or by name. The first one is used by default. Each line has these values:

- **Name:** For example *NTSC* or *PAL*.
- **Frame duration:** The expected duration of a frame in milliseconds.[2] This is only used as a reference to estimate the placement for the blocks within the file before calculating the precise *frame rate* of the recording as captured by the *audio interface*. After that is calculated, each signal uses its own timing in order to be fully aligned. *Frame rate* variations in the order of *0.001 ms* are natural, since we have an error of 1/4 of a millisecond[3] during alignment, and differences also occur by the deviations in *sample rates* and audio interface limitations[4].
- **Line count:** The number of scan lines per frame. It is used to predict the frequency of the *horizontal scan* noise.
- **Pulse frequency:** The frequency of the *sync pulse* tone in *Hz*.
- **Pulse length:** The length of each pulse, used while detecting the pulse train.
- **Pulse count:** The number of pulses in the train.

C.3 Clocks

The lines between the video modes and *[MonoBalanceBlock]* define the *clocks* to be measured. Each one can be followed by a *[CLKID]* section with the known values for that clock. A profile that doesn't measure clocks has a single disabled line, *CLK n*, as shown above. Clocks are described in detail in appendix D.

[2] For more details regarding frame rate, see appendix G.

[3] For reference 1/4 of a millisecond corresponds to 0.00025 seconds. Modern systems have different frame rates adapted for modern displays, small differences are more likely caused by the audio capture hardware [18].

[4] See appendix P.1.

C.4 Mono balance block

The *[MonoBalanceBlock]* line indicates which block is used to detect and compensate *stereo imbalance.*[5] It is the number of the block counting from zero, and it must be a *mono* block. If it is *0*, the first mono block in the profile is used.

C.5 Tone lines

The *[ToneLines]* line indicates how many block lines follow. Each line has these values:

- **Name:** The name of the block, used in the graphs and file names. Underscores are shown as spaces.
- **Type:** A number for the blocks that are analyzed, or a letter for the special types listed below. Blocks with the same number are compared and graphed together.
- **Element count:** How many elements of the same duration the block has, for example one for each note.
- **Frames:** The duration of each element in frames.
- **Cut frames:** The number of frames at the end of each element that are excluded from the analysis, for example to leave out the decay of a note. Negative values are accepted and treated as positive.
- **Color:** The color used in the graphs for this type.[6]
- **Channel:** *m* for *mono*, *S* for *stereo*, *s* for *stereo* that can also be compared against a *mono* recording, *n* for *noise*, and *C* for a *stereo* block that measures *channel separation.*[7] Stereo types are analyzed per channel.[8]
- **Mask (optional):** -, or no value, is the default: each file is analyzed with the full duration of the block at its own *frame rate*, and a window of that length. With ***, the file with the longer frames is cut to the duration the block has at the shorter frame duration, and the window for that shorter duration is used, so both files analyze the same amount of audio. This was the original behavior, and it is used by the *Mega Drive/Genesis*, *Sega CD*, *MSX* and *MS-DOS* profiles.
- **Flags (optional):** Letters after the mask.
  - *x* analyzes and plots the block but leaves it out of the comparison, for example the blocks that hold clock tones.

[5] See appendix J.
[6] Available colors are listed in appendix R.
[7] See appendix H.
[8] This is important for stereo audio balancing, see appendix J.

  - *b* compares a noise block by its averaged spectrum in bands instead of by its frequencies, as described in appendix I.

- A semicolon starts a comment.

For example, FM audio has been named *"FM"*, type *1*, *96* elements of *20* frames each and will be colored in *green*. Definition is in frames since some hardware variants, emulators and *FPGA* implementations sometimes run at different frame rates than the vintage retail platform, which result in different durations. We align them with the video signal and gain information along the way, which also helps when comparing systems from different regions that have their own quirks.

When a type number is used again in a later line, that block is treated as *extra data* of the first one. It is graphed with the same type and must use the same color. Extra data can be ignored with *-X*.

These are the special types:

- **s:** A *sync pulse* train, used to automatically recognize the starting and ending points of the signal inside the file. There must be exactly two: one at the begining and one at the end.

- **n:** Null audio, or *silence*. This *silence* is used to measure the *background noise*[9] as recorded by the audio interface. We recommend only marking the starting and ending ones, so that no decay noise from the test patterns is used. We have traditionally used 20 frames for each, but longer gives you more precision.

- **N:** A *silence* block that overrides the regular ones. When present, it is used to measure the noise floor and the *n* blocks are skipped.

- **k:** A block that is skipped. It is used for waits between sections, or for audio that should not be compared.

- **t:** A *time domain* block. It is not compared in the frequency domain, but its waveform is plotted for both files for visual analysis or future expansions.

- **i** and **I:** *Internal sync*. Used when part of the signal starts at a variable time, such as *CD-ROM* playback. A pair of these lines encloses the blocks that follow the internal sync. These lines have no *cut frames* value, and add two values after the channel: the frequency of the sync tone in *Hz* and its length in seconds. The closing line uses *0 0*. Half of the sync length is the tone, and half is the silence that follows it. Both types search for the tone from the start of the block, and fail if the tone found is shorter than *90%* of its expected length, unless *-T* is used. The audio starts after the full sync length, counted from the start of the tone.

[9] How analysis is affected by this is described in section 3.5.

- *i* moves only the blocks enclosed by the pair to the detected position. The blocks after the closing line keep their positions from the main sync, so the variable delay must fit within the frames defined for the enclosed section.
- *I* takes the tone as exactly half the sync length. If a longer tone is detected, the start is moved forward by the difference. It then moves all the remaining blocks of the profile, up to its end, to the detected position.

Here is a fragment of the *Sega CD* profile with *skip* and *internal sync* blocks:

```
Wait            k   1   4  0 red     m
PCMSTART        k   1   2  0 red     m
PCM             4  28   5  0 magenta m
PCMEND          k   1   2  0 red     m
CD-Paused       i   1   2    red     m 8000 0.300
CD-DA           5 128   5  0 orange  S
CD-Paused       i   1   2    red     m 0 0
```

The detected delays are reported in the log as the command delay, in milliseconds and frames.

- **W:** A *watermark* block, used to verify a setting from the recording, such as the console's audio mode. It has no *cut frames* value, and adds three values after the channel: the frequency of the valid tone, the frequency of the invalid tone and the name of the setting, for example *TRIANGID W 1 2 red m 1000 120 Phase*. A warning is shown in the graphs when the invalid tone, or neither, is found. You can view the *NES* or *MS-DOS* profiles for reference.

  This fragment from the *MS-DOS* profile has a watermark for the MIXER values being set to a known state with 1khz or an unknown state with 2khz.

```
MIXER           W   1  10    salmon  m 1000 2000 Mixer
```

  A second pair of tones can follow the name, with the name of an alternate condition the recording was made under. The valid tone of that pair means the setting was right under that condition, and the invalid one that it was not. The *MS-DOS* profile uses it for recordings made from a *Windows DOS* box, at *1.5kHz* and *2.5kHz*:

```
MIXER           W   1  10    salmon  m 1000 2000 Mixer 1500 2500 Windows
```

  These are reported as a note, or as a warning when the setting is invalid, both in the log and in the graphs:

```
- NOTE: Comparison signal was recorded under Windows. Results are probably incorrect
- WARNING: Reference signal was recorded under Windows with Mixer difference.
  Results are probably incorrect.
```

Each tone is accepted within *100Hz* of its frequency, and only when it is within *20dB* of the strongest tone in the block. A profile with a single pair works as before.

C.6 No Sync profiles

Files without *sync pulses* can be compared with a *No Sync* profile, which has a simpler format:

```
MDFourierNoSyncProfile 2.3
Synthetic test
16.6667
16.6667
D
2
Silence         n   1  30    gray    m
Tone            1  60  20    green   s
```

The third and fourth lines are the frame durations for the *Reference* and *Comparison* files. The fifth line selects how the start of the signal is found:

- **A:** Automatic, the start is detected from the audio.
- **M:** Manual, both files must be trimmed identically. The *Comparison* frame rate is estimated from its length.
- **D:** Digital, the signal starts at the first sample that is not zero. Used for recordings from digital sources.
- **L:** Length, the file is used from its first sample for the duration of the profile, and no normalization is applied.

The sixth line is the number of blocks, and each block line has a *name*, *type*, *element count*, *frames*, *color* and *channel*.

Appendix D

Clock measurement and adjustment

The sound chips in a system are driven by a *clock*, a crystal or oscillator that sets the pace for everything they produce. When two systems use clocks that differ slightly, every frequency they generate is shifted by the same ratio. The notes are at a slightly different pitch, and the comparison shows it as differences that have nothing to do with the analogue parts. It must be noted that these are real differences in how the systems sound, and the pitch difference is measured and displayed by the software.

MDFourier can measure each clock from a known tone, identify the chip or card that produced it, and correct the signal for each clock separately. The feature was created for the *Super Nintendo*, and later extended for systems with several clocks, such as *MS-DOS* sound cards.

The audio unit of the *Super Nintendo* is driven by a *ceramic resonator* instead of a quartz crystal. Ceramic resonators are cheaper but less stable: their frequency changes with temperature, so the pitch of the console drifts slightly with the temperature of the room and with how long it has been on.[5] The change is small, but *MDFourier* is sensitive enough to show it. While the *Super Nintendo* profiles were being developed, no two recordings matched, not even two recordings of the same console made at different times. This also happens with *S/PDIF* mods, which report a fixed *32kHz sample rate*, but whose real rate follows the ceramic resonator frequency.

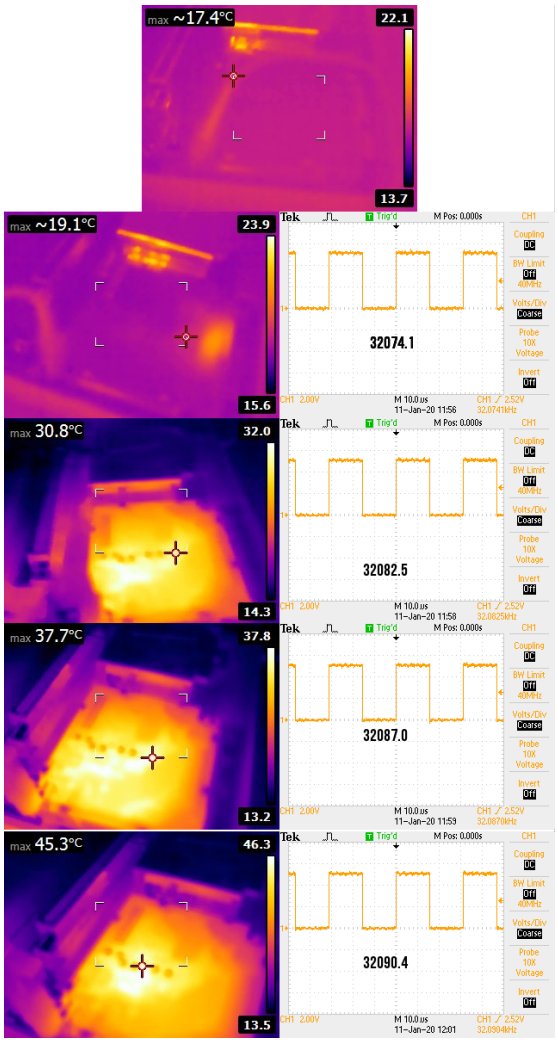


Figure D.1 Ceramic resonator as measured with a scope from a *Super Nintendo* along the thermal photos taken with each measurement.

The emulation community had found this many years before. The audio of the *Super Nintendo* is specified to run at *32kHz*, but real consoles always run slightly faster, since the resonator is never as cold as the reference temperature. Some games did not work correctly when emulated at the exact rate, and emulators settled on a value a little above *32kHz*.

Since keeping the temperature constant while recording would require a controlled environment, the solution was to measure the clock instead. The *Super Nintendo Tone Generator* plays a *8kHz* tone, and *MDFourier* measures how far from *8kHz* it really is. Multiplied by *4*, that gives the rate at which the audio unit of that console was running at the time of the recording, and the *Comparison* can be corrected to match the pitch of the *Reference* with very precise and reliable results. This is the clock line in the *Super Nintendo* profile:

```
APU y 54 8000 4
```

In this context, the *ceramic resonator* turned from a problem into a measurement: the clock reported by *MDFourier* shows how fast that console was running when it was recorded, which depends on its temperature at the time.

D.1 Clock lines

Each clock is defined by a line between the video modes and *[MonoBalanceBlock]* in the profile.[1] The *Super Nintendo* line shown above measures the tone in block *54*, expected at *8000Hz*, and multiplies it by *4*. Since it lists no types, it applies to all of them.

This is the beginning of the *MS-DOS* profile, with four clocks: the *FM* chip, *8 bit* mono and stereo *PCM*, and *16 bit PCM*. The last two are *optional*, since older cards can't play them:

```
FM y 2 6002.078595 8.283114 1 2 3 4 5 6 7 8
[CLKID] 4
OPL2/3      49715.9028   500ppm
CQM         49804.4221   500ppm
ESFM        49624.7075   500ppm
OPL3-SAx    49515.7895   500ppm
PCM y 305 2777.7777778 8.0 9 10 11
[CLKID] 5
Nominal     22222.2222   500ppm
DSP2        21739.1304   500ppm
SB16        22050.0      500ppm
OPL3-SAx    22138.1      300ppm
ESS         22158.0      500ppm
PCMS o 381 2756.25 8.0 13 14
[CLKID] 4
SB16        22050.0      500ppm
SBPro       21739.1304   500ppm
ESS         21676.3131   300ppm
OPL3-SAx    21656.8490   300ppm
PCM16 o 386 5512.5 8.0 15 16 17 18
[CLKID] 2
SB16        44100.0      85ppm
ESS         44107.6      85ppm
```

The blocks with the clock tones are marked with the *x* flag, so they are measured but not compared:

[1] See appendix C.

```
CHIPID          1   1  60  0 orange  m - x
PCM_CLK         9   1  60  3 indigo  m - x
PCM8_CLK       13   1  60  3 blue    s - x
PCM16_CLK      15   1  60  3 purple  s - x
```

Each line has these values:

- **Name:** Up to 19 characters. It must be unique, since it is used in labels and file names.
- **Mode:** *y* to measure the clock, *n* to disable the line, and *o* for an *optional* clock as described below.
- **Block:** The number of the block, counting from zero, that has the clock tone.
- **Frequency:** The expected frequency of the tone in *Hz*.
- **Ratio:** The value that converts the tone frequency to the clock. The measured clock is the detected tone frequency multiplied by this ratio.
- **Types (optional):** The block types that run from this clock. If none are listed, or * is used, the clock applies to all types not listed by another clock.

Up to *8* clocks can be defined. When the clocks are adjusted with *-j*, or when there are *optional* clocks, the lists are checked when loading the profile: each listed type must exist, a type can only belong to one clock, only one clock can apply to all types, and the block with each clock tone must belong to that clock.

D.2 Measurement

The clock tone is analyzed with its own *DFT*: a *Hamming* window with zero padding to $1/16Hz$ bins, and the peak is interpolated between bins. This gives a precision of a fraction of a *part per million* (*ppm*).

The tone is accepted when the strongest frequency within *5%* of the expected one is found, and it is above the *minimal significant amplitude* relative to the strongest frequency of that file. Otherwise it is reported as *not found*: the hardware under analysis might not produce that audio, or the block might only have noise. A warning is also shown when a harmonic or noise is stronger than the tone.

For each clock, the log reports the tone and clock for both files, and the difference in *cents*:[2]

[2] A *cent* is $1/100$ of a semitone. A difference of *1* cent is about *578ppm*.

```
* Estimated FM Clocks based on expected 6002.0786 Hz
  on note CHIPID# 0:
 - Reference: 5991.0682Hz -> 49624.7005Hz
 - Comparison: 6001.4791Hz -> 49710.9359Hz
 - Reference identified as: ESFM [-0.14 ppm from 49624.7075Hz]
 - Comparison identified as: OPL2/3 [-99.91 ppm from 49715.9028Hz]
 - Pitch difference in cents: -3.00584
```

When the clocks differ by *0.08* cents or more, a warning is shown in the log and in the graphs.

D.3 Identification

A *[CLKID]* section after a clock line lists known values for that clock. The first line has the number of entries, and each entry has a *name*, the *nominal* clock in *Hz* and a *tolerance*. The tolerance can be in *ppm*, in cents (*c* or *cents*) or in *Hz*, for example *250ppm*, *0.5c* or *10Hz*.

The measured clock of each file is matched against these ranges. When it is within one, it is reported by name, along with how far it is from the nominal value. When it doesn't match any, it is reported as *Unknown* along with the nearest entry. A warning is shown when loading a profile with overlapping ranges.

D.4 Adjusting clocks with *-j*

The *-j* option corrects the clock differences for analysis. This is useful when the differences need to be verified as if the signals were being played using the same clock, similar to using auto-tune to match the *Reference*. For each clock that differs from the *Reference*, the *Comparison* blocks of the types that belong to that clock are analyzed again with a corrected sample rate, so that their frequencies line up with the *Reference*. The rest of the blocks, the frame rate and the waveforms are not changed.

When *-j* is used, all blocks are analyzed with zero padding to *1Hz* bins, as with *-z*, so that the corrected frequencies of both files fall on the same bins. A note is drawn in the graphs for each adjusted clock. A clock is not adjusted when the difference is below *0.08* cents, or when its tone was not found in one of the files.

D.5 Optional clocks

Some systems cannot produce all the audio in a test, for example *16 bit PCM* in an older sound card. When the clock is marked with *o*, and its tone is not found in either file, all the block types in its list are skipped from the comparison. This keeps a silent section from affecting the normalization and the results.

An *optional* clock must list its types. Keep in mind that skipping works by type, across the whole profile.

D.6 Clock data in the graphs

The clock data is drawn in the upper left area of the graphs, one group for each clock:

```
FM: -3.01c adjusted (-j)
  RF 49624.70Hz [ESFM]
  CM 49710.94->49624.70Hz [OPL2/3]
PCM16: tone not found in RF (skipped)
  RF not found
  CM 44103.06Hz [SB16]
```

The first line has the name of the clock and its state. It is yellow when the clocks differ or the tone was not found. The following lines have the measured clock and identity for each file. A graph of each clock tone is saved in the *CLK* folder.[3]

[3] See section B.1.4.

Appendix E

MDWave

MDWave is a companion tool to *MDFourier*. During development and while learning about *DSP*, I needed to check what I was doing in a more tangible way. So, in order to visualize the files in an audio editor and listen to the results *MDWave* was born.

It takes a single audio file as argument, and loads all the parameters defined in the configuration file in order to verify the same environment.[1] To run it, simply press the button next to the selected profile, or use *mdwave -P profile.mfn -r file.wav*.

The output is stored under the folder *MDWResults*, in a subfolder for the profile and another one with the name of the input audio file. Two *WAV* files are created in a single pass:

- *1_Used*: the reconstructed signal from the original file, with only the frequencies that are used for the comparison.
- *2_Discarded*: the rest of the signal, all the audio information that was deemed irrelevant and discarded by the specified options.

This means that it does a *Fourier Transform*, applies the selected *window*[2] and estimates the noise floor. The highest amplitude frequencies are identified and limited by range for each element defined in the configuration file, and the rest are discarded. An *Inverse Fourier Transform* is applied to each part in order to reconstruct both files. The frequencies are selected with the window applied, but the window is not applied to the reconstructed audio, so both files keep the original envelope. For the analyzed blocks, adding *1_Used* and *2_Discarded* together results in the original signal. The *sync* blocks are kept in both files as a reference. With this you can listen to the produced files and determine if a more thorough comparison is needed.

[1] See appendix C.
[2] See appendix B.1.1.

In addition, the *-c* option creates a *WAV* file for each element of the profile, trimmed using the detected frame rate. These are saved in the *Chunks* folder: *Source* has each element without modification, and *Processed* has the same element in its *Used* and *Discarded* versions.

The *-n* option only cuts the file into its elements, without performing the *DFT*. If the profile has *internal sync* blocks, a *SyncRemoved* file is saved with the variable delays removed.

These are its options as of version *1.20*. You can type *mdwave -h* in your *mdfourier* folder for details. Unlike *MDFourier*, *MDWave* doesn't create a log file unless *-l* is used.

```
usage: mdwave -P profile.mdf -r audio.wav
 FFT and Analysis options:
  -c: Enable Audio <c>hunk creation,
      an individual WAV for each block
  -w: enable <w>indowing. Default is a custom Tukey window.
     'n' none, 't' Tukey, 'h' Hann, 'f' FlatTop & 'm' Hamming
  -i: <i>gnores the silence block noise floor if present
  -p: Define the noise floor value in dBr
      (0 to disable auto adjust)
  -f: Change the number of <f>requencies to use from FFTW
  -s: Defines <s>tart of the frequency range to compare with FFT
  -e: Defines <e>nd of the frequency range to compare with FFT
  -z: Uses Zero Padding to equal 1 Hz FFT bins
  -B: Do not do stereo channel audio <B>alancing
  -C: Use <C>omparison framerate profile in 'No-Sync' compare mode
  -Y: Define the Video Format from the profile (number)
  -I: <I>gnore frame rate difference
  -T: Increase Sync detection <T>olerance (ignore frequency, can be added up to -TTT)
      -TTTT also accepts pulses of half a frame (emulated audio)
  -m: Set <m>anual sync samples, takes format [r|c]:<start>:<end>
  -n: Just cut the wav file without performing DFFT
  -9: Debug, flattens the profile into one block per type
      (also -q)
 Output options:
  -v: Enable <v>erbose mode, spits all the FFTW results
  -l: Enable <l>og output to file
  -k: cloc<k> FFTW operations
  -y: Output debug Sync pulse detection algorithm information
  -0: Change output folder
```

Appendix F

Normalization and amplitude matching

Since each signal is probably at its own distinct amplitude, we need to perform a *normalization* process in order to have a common comparison point between them.

Each type of *normalization* has its own strengths and weaknesses, but the *frequency domain* normalization designed for this process is always accurate with respect to the *Reference* signal, but might be confusing in some corner cases[1] if the underlying causes are not well understood when interpreting the results.

Since silence can't be used as reference[2], the only other option is fixing points from within the signal. Every normalization process used follows a different logic for fixing one such point for comparison.

The default is to do this in the *frequency domain*[3], but there are other options available via command line with *-n*: *f* for the default, *t* for *time domain*, *a* for *average* and *n* for no normalization[4]. All three methods are described in the following sections.

[1] One such example is available in appendix L.

[2] Noise floor will vary by console and by audio interface.

[3] This means it is performed after the *Discrete Fourier Transform* and using the data generated by it.

[4] These can be enabled via the *extra command* option from the *GUI* as explained in section B.1.5.

F.1 Frequency domain normalization

This is the default option used by the software. It involves finding the *highest magnitude*[5] from the *Fourier Transform* of the *Reference* signal before amplitudes are calculated. Then a corresponding match in the *Comparison* signal's frequency spectrum for the same block is searched for.

This means that the exact same fundamental frequency with the highest magnitude value is searched for, occurring at the same position in time–which corresponds to the block. Having both points, a meaningful *reference point* is set for the comparison, and the relative amplitudes between the signals can be calculated.

In order to calculate the amplitudes, *0dBr* is matched against the absolute highest magnitude from the *Comparison* file[6], after both signals have been relatively normalized in amplitude against the reference point.

This method has shown to be always accurate within the tests. However, results can be unexpected in certain corner cases as the one shown in appendix L.

If the matching frequency in the *Comparison* block is not found, or its amplitude is far from the expected one, a nearby peak in the same block is used instead. This happens when the block was generated with a different clock, and the peak sits at a slightly different frequency.[7]

When an *optional* clock is not found, its blocks are skipped before normalization, so a silent section can't be used as the reference point.

F.2 Time domain normalization

The process is very similar to the *frequency domain* variant but it is done using the raw audio samples directly from the *WAV* file.[8]

The highest amplitude is searched for within the samples of the meaningful audio signal, as dictated by the configuration file.[9] The corresponding segment in time is then located in the *Comparison* file, and a pre-defined duration is searched for the sample with the *maximum local* value.[10]

The *Comparison* signal is then absolutely normalized to *0dBFS*,[11] and the

[5] At this point in the analysis there are no amplitudes defined, since we have no reference points. Hence, raw magnitude values from each transform are used.

[6] Since the highest point of the *Reference* signal was matched to one other point in the *Comparison* file, there are only two options: either both peaks are in the exact same spot, or there is a higher peak in the *Comparison* file.

[7] See appendix D.

[8] Values are changed only in RAM during execution, input files are never modified.

[9] Described in section C.

[10] One frame in the current implementation.

[11] Relative to the highest value that can be represented with the *bit depth* of the file.

Reference signal is then relatively normalized using the adjusted *local maximum* value. This follows the same rationale described for the *frequency domain* equivalent.

Although the results can sometimes be deceivingly familiar, they are not correctly referenced in corner cases,[12] and they do not represent the real relation between the signals. However, they can be useful for analysis while in the process of understanding how to interpret the graphs.

F.3 Highest fundamental average normalization

This normalization option also takes place in the *frequency domain*. The idea is to average the highest magnitudes–the fundamentals–from all segments and use the resulting ratio calculated between both signals to normalize them.

The results are always centered around the *0dB* line in the *Differences graph*, allowing a globalized view. However, the amplitude differences are not to be relied upon for calibration, since they are not relative to a fixed point from the *Reference* signal. This option is just available for cases when both signals present a high difference rate between them, which is unexpected but possible.

[12] See appendix L for an example.

Appendix G

Frame rates and their importance

The *frame rate* is the amount of frames per second a system sends to the display. Since the *Tone Generator* runs from within the target system, we are subject to the internal timing. Every time a frame starts the process for being sent to the display, an event called *vertical sync* occurs. This is the driving clock for the whole console.[1]

It is vital for the process to have a basic unit of time, in order to segment the file in the chunks needed for analysis and to send the correct values to the *Discrete Fourier Transform*.

Signals generated from different sources can have dissimilar *frame rates*, even when running the same programs and representing the same platform. This can be due to several reasons: having a different display technology as target, hardware inconsistencies between revisions, etc.

Another source for variation is the *audio capture device*, since these can vary slightly in their sample rate clocks. But usually these are lower variations that are reported and compensated for internally. It has also been found that some audio interfaces use a more inaccurate clock when sampling at *44.1kHz* and a very accurate one when using *48kHz*.[2]

The frame duration is the basic unit of measurement, since we are dealing with signals that are well below one second. Because of this, the configuration file[3] defines the expected frame duration in milliseconds as measured from a vintage console. One such example is the frame duration as measured from an *NTSC Sega Genesis* using an *oscilloscope*.[4] as shown in Figure G.1

[1] As a matter of fact, almost all the code for running games–or the *Tone Generator* in out case–is executed during a segment called *vertical blank*.

[2] The internal *M-Audio 192* exhibits this behavior. See appendix P.1.1.

[3] See appendix C.

[4] Electronic tool for measuring electric signals.

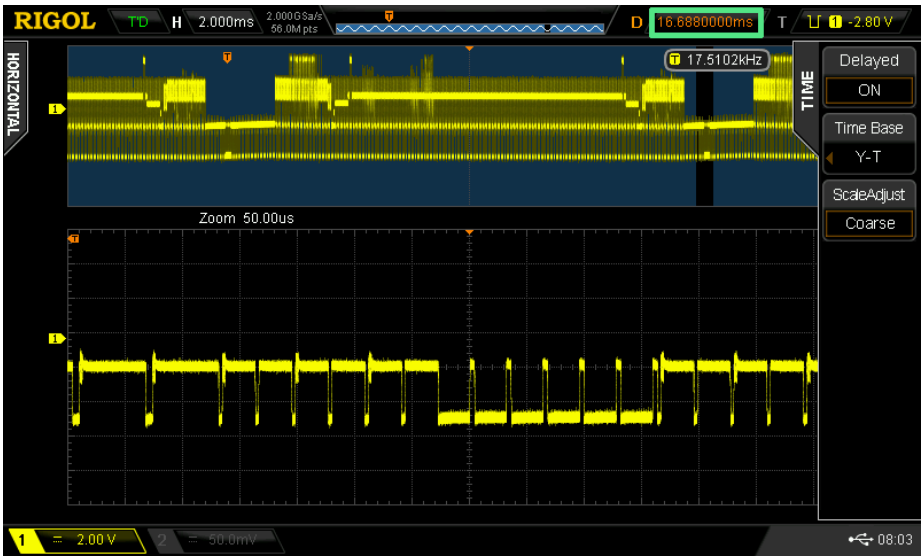


Figure G.1 Frame duration as measured with a scope from a *Model 1 VA3 system*.

The oscilloscope shows a frame duration of *16.88 ms*, and this is the value used in the configuration file[5] for the initial frame rate calculations.

After the *sync pulses* are detected,[6] a new frame rate value is calculated from the audio file. This new frame rate is used across the whole process, and can vary from the vintage console due to a combination of different reasons.

Every audio interface can have a slightly different *sample clock*,[7] and this variation will affect the starting and ending points within the recording. Such variation can be detected and estimated from the audio signal, and it is reported if the *verbose log*[8] option is enabled.

The selected *sample rate* for recording can also affect the *sample clock* precision even while using the same interface.[9]

The most important case, and the one for which the *sync pulse* solution was implemented, is when comparing a modern system such as an *emulator* or a *FPGA* implementation. These usually run at modern *refresh rate*[10] for better compatibility with current display technology, such as *HDTVs*, since the vintage hardware has *refresh rates* that are slightly off-spec.[11]

Since *frame rates* differ and the system uses the frame rate as a master clock

[5] Described in appendix C.
[6] Described in appendix C.
[7] See appendix P.1.
[8] See section B.1.6.
[9] This is the case when using the *M-Audio 192* [32] in *44100Hz*, but when using *48000Hz* the sample clock is spot on.
[10] The refresh rate is the inverse of the frame rate. For example with *16.88* it would be 1/16.88 which is *59.92 frames per second*.
[11] *NTSC* is *59.97 frames per second*.

for program execution, the resulting audio recordings also have a difference in length. For example when using an implementation adjusted for *NTSC*, there is a *0.001* second difference in each block of 20 frames. Although this seems to be negligible, it adds. In a one minute recording, such as the ones used for *MDFourier*, every note was misaligned due to this difference.

The software adjusts for these differences, carrying the fractions of a sample over to the next block. Blocks marked with the * mask are instead cut in the file with the longer frames, so both files analyze the duration the block has at the shorter frame duration.[12] Since the notes are not played during the whole 20 frames and the window filters help to compensate the signals, the comparison is perfectly aligned for the tests. The reader can verify these results via the *MDWave* tool.[13]

If the detected *frame rate* differs too much from the one in the profile, an error is shown since the profile or video format is probably wrong. It can be ignored with *-I*. When the recording is shorter or longer than expected because the *sample rate* of the capture is off, the *-R* option estimates the real *sample rate* from the length of the file, for the *Reference* (*r*), the *Comparison* (*c*) or both (*b*).

If the *sync pulses* are not detected, the *-T* option makes the detection more tolerant. It can be repeated up to four times, each level relaxing the detection further: *-T* ignores the frequency of the pulses, *-TT* and *-TTT* accept noisier and less regular pulse trains, and *-TTTT* also accepts pulses as short as half a frame.

The last level is meant for emulated audio, such as the *Sound Blaster* emulation of *PCI* cards under *Windows* or with their *DOS* drivers. These can delay each write to the *FM* chip to a fixed time grid, so the pulses come out shorter or longer than a frame. With *-TTTT* each pulse is used where it was found, without centering it by its length, so the alignment can be off by a fraction of a frame and the results should be read with that in mind. The option only changes the detection when it is used, so the recordings that already worked are not affected, and the log shows it is enabled:

```
-Sync pulses down to half a frame accepted (-TTTT), for emulated audio
```

As a last resort, the position of both pulse trains can be given in samples with *-m*, and *-y* adds the details of the detection to the log.

[12] See appendix C.
[13] See appendix E.

Appendix H

Channel separation

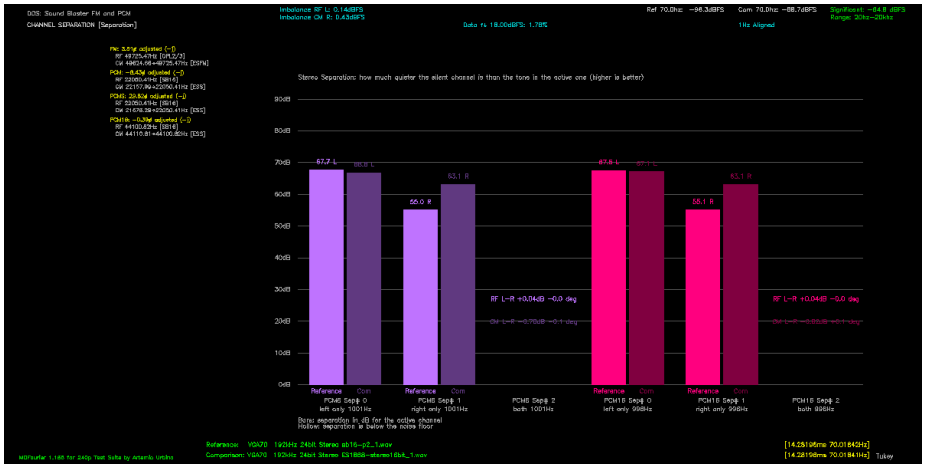


Figure H.1 Channel separation between a SB16 and ES1869

Channel separation measures how much of a tone played in one channel leaks into the other. It is measured on the blocks with *C* as their channel in the profile, which are analyzed as *stereo*. These blocks have their own report and graph, so they are left out of the comparison, and their waveforms are always plotted. Both files must be stereo. This fragment of the *MS-DOS* profile has three tones for *8 bit* and *16 bit PCM*, in the left channel, the right channel and both:

```
PCM8_Sep        14   3  20  4 violet  C *
PCM16_Sep       18   3  20  4 rose    C *
```

For each block, the strongest frequency within the analyzed range is found in each channel, and the louder channel is taken as the *active* one. In the other

channel, the *leak* is the strongest value within two bins of that frequency, and its *noise floor* is the median of the bins around it, leaving out the closest eight bins to each side. The separation is the level of the active channel over the leak, in *dB*.

If the leak is less than *6dB* above the noise floor of the silent channel, it can't be told apart from the noise. The separation is then measured against the noise floor, and reported as a minimum, for example $\geq$ *72.4dB*. If the silent channel has no signal at all, it is reported as *digital silence*.

When the leak is less than *10dB* below the active channel, the tone is considered to play in both channels. The level difference between left and right, and the phase difference between them, are reported instead.

The log has a line for each block with both files. When the active channel is the same in both, the difference between them is added. It then reports the lowest separation found, how many results were limited by the noise floor, and warnings when the channels are inverted between the files or when one file plays a tone in both channels and the other in one. The summary lines have this format, and the results are drawn in their own graph.[1]

```
* Channel separation (tone in one channel, the other read at the same frequency):
- Lowest separation: 55.0dB in PCM8_Sep# 1
```

[1] See section B.1.4.

Appendix I

Noise compared by bands

Since the noise is pseudo random, two recordings of the same noise channel from the same console have different peaks, so comparing them frequency by frequency shows differences that aren't there.

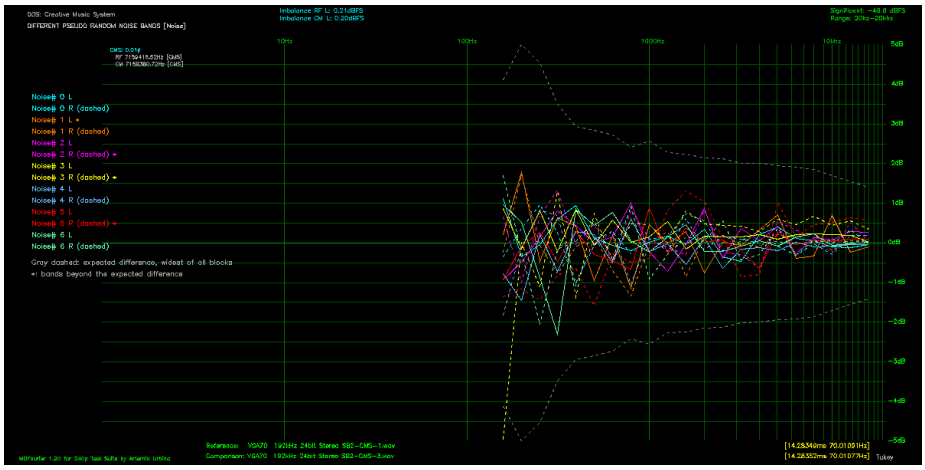


Figure I.1 Noise compared by 1/3 octave bands between two CMS recordings

Its average spectrum is not random, so that is what blocks marked with the *b* flag compare. Examples are the noise channels of the *CMS* and the *PC Engine*. This fragment of the *CMS* profile has one:

```
Noise          6   7  40  8 aqua    s - b
```

The power of the bins is averaged in each *1/3 octave* band of the analyzed range. A single bin of noise varies by about *5.6 dB*, and a band with n independent bins by $4.34/\sqrt{n}$ *dB*. That number is the width of the band multiplied

by the duration of the block, so longer blocks reach lower bands. Only bands with at least *20* are used, about *1 dB* each. Since the power is averaged rather than added, blocks of different lengths and from different frame rates can be compared.

Noise clocked slower than a band's frequency repeats its pattern across the spectrum, so its bands scatter more than that. Each recording measures its own scatter by comparing the two halves of every block. The allowed range is widened to match cases where the halves differ more than expected, over the band and the two on each side. Two recordings of the same noise then almost always stay within three standard deviations.

These blocks are left out of the normalization and of the regular comparison. The log reports how many blocks have bands beyond the expected difference, and the largest difference found. With *-v* it adds a line for each block and channel, with the number of bands compared, the average and the largest difference, and how many bands are beyond it:

```
* Noise compared by 1/3 octave bands:
- 2 of 7 blocks have bands beyond the expected difference
- Largest band difference: -13.05 dB in Noise# 2 L at 16000 Hz
```

Each type has its own graph, *RAND_DIFF*, with the difference of every band, one color per block, the right channel dashed. The gray dashed line is the expected difference, the widest of all blocks, and the blocks marked with an * have bands beyond their own.

Appendix J

Stereo Audio Balancing

Sometimes the audio recording has different amplitude levels between the right and left audio channels, this is referred to as *stereo imbalance*. Whenever possible, *MDFourier* detects and auto compensates this imbalance in order to give a more uniform graph result. The default behavior can be disabled if desired with option *-B*.[1]

This should only be a concern if an imbalance in the source system is suspected.

J.1 Possible causes

Two causes for imbalance have been identified. At the time of analysis *MDFourier* cannot possibly discern between them, and both can be present at the same time.

- Amplification deficiencies: The system being recorded has an imbalance due to aging capacitors or other issues in its internal amplification circuitry.
- Audio interface balance: Another possible cause for audio imbalance are the individual gain knobs in some *USB* audio interfaces. This happens because it is very hard to adjust both of them to the exact same resistance vale, as shown in Figure J.1.

It must be noted that the internal *PCI* audio interface used during testing has minimal audio imbalance–*0.0018%* on average–and this allows detecting *system amplification deficiencies* with consistency.[2]

[1] See appendix B.1.5 for details on how to use *extra commands*.
[2] See appendix P.1 for information regarding *audio interfaces*.



Figure J.1 Lexicon Alpha volume knobs.

The audio interface balance issue can be minimized by adjusting the knobs carefully. *MDFourier* can be useful for compensating this via repeated testing, since any imbalance is reported in the output. It has also been found that leaving gain at zero in some cards, like the *M-Track*, produces more pronounced imbalance than at *50%*. However this requires recording from a source that is known to be balanced.



Figure J.2 M-Track volume knobs at 50%.

In order to detect and compensate audio imbalance, at least one *monaural* block must exist within the *mfn*[3] configuration profile being used. The block is selected with the *[MonoBalanceBlock]* line, or the first *monaural* block is used.

The software assumes that the monaural signal must be the exact same amplitude between left and right audio channels, and compensates by normalizing the *samples* from one unbalanced channel by adjusting them via the proportional ratio.

[3]See appendix C for details.

Appendix K

Differences due to temperature

During the initial development of the software there was a single instance where unexpected variations were present between different recordings of the same hardware using the same audio interface as shown in Figure K.1.

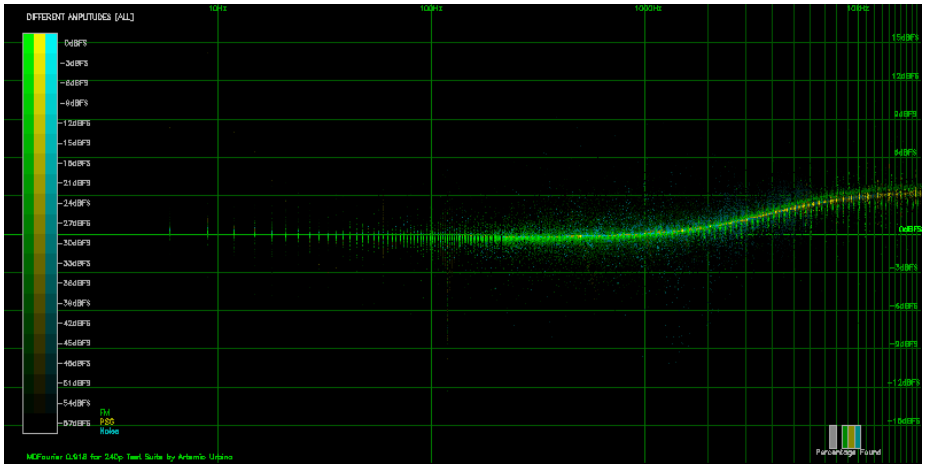


Figure K.1 Audio difference caused by heat in *Model 1 VA3 system.*

After analysis, it was found that the difference was present only in the left channel and was caused by a severe difference in temperature while recording. The room was at 29°C[1] as shown in Figure K.2, and the *Sega Genesis Model 1 VA3* under testing had been powered on for around 30 minutes.

[1] 84.2°F.

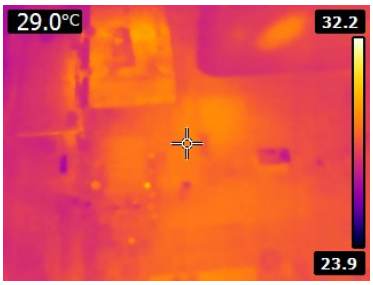


Figure K.2 Console at room temperature *Model 1 VA3 system*.

At first, warm up differences were suspected. But after some cooling, *cold boot*[2] and measurements using a thermal camera it was determined that the room temperature on top of the warm up was the culprit, since the results could not be replicated when the room had cooled down to 22°C[3].

In order to replicate the results, its vents were obstructed to prevent heat dissipation[4] for a brief period. The results could be replicated at will under these conditions after just a few minutes of use.

It must be noted that the temperature of the headphone amplifier[5] of the console reaches 82.9°C[6] with cover removed as shown in Figure K.3, so higher temperatures are expected with it closed. This is important since some of the stock capacitors around the headphone amplifier are radiated due to proximity, and can probably reach temperatures beyond their intended rating in warm climates.

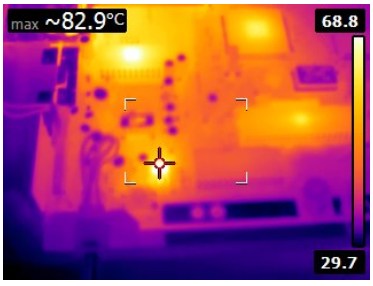


Figure K.3 *Model 1 VA3 system* with amp at 80°C.

[2]Powering on the console after it had cool down, and immediately making the recording.
[3]71.6°F.
[4]Ventilation were covered with electrical tape.
[5]CXA1034
[6]181.22°F.

Appendix L

Corner cases

While testing the default *frequency domain* normalization,[1] I found that one artificially generated case would produce results that were unexpected and surprising. After analysis, I believe them to be the correct handling of the data, and the appropriate comparison results.

For this example we'll modify a file as detailed in *Scenario 3*,[2] but this time a *1zkHz 6dB* tone will be added instead of the *500Hz* from that previous case.

I was expecting the result detailed in *Scenario 3*, but was greeted with the graph shown in Figure L.1:

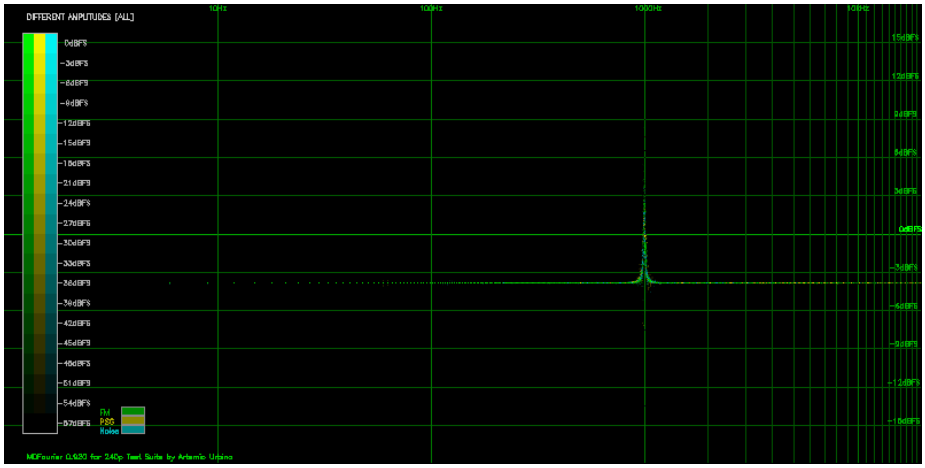


Figure L.1 Signal compared to itself with *1kHz 6dB* artificially inserted.

I thought it was a bug in the processing of *frequency domain normalization*. But after careful analysis, this is the correct result from such process.

[1] See appendix F.
[2] See section 4.3.

It is all due to a big coincidence: the maximum global *amplitude* of the original signal used for this scenario is located at *1005.99Hz*, at just *5.99Hz* from the *1kHz* peak that was artificially inserted. As you might sharply point out, Figure L.1 doesn't show the peak *6dB* below zero. But that is because it is not centered at *1005.99Hz*.

Here is the result when centering the peak at *1005.99Hz*:

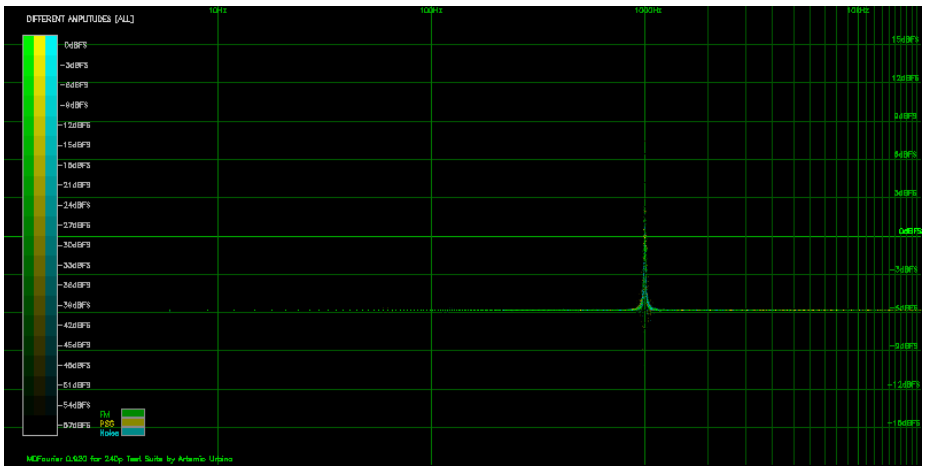


Figure L.2 Signal compared to itself with *1005.99kHz 6dB* artificially inserted.

Now the majority of the curve is *6dB* below zero, and the peak is at zero. But why?

As explained in appendix F, the process of frequency normalization consists in finding the highest amplitude in the signal and use that as a reference point for matching both signals. In this case, both signals have their maximum at *1005.99Hz*, but we also modified the *comparison* file to have a *6dB 1005.99Hz* peak. As a result when using the original *reference* file as a model and judging the *comparison* file against it, the rest of the signal has been lowered from this perspective.

This would mean that if two systems had these audio signatures, the resulting graph would tell us that the *comparison* console is–in general–producing the frequencies at lower amplitudes. If the *comparison* system had to be modified to match the reference system, the rest of the frequencies in the signal should be raised by *6dB*, since the single point at *1005.99Hz* already matches the reference signal. It is just a matter of perspective. It helps keeping in mind that *MDFourier* does relative comparisons, and in this framework the *reference* file is the absolute model.

In case we'd like to see a graph that would probably be easier to understand–but which I now believe is not as accurate in representing the relative differences–one could enable the *time domain normalization*. The output graph would be the by now familiar result from Figure L.3:

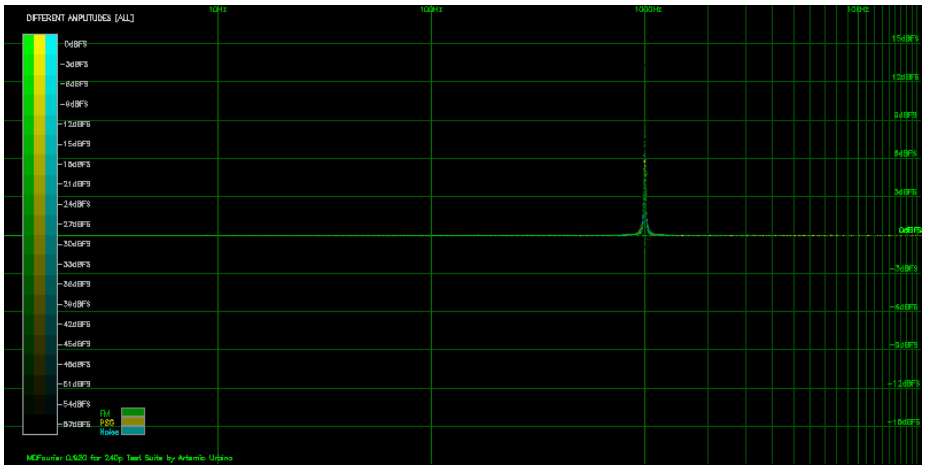


Figure L.3 Signal compared to itself with *1005.99Hz 6dB* artificially inserted, but normalized in the time domain.

This graph tells us the exact same thing: if the *comparison* signal is to be modified to match the *reference* signal, the peak at *1005.99Hz* must be lowered by *6dB*. Since the results in both cases would be the same, both interpretations are equivalent.

However, I am inclined to adopt the first position, since it has an absolute reference point. Whereas the second graph and interpretation must be modified depending on the input. For this second scenario to work, our framework needs to be modified for special cases–that are highly unlikely to be found in the wild. And since that creates unnecessary inconsistencies and results in imprecision, the *time domain normalization* is not used by default.

Appendix M

Known Issues

I'm positive several deficiencies in my implementation still elude me. However here are a few I'm aware of and have not addressed yet.

- *PCM* generated by the *FM* chip is not covered in the current *Mega Drive/Sega Genesis* implementation. It is assumed the response will follow the *YM2612* curve.
- Recordings made from a *Windows DOS* box, or of *Sound Blaster* emulations, are less precise than those from plain *MS-DOS*. See the following section.
- A better signal could be generated to cover either more or specific cases.

M.1 *MS-DOS* under *Windows*

The *MS-DOS Tone Generator* can run from a *DOS* box under *Windows 3.x* and *Windows 9x*, but not under *Windows NT*, *2000* or *XP*. These recordings carry a different watermark, at *1.5kHz* or *2.5kHz* instead of *1kHz* or *2kHz*, and are reported as recorded under *Windows* in the log and the graphs.[1] The best results come from a full screen, exclusive *PIF* with *Detect Idle Time* disabled and no *Monitor Ports*, but even then they should be read with care.

The *Tone Generator* starts every event at the vertical retrace, within a fraction of a millisecond. *Windows* keeps the overall timing right, but it can't hold that precision:

- The *DOS* box doesn't see the real timer. Under *Windows 9x* the *Tone Generator* uses the *CPU* time stamp counter instead, measured against

[1] See the watermark block in appendix C.

the *BIOS* ticks. Under *Windows 3.x* on a *486*, without that counter, the virtual timer counts at half the expected rate and is read as such. Either way the total length matches the one from *MS-DOS*, and the */T* option checks it.

- *Windows* interrupts the program to deliver its own timer ticks and run other tasks. When a retrace goes by, the *Tone Generator* catches up instead of stopping, so the events of that frame start late. Under *Windows 3.1* this happened in about one frame out of four, by up to *52 ms*. The report at the end lists the late and missed frames.

- The interrupts from the *Sound Blaster* can arrive late, or not at all, when a *Windows* driver also uses the card. Then the *PCM* sections can be missing or shifted. Under *Windows 3.1*, removing the *Sound Blaster* driver from *SYSTEM.INI* and adding *TrapTimerPorts=OFF* to its *[386Enh]* section can help.

- The *ESS Extended* mode and *WSS* codecs are left to *Windows*, so *16 bit PCM* is played only through a *DSP 4.xx*.

- *PCI* cards such as the *Sound Blaster Live!* emulate a *Sound Blaster*, under *Windows* or with their *DOS* drivers. The writes to the *FM* chip are delayed to a fixed time grid, so the sync pulses come out shorter or longer than a frame, and need *-TTTT*. The *FM* and *PCM* sections can also be offset by about a frame from each other, since each one has its own latency.

Late frames move the sync pulses too, and short noises such as clicks can be taken for them. When the sync is not detected, or is detected in the wrong place, the pulses can be given manually with *-m*.[2] Even when aligned, the events within the blocks don't start at the same time as in a recording from *MS-DOS*.

Games ran under these conditions because they didn't need this precision. Music was played from the timer interrupt, where a few milliseconds between notes are not audible, and digital audio was played from large buffers that hid the late interrupts. Games that needed precise timing asked to be run from *MS-DOS*, which is why *Windows 95* included an *MS-DOS mode*. The same applies here: for reference recordings, use plain *MS-DOS*.

Please contact me if you are aware of any details that I have missed.[3]

[2] See section B.1.5.
[3] See appendix T.

Appendix N

Window Function equations and graphs

This appendix lists the equation and curve of each *Window Function* used to limit *spectral leakage* as described in section B.1.1.

N.1 Tukey

The default is a *Tukey* window selected for this purpose. It uses $\alpha = 0.6$, zeroing just a few samples, with minimal *spectral leakage* and good amplitude response.

The following equation is used to create the slopes:

$$tukey(x) = \frac{1}{2}(1 + \cos(\frac{\pi(|x - \frac{N-1}{2}| - \alpha\frac{N-1}{2})}{(1-\alpha)\frac{N-1}{2}})) \tag{N.1}$$

And this is the resulting graph of the *Tukey* window, ranges are *0.0* to *1.0* horizontally and *-0.1* to *1.1* vertically.

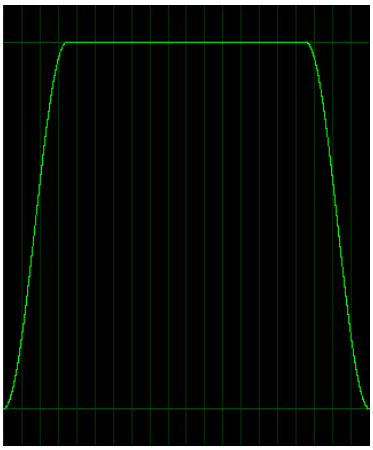


Figure N.1 Tukey window used by *MDFourier*, vertical lines are frames on a 20 frame signal.

Detailed information can be found in the reference webpage [31].

N.2 Hann

When selected, a typical *Hann* window is used. This should be used to get the last *spectral leakage*, with a very small trade off in amplitude accuracy.

$$hann[x] = \frac{1}{2}(1 - \cos(\frac{2\pi(x+1)}{n+1})) \tag{N.2}$$

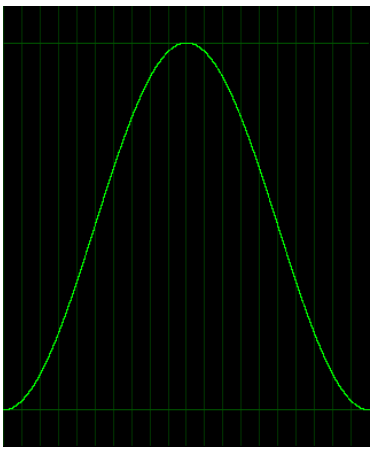


Figure N.2 Hann Window.

N.3 Flattop

A typical *Flat top* window is used, selecting this will target amplitude accuracy against frequency bin precision.

$$flattop(x) = 0.21557895 - 0.41663158\cos(2\pi\frac{x}{n-1}) + 0.277263158\cos(4\pi\frac{x}{n-1}) \\ -0.083578947\cos(6\pi\frac{x}{n-1}) + 0.006947368\cos(8\pi\frac{x}{n-1})$$

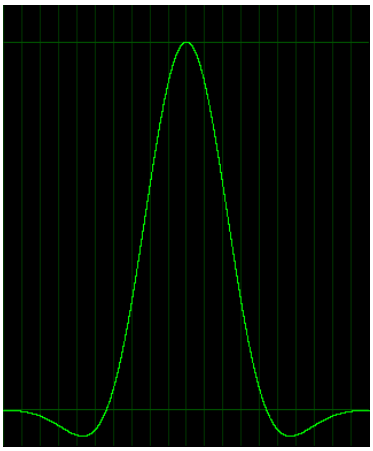


Figure N.3 Flat Top window.

N.4 Hamming

A typical *Hamming* window is used, presented for completeness and reference since the samples are never zeroed out.

$$hamming[x] = 0.54 - 0.46\cos(\frac{2\pi x}{n-1}) \tag{N.3}$$

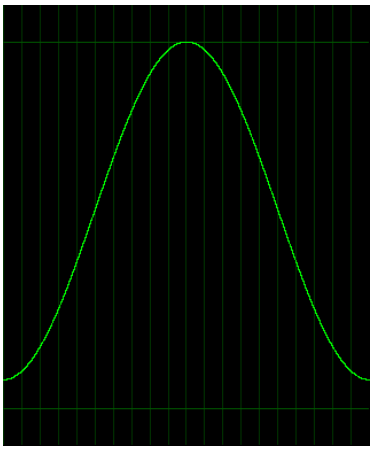


Figure N.4 Hamming window.

N.5 No Window

When this option is selected, no filtering window is applied to the signal before applying the *DFT*, which is equivalent to a *rectangular window*. Since the signal is unprocessed, any uncontrolled decay or even the noise floor can be factored in as part of the signal.

There is more information on windows and their usage in the reference webpage linked from [15].

Appendix O

Color Filter Function details

This appendix contains a description, graph and example of each *Color Filter Function* from section B.1.2.

O.1 None

No filtering is applied, as a result all differences are plotted using the brightest color.

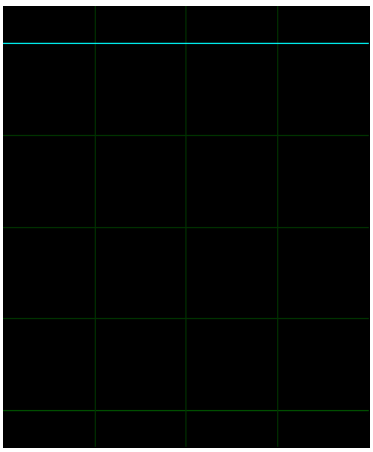


Figure O.1 No Filter.

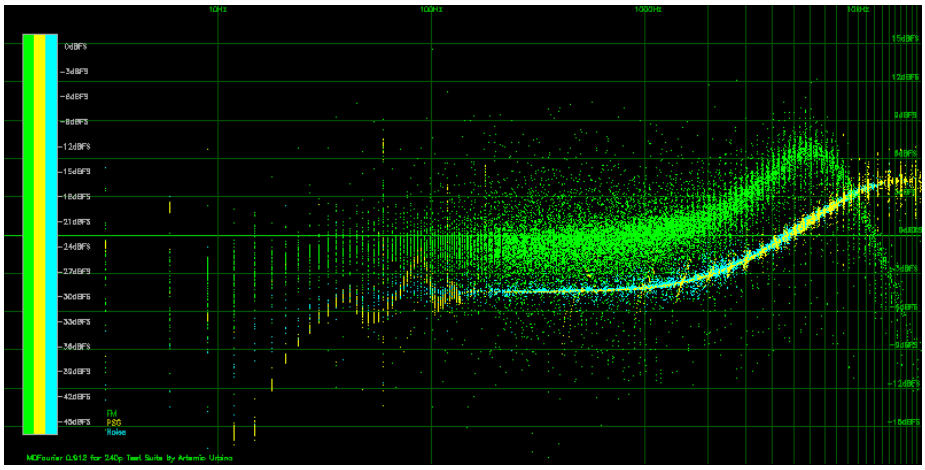


Figure O.2 No Filter Applied.

O.2 Bright

A square root function will only attenuate the lowest amplitude differences. This applies the formula $\sqrt{|dBr|}$ to the amplitude from the *reference* signal, in order to assign the brightness.

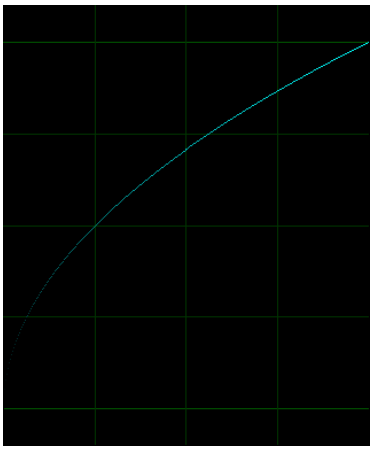


Figure O.3 Square Root filter.

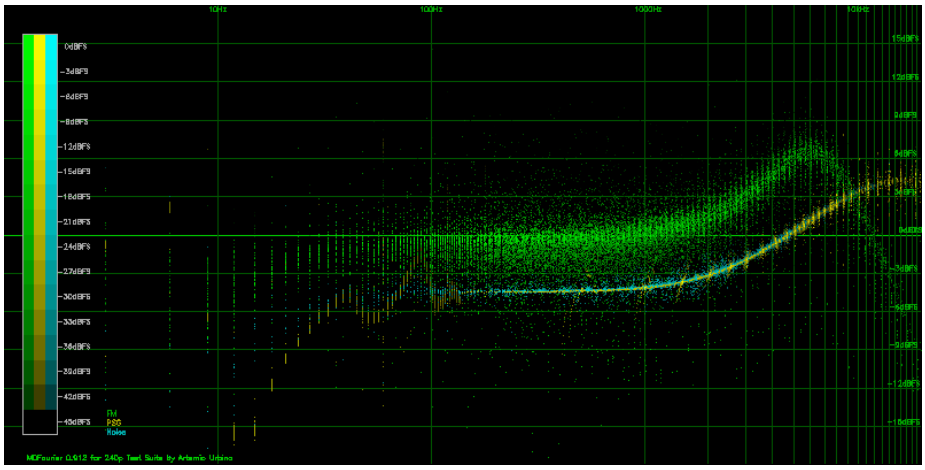


Figure O.4 Square Root filter Applied.

O.3 High

A Beta Function filter with parameters $\beta(3,3)$ will attenuate a bit more from the lower range, still showing most of the differences.

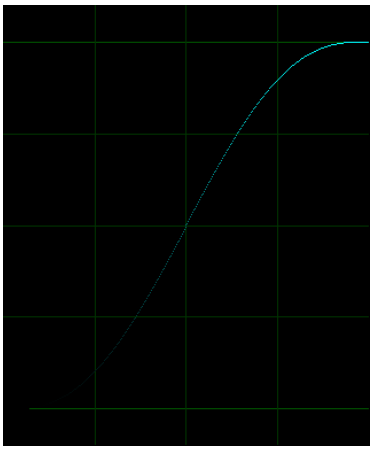


Figure O.5 Beta Function(3,3).

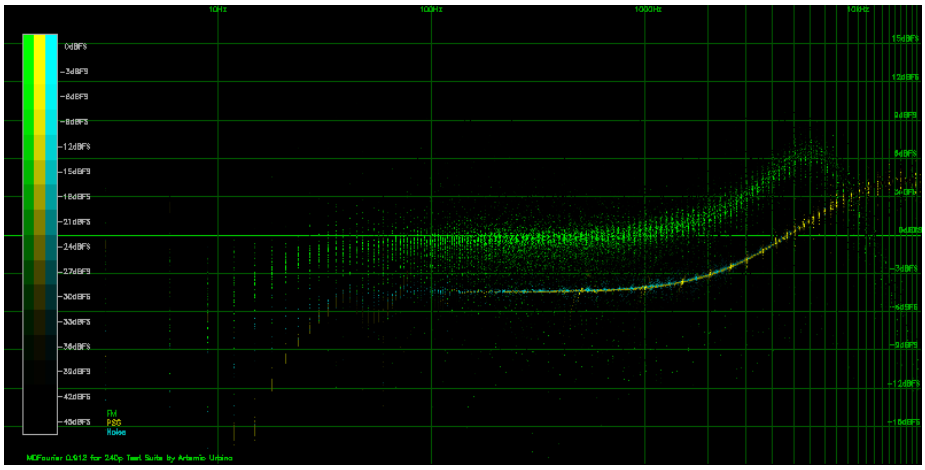


Figure O.6 Beta Function(3,3) Applied.

O.4 Neutral

The linear function is the default selection, and has no bias. Half the dynamic range corresponds to half the color rage.

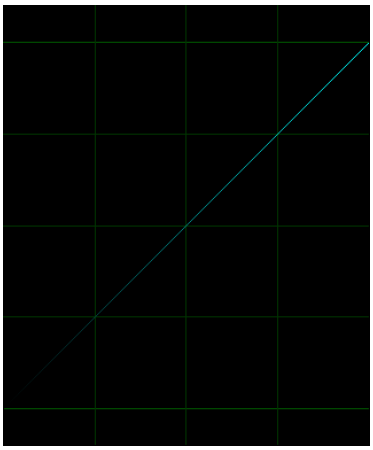


Figure O.7 Linear Function.

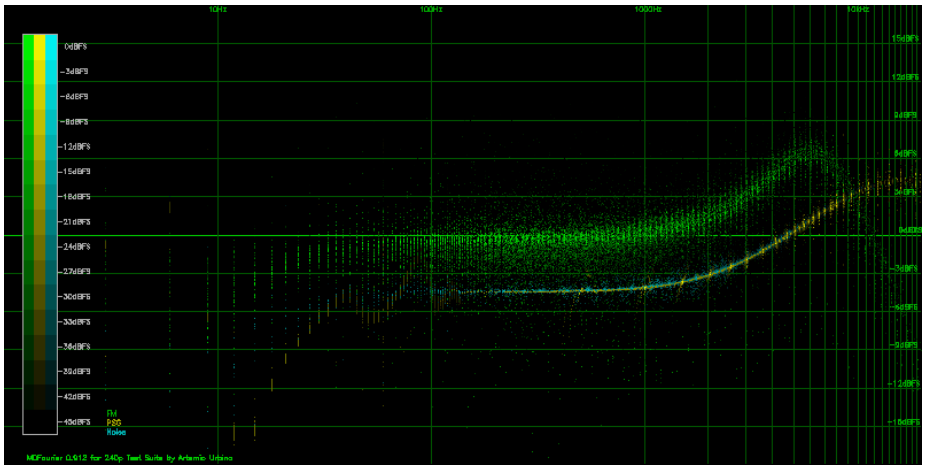


Figure O.8 Linear Function Applied.

O.5 Low

A squared function dBr^2 will attenuate a lot more differences. As a result, only the frequencies with the highest amplitudes from the *reference* signal will be brighter.

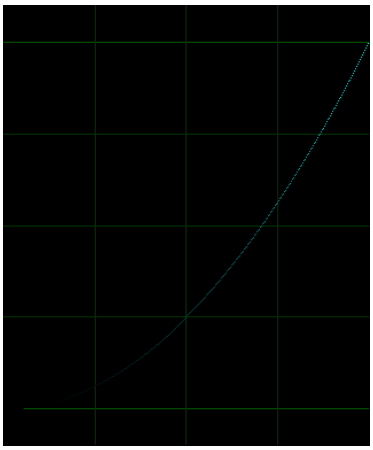


Figure O.9 Linear Function.

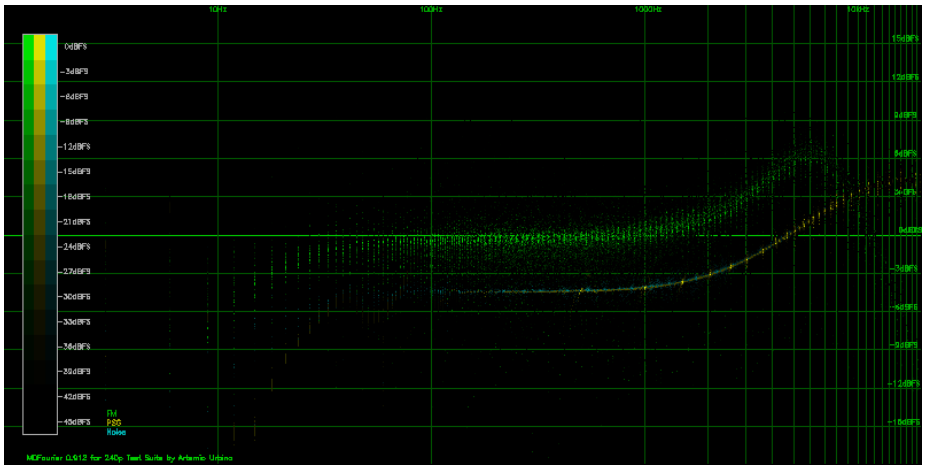


Figure O.10 Linear Function Applied.

O.6 Dimm

A *Beta Function filter* with parameters $\beta(16, 2)$ will attenuate almost all the differences, and only the frequencies with the highest amplitude in the reference signal will be brighter.

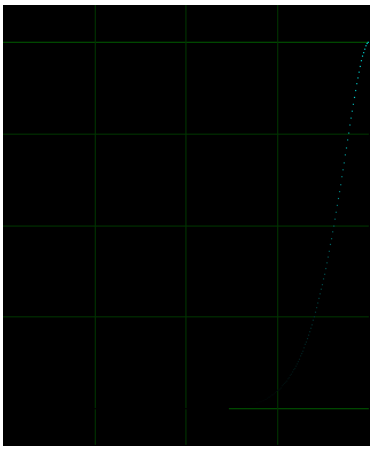


Figure O.11 Beta Function(16,2).

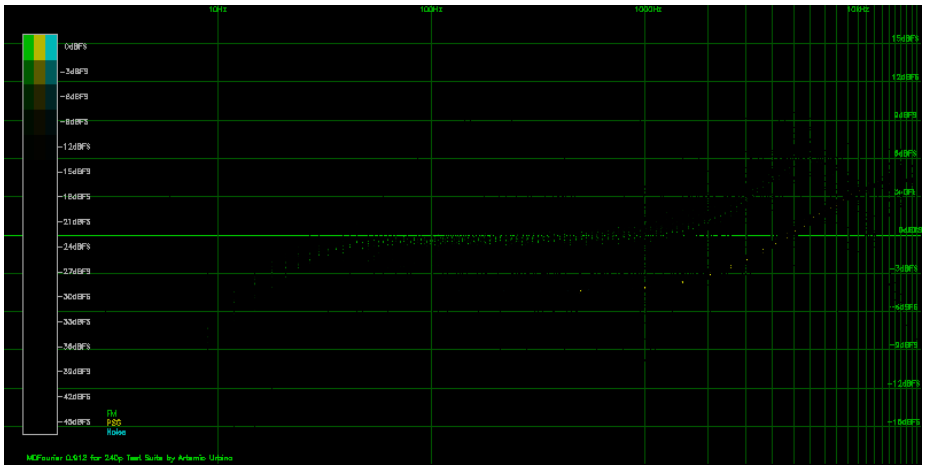


Figure O.12 Beta Function(16,2) Applied.

Appendix P

Requirements

P.1 Audio capture device

For capturing the audio files, an *audio capture device* is needed. It is recommended to use a musical grade *audio interface* in order to get a *flat frequency* response across the human hearing spectrum. Fortunately, these devices have become more affordable.

So far five audio interfaces have been used in my personal setup. The sample recordings available for download[1] were made with three of them, a complete set with each one. The *MOTU M4* was validated more recently, as a current model.

I have no association or business relationship with these products, they are just presented as working examples. As more people use the software and we–as a community–compare files, this list can be expanded with recommendations.

- **MOTU M4:** A USB-C audio interface with four inputs and four outputs, recording at up to *24 bit* and *192kHz*. [34]
- **Lexicon Alpha:** A USB audio interface[2].
- **M-Audio M-Track:** A USB audio interface[3].
- **M-Audio M-Track Pro:** A USB audio interface, adds *Sony/Philips Digital Interface* (*S/PDIF*) over the regular model[4]. [35]

[1] See appendix A.

[2] Uses a CS4270 24-Bit, 192 kHz Stereo Audio CODEC and a TAS1020B for USB communications. Verified by opening the device. [33].

[3] Uses a CS5341 105 dB, 192 kHz, Multi-Bit Audio A/D Converter and a TAS1020B for USB communications. Verified by opening the device. [35].

[4] Not opened for component verification, but assumed to use the same components as the regular version.

- **M-Audio Audiophile 192:** An internal *PCI* audio interface[5] that is no longer available in the market [32].

Some interfaces marketed for podcasting and audio cassette digitization were tested,[6] but they didn't show a flat frequency response. You should try to use an audio interface that is aimed at musicians or instrument recording.

P.1.1 Sampling clocks

Audio interfaces have their own internal clock for sampling,[7] which can sometimes deviate enough so that *MDFourier* can report *frame rate* differences. From my tests, this is more prominent when using the *44.1kHz* sample rate, and minimized when using *48kHz* or higher. The effect is compensated for while loading the file and during trimming, so it shouldn't be a problem due to the small variations this produces in the frequency domain. Larger deviations can be compensated with *-R*.[8]

The clocks of the system under analysis are a different matter, since they change the frequencies that are generated. These can be measured and compensated as described in appendix D.

P.2 Computer

Any computer and operation system can be used if you are compiling the *source code* from scratch,[9] but a statically linked *Microsoft Windows executable* is provided for convenience, alongside a *front end*. See chapter B for instructions on using the *GUI*.

P.3 Game Consoles or emulators

You'll need either the provided example audio files[10] or to create your own, by recording from your own source. This is probably the desired route, since you will want to compare against your files.

[5] Uses a VT1720 Envy24PT PCI Multi-Channel Audio Controller.

[6] Gear of this type could exist with a *flat frequency* response, current technology should allow any product to offer this basic requirement. But alas, the ones we've tested failed in our tests.

[7] For more details, see [18] [19] [20].

[8] See appendix G.

[9] See appendix A for download links.

[10] See appendix A for download urls.

P.4 Flash cart, or means to run the *Tone Generator*

The console needs to run the *Tone Generator* created for that platform–which are provided with the rest of *MDFourier*. If possible this will be integrated into each version of the *240 test suite* [7].

In order to run these, you'll either need a *flash cart* or a custom loading solution compatible with the target platform.

P.5 Cables and adapters

You'll need cables–and maybe some adapters–to connect the audio output from the console to the input of your *audio interface*.

P.6 Audio capture software

Your *audio interface* will probably be bundled with some audio editing software, or you can use *Audacity* [36] or *Goldwave* [37] depending on your operating system.

Appendix Q

Compiling from source code

MDFourier needs a few libraries to be compiled. In *Linux*, *UN*X* based systems, *MSYS2* [25] and *Cygwin* [30], you can link it against the latest versions of the libraries.

- Fastest Fourier Transform in the West. (fftw) [8]
- The GNU plotutils package. [11]
- *PNG* Reference Library: libpng. [9]
- *FLAC* Reference Library: libFLAC. [10]

The following implementations are also used and included with the source files:

- sort.h for tim sort. [12]
- Incomplete Beta Function. [14]

The pre-compiled executable of the *Analysis Software* for *Windows* is created with *MSYS2 UCRT64*, and statically linked for distribution against these libraries:

- fftw 3.3.10-5 [8]
- plotutils-2.6 [11]
- libpng 1.6.53-1 [9]
- flac-1.5.0 [10]

The *makefile* provided with the source code [6] detects the system and selects the matching target when you run `make`: `msys-s` for *MSYS2* (static), `linux`, `cygwin`, `mac`, `macarm` and `macppc`. The `debug` and `debugsan` targets build with debug symbols, the latter with the address sanitizer. Both `mdfourier` and `mdwave` are built.

Processing and plotting are done in parallel when compiled with *OpenMP* support. The *MSYS2*, *Linux* and *Cygwin* targets enable it; the *macOS* targets do not.

Q.1 Linux

Install the compiler and the development packages of the libraries from your distribution, then run `make` in the source folder. For example, in *Debian* and *Ubuntu*:

```
sudo apt install build-essential libfftw3-dev libplot-dev \
                 libpng-dev libflac-dev
```

And in *Fedora*:

```
sudo dnf install gcc make fftw-devel plotutils-devel \
                 libpng-devel flac-devel
```

Q.2 macOS

Q.2.1 Intel and ARM

If *XCode* or the *Command Line Tools* are not installed, open a *Terminal* and run `make`. A prompt will offer to install the *Command Line Tools*; accept it, and the development tools will be installed without the whole *XCode* package.

Install *Homebrew* [26] following its instructions, and then install the libraries with `brew install <package name>`:

- fftw
- plotutils
- libpng
- flac
- libogg

Homebrew resolves and installs the dependencies. Clone the repository and run `make`.

Q.2.2 PowerPC

Install *XCode* 2.5, and *X11* from the *Mac OS X* installation disc (in the *Optional Installs* package). Run the system updater to get the last versions of *XCode* and *X11*.

Install *Tigerbrew* [27] following its instructions carefully, and then install the same libraries listed above with `brew install <package name>`. Clone the repository and run `make`.

Q.2.3 Redistributable static binaries

A guide on creating a redistributable *MDFourier* binary with the libraries statically linked on *macOS* is available at [28].

Q.3 Windows with MSYS2

Install *MSYS2* [25] and open the *UCRT64* environment, which is the one used for the distributed executable. The *CLANG64* environment also works; the *makefile* then uses `clang` and links `winpthread`. The steps below use *UCRT64*; for *CLANG64*, replace `ucrt-x86_64` with `clang-x86_64` in the package names and `/ucrt64` with `/clang64` in the prefixes, and add `CC=clang CXX=clang++` to the `configure` commands.

Use the terminal of that environment, *MSYS2 UCRT64* or *MSYS2 CLANG64*, for every step below, including building *MDFourier*. In it, `echo $MSYSTEM` must print `UCRT64` (or `CLANG64`), and `which gcc` must print `/ucrt64/bin/gcc` (or `which clang`, `/clang64/bin/clang`).

Q.3.1 Compiler and libraries

```
pacman -Syu
pacman -S make
pacman -S mingw-w64-ucrt-x86_64-toolchain
pacman -S mingw-w64-ucrt-x86_64-fftw
pacman -S mingw-w64-ucrt-x86_64-libpng
```

For *CLANG64*, also install `mingw-w64-clang-x86_64-clang`, `mingw-w64-clang-x86_64-headers`, `mingw-w64-clang-x86_64-lld` and `mingw-w64-clang-x86_64-lldb`.

Q.3.2 plotutils-2.6

plotutils needs to be patched to build against current versions of *libpng*. A pre-patched version is available [29]. To patch it yourself, download it from [11] and edit `libplot/z_write.c`, changing both occurrences of `png_ptr->jmpbuf` to `png_jmpbuf(png_ptr)`. Under *UCRT64*, also remove the `bool` definition in lines 256 to 265 of `include/sys-defines.h`; the pre-patched version already has it removed. Under *CLANG64* that definition is needed, so keep it when patching it yourself, or add it back at line 256 of the pre-patched version:

```
#define bool int
#define true 1
#define false 0
```

Then build and install it:

```
./configure --prefix=/ucrt64 --enable-static --disable-shared \
            LDFLAGS=-static
make
make install
```

Q.3.3 libFLAC 1.5.0

To build it statically, install the build tools and download the sources from [10]:

```
pacman -S autoconf automake pkg-config libtool
./autogen.sh
./configure --prefix=/ucrt64 --enable-static --disable-shared \
            --disable-ogg
make
make install
```

With the libraries installed, run `make` in the source folder.

Appendix R

Colors available for graphs

This is the list of colors that can be used in the *MFN configuration file* described in appendix C:

- red
- green
- blue
- yellow
- magenta
- cyan (also *aqua* or *aquamarine*)
- purple
- orange
- gray (also *grey*)
- pink
- lime
- violet
- sky
- salmon
- mint
- indigo
- emerald
- rose

- canary
- cobalt
- scarlet

If a profile uses a color that is not in this list, the error message lists the available ones.

Appendix S

Licensing

MDFourier Copyright (C)2019-2026 Artemio Urbina

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, see `https://www.gnu.org/licenses/`.

Appendix T

Contact the author

You can contact me via twitter `http://twitter.com/Artemio` or e-mail me at *aurbina@junkerhq.net*.

Appendix U

Acknowledgments

I'd like to thank the following people for helping me so this software and the present documentation could be completed.

First and foremost to Michelle, you helped me find the time to learn, code, experiment and write for this project. It would have been impossible without your support, thank you.

To Ricardo Pestaña (*Rhe7oRPG*) for guiding me through the unfamiliar process of giving shape and sense to this text. I am not good at writing, and this would not be close to its current form without your feedback and time.

Fernando Villaseñor for your patience and attention when discussing the ideas while I was shaping this project. Your comments gave me confidence that I could transmit the ideas I wanted to with some clarity, and pushed me forward in order to complete and publish the project.

Tianfeng for assisting me by proofreading the text and helping to make it readable. Having a native speaker check the documentation greatly reduced the awkwardness of my style, and helped the text flow better. Most importantly, thanks for taking the time and resources to create your own audio recordings, and to compare results when the project needed it.

Bernard Bygott for his enthusiasm, ability, critical analysis and bug finding skills. Your help has been crucial for this project. The time and resources you've poured to make this a better solution have been a driving force.

Robert Neal for your ability to create communities and your disposition to help others. The value of being able to reach out for feedback cannot be appreciated enough.

David from Plogue, who believed in the project and has pushed me to learn and improve it beyond what I could have done. Your input, use, advice and requirements have been invaluable while making this grow into what it has

become. I cannot imagine having a better use case than the great products you guys create.

donluca from the 16-bit Audiophile Project for creating the MacOS port, a lot of documentation, testing, idea sharing and many more contributions to MDFourier. Thank you my friend.

PinoBatch for making so many excellent versions of the Suite and MDfourier tone genrator, as well as always having input and precision. You also made the AudioEquipment test and the tone generators for it, which I could have never been able to do.

Ace for writing the overview of the Sega Genesis/MegaDrive models and then working to make MDfourier better with suggestions, comparisions, improvements and always valuable data.

Marc "Try4ce" Duddleson and Coury Carlson from *"My Life in Gaming"*, who indirectly seeded this project while they worked on a script. The feedback loop we've created helped me bounce ideas with you from time to time. Marc's feedback while proofreading allowed me to remove unnecessary complexity from the text, and your input made me notice the need of what became the *Audio Basics* chapter.

Enforcer for detecting a bug in the audio routines of the *240p Test Suite*, and making sure I didn't carry that on to the *MDFourier* test program. Thanks for reaching out.

To Oscar Toledo G. (*Nanochess*) for proofreading the document and pointing out some quirks about my writing style. This helped to improve the flow of the text.

Mauricio Lechuga, thank you for your time and attention to the project. our feedback helped me push in the right direction to create the current version of this document.

Aldo Martinez for listening and showing interest in the project. Talking about the ideas helps to shape them out clearly.

John Linneman thank you for detecting a bug in my audio program for the system and for using the early versions of the software for comparisons.

To Rolando Cedillo for listening to my thoughts and for letting me borrow an audio interface for testing. This helped me shape the results and confirm some facts that were very important for the validity of my results.

M.I.J.E.T. as always thanks for your well thoughts comments and ideas, they helped me to add a bit more rigor to my methods and process.

Byuu thanks for reading and believing in the possibilities this project may offer.

Special thanks to *Soltan_ G42* and *Rysha* for your enthusiasm and feedback,

it was very encouraging at a time the project was shaping out.

Extrems for asking that floating point frame rate be implemented for precision nd all your input, usage and extended support on using your tools for learning. Without mentioning patches, code examples and guidance.

And thanks to *Psikyo*, since repairing and playing their *Strikes 1945 PCB* helped me think about solutions and to clear my mind between coding and writing sessions.

Bibliography

[1] Bracewell, Ronald N. *The Fourier Transform and Its Applications (2 ed.)*. McGraw-Hill. ISBN 978-0-07303938-1.

[2] Ace, *GUIDE: A complete overview of the many Sega Genesis/MegaDrive models*, Sega-16, http://www.sega-16.com/forum/showthread.php?7796-GUIDE-Telling-apart-good-Genesis-1s-and-Genesis-2s-from-bad-ones

[3] The Best Version of the Genesis, RetroRGB https://www.retrorgb.com/genesisversions.html

[4] Sega Mega Drive/Hardware revisions, Sega Retro, https://segaretro.org/Sega_Mega_Drive/Hardware_revisions

[5] Jonas Rosland, *Artemio Urbina - Pioneering Tools to Keep Retro Gaming Alive*, Hit Save! interview, 2023. https://hitsave.org/artemio-urbina-pioneering-tools-to-keep-retro-gaming-alive/ Transcript: https://github.com/hitsave/interview-transcripts/tree/main/2023/Artemio%20Urbina

[6] Github, *MDFourier source code C99*, https://github.com/ArtemioUrbina/MDFourier/.

[7] 240p test Suite, *Wiki web page*, http://junkerhq.net/240p/.

[8] Fastest Fourier Transform in the West., *web page*, http://fftw.org/.

[9] libPNG, *web page*, http://www.libpng.org/pub/png/libpng.html

[10] libFLAC, *web page*, https://xiph.org/flac/, releases at https://ftp.osuosl.org/pub/xiph/releases/flac/

[11] GNU Plot Utils, *web page*, https://www.gnu.org/software/plotutils/, sources at https://ftp.gnu.org/gnu/plotutils/

[12] An implementation of a ton of sorting algorithms in C with a user-defined type, *web page*, https://github.com/swenson/sort/

[13] *Human Hearing: Amplitude Sensitivity*, Mark Sanfilipo — April 04, 2005, https://www.audioholics.com/room-acoustics/human-hearing-amplitude-sensitivity-part-1

[14] incbeta, *Incomplete Beta Function in C*, https://codeplea.com/incomplete-beta-function-c/.

[15] Window Types: Hanning, Flattop, Uniform, Tukey, and Exponential , *web page*, https://community.plm.automation.siemens.com/t5/Testing-Knowledge-Base/Window-Types-Hanning-Flattop-Uniform-Tukey-and-Exponential/ta-p/445063/.

[16] Decibel, suffixes and reference values, *Wikipedia*, https://en.wikipedia.org/wiki/Decibel#Suffixes_and_reference_values

[17] dB Full Scale, https://www.sweetwater.com/insync/dbfs/

[18] Sound Card Sampling clock variation http://www.stu2.net/wiki/index.php/Calibrate_Sound_Card/

[19] So how accurate is a typical PC sound card? How stable is the output? How would one measure this? Experiments with a PC sound card by leapsecond http://www.leapsecond.com/pages/sound-1pps/

[20] Each sound card has it's own sampling clock, Goldwave support forums http://www.goldwave.ca/forums/viewtopic.php?p=17470

[21] D/A and A/D | Digital Show and Tell (Monty Montgomery @ xiph.org) https://www.youtube.com/watch?v=cIQ9IXSUzuM

[22] Harris, Fredric j. (Jan 1978). *"On the use of Windows for Harmonic Analysis with the Discrete Fourier Transform"*. Proceedings of the IEEE. 66 (1): 51–83. https://web.mit.edu/xiphmont/Public/windows.pdf

[23] Spectral Leakage and Zero-Padding of the Discrete Fourier Transform https://dspillustrations.com/pages/posts/misc/spectral-leakage-zero-padding-and-frequency-resolution.html

[24] Why is it a bad idea to filter by zeroing out FFT bins? https://dsp.stackexchange.com/questions/6220/why-is-it-a-bad-idea-to-filter-by-zeroing-out-fft-bins

[25] MSYS2, *Software distribution and building platform for Windows*, https://www.msys2.org/.

[26] Homebrew, *The Missing Package Manager for macOS*, https://brew.sh/.

[27] Tigerbrew, *Homebrew fork for PowerPC Mac OS X*, https://github.com/mistydemeo/tigerbrew/.

[28] Compiling static binaries on macOS, https://donluca.theclassicgamer.net/compiling-static-binaries-on-macos/.

[29] plotutils-2.6 patched for current libpng, https://github.com/ArtemioUrbina/Plotutils.

[30] Cygwin, *a large collection of GNU and Open Source tools which provide functionality similar to a Linux distribution on Windows.* https://www.cygwin.com/

[31] Tukey window, Recording Blogs https://www.recordingblogs.com/wiki/tukey-window

[32] M-Audio Audiophile 192, *Specifications*, https://www.soundonsound.com/reviews/m-audio-audiophile-192/.

[33] Lexicon Alpha USB card, *Product web page*, https://lexiconpro.com/en/products/alpha/.

[34] MOTU M4, *web page*, https://motu.com/en-us/products/m-series/m4/.

[35] M-Audio M-Track USB card, *Product web page*, https://www.m-audio.com/products/view/m-track.

[36] Audacity *web page*, https://www.audacityteam.org/.

[37] Goldwave *product web page*, https://www.goldwave.com/.

[38] Adam Hayes, Simple Moving Average - SMA Definition https://www.investopedia.com/terms/s/sma.asp

[39] Human Speech Spectrum, Frequency Range, Formants, *web site*, http://www.bnoack.com/index.html?http&&&www.bnoack.com/audio/speech-level.html.

[40] Noise Level Chart, *web site*, https://www.noisehelp.com/noise-level-chart.html

[41] Audio Spectrum Explained *web site*, https://www.teachmeaudio.com/mixing/techniques/audio-spectrum/.